



FACULTY OF ENGINEERING, COMPUTING AND THE ENVIRONMENT

School of Engineering
Department of Mechanical Engineering
ME7743 Dissertation

MSc DEGREE IN *Mechatronic Systems*

Student name: Sarath Kumar Komathukattil

ID Number: K2454461

Project Title: Computer vision-based navigation of self-driving cars

Date: 7 January 2026

Supervisor: Dr Duran, Olga C

Kingston University London

WARRANTY STATEMENT

This is a student project. Therefore, neither the student nor Kingston University makes any warranty, express or implied, as to the accuracy of the data or conclusion of the work performed in the project and will not be held responsible for any consequences arising out of any inaccuracies or omissions therein.

Table of Contents

ABSTRACT	10
1. INTRODUCTION	12
1.1. Background and Motivation	12
1.2. Research Aim and Objectives	14
2. LITERATURE REVIEW	16
2.1. Introduction.....	16
2.2. Computer Vision for Autonomous Perception.....	16
2.3. Multi-Sensor Fusion for Robust Perception	24
2.4. Control Systems for Autonomous Navigation	24
2.5. Simulation Environments and the Reality Gap.....	25
2.6. Hardware Integration and Embedded Deployment	26
2.7. Critical Evaluation and Research Gaps.....	27
2.8. Conclusion.....	28
3. METHODOLOGY	29
3.1. Overview.....	29
3.2. Development Platforms	30
3.3. Dataset Preparation and Annotation	35
3.4. Model Training and Optimisation	39
3.5. Control System Design.....	46
3.6. System Integration Architecture	51
3.7. Testing and Validation Methodology.....	51
3.8. Summary	53
4. RESULTS	55
4.1. Object Detection Training Results	55
4.2. Lane Segmentation Training Results	63
4.3. Qualitative Detection Performance.....	67
4.4. Robustness Testing Results	71
4.5. Lane Following Performance.....	84

4.6. Chapter Summary	90
5. DISCUSSION.....	91
5.1. Interpretation of Key Findings	91
5.2. Simulation-to-Reality Transfer Analysis	93
5.3. Hardware Integration Challenges and Solutions.....	94
5.4. Practical Implications	96
5.5. Study Limitations	97
5.6. Summary	98
6. CONCLUSION.....	100
6.1. Research Summary	100
6.2. Key Contributions	100
6.3. Research Objectives Achievement.....	103
6.4. Future Research Directions	104
6.5. Closing Remarks	106
7. REFERENCES	108
8. APPENDIX	112
Appendix A: Key System Implementations	112
Appendix B: Unified Detector Training and Evaluation	118
Appendix C: Implementation Evidence and Demonstration Materials	120

List of Tables

Table 2.1: Comparison of Deep Learning Neural Network Architectures (Grigorescu et al., 2020; Carion et al., 2020).	18
Table 2.2: Comparison of Segmentation Architectures (Minaee et al., 2021).	21
Table 3.1: QLABs Virtual QCar Platform Specifications	31
Table 3.2: Hardware Unified Dataset Composition	37
Table 3.3: Data Augmentation Strategy for Lane Segmentation Training	39
Table 3.4: YOLOv8 Architecture Benchmarks (Ultralytics, 2023). Jetson TX2 benchmarks for model selection; final deployment used distributed laptop inference.....	39
Table 3.5: YOLOv8 Training Configuration for Simulation Baseline Models.....	40
Table 3.6: Model Architecture Evolution - Simulation (separate detectors) to Hardware (unified 5-class detector)	42
Table 3.7: Hardware Unified Model Training Configuration	42
Table 3.8: Classical Lane Detection Pipeline Configuration (QLabs Simulation)	43
Table 3.9: Architecture Specifications - U-Net vs Enet	44
Table 3.10: Performance Evaluation Metrics	52
Table 4.1: Simulation Training Results Comparison.....	60
Table 4.2: Sim-to-Real Performance Comparison	63
Table 4.3: U-Net vs ENet Architecture Comparison	67
Table 4.4: Traffic Light Detection - Occlusion Robustness (at 0.6m). Values represent YOLO confidence scores (0-1 range). "Failed" indicates no detection.	71
Table 4.5: Traffic Light Detection - Distance Robustness. Values represent YOLO confidence scores (0-1 range) at different distances from the traffic light.....	72
Table 4.6: Traffic Light Detection under lighting conditions tested at a fixed 0.6m distance. Values represent YOLO confidence scores (0-1 range)	73
Table 4.7: Traffic Sign Detection - Distance Robustness. Values represent YOLO confidence scores (0-1 range) at different distances.	74
Table 4.8: Stop Sign & No-Right-Turn - Occlusion Robustness tested at 0.3m distance. Values represent YOLO confidence scores (0-1 range).	74

Table 4.9: Stop Sign & No-Right-Turn - Lighting Conditions Lighting conditions tested at 0.3m distance. Values represent YOLO confidence scores (0-1 range)	75
Table 4.10: Lane Segmentation - Occlusion Robustness (Stationary Vehicle). Values shown as yellow lane pixels / Roundabout (white) pixels detected.	76
Table 4.11: Lane Segmentation performance values under different lighting Conditions.....	78
Table 4.12: Obstacle Detection (RealSense) – Distance & Size Performance." P" = Successfully detected, "X" = Failed detection. "Lighting Independent" indicates identical performance across Bright, Normal, Dim, and Dark conditions.	80
Table 4.13: Obstacle Detection (LiDAR) - Distance & Size Performance." P" = Successfully detected, " x " = Failed detection. "Lighting Independent" indicates identical performance across Bright, Normal, Dim, and Dark conditions.	81
Table 4.14: Combined Sensor Fusion (RealSense + LiDAR) – Distance & Size. "Both" = Both sensors detected (AND-logic trigger), "LiDAR Only" = Only LiDAR detected (no trigger), "RealSense Only" = Only RealSense detected (no trigger), "None" = Neither sensor detected.	82
Table 4.15: Lane Following Performance - U-Net vs ENet Speed Comparison.....	84
Table 4.16: U-Net Lighting Condition Performance at speed 0.060.	88
Table A.1: Sensor fusion decision truth table.	116

List of Figures

Figure 2.1: Approaches to self-driving. Classical modular pipeline (top) vs. monolithic end-to-end learning approach (bottom). (Janai et al., 2020).	17
Figure 2.2: YOLOv8 performance comparison showing superior parameter efficiency (left) and inference speed (right) compared to previous versions, enabling real-time embedded deployment (Ultralytics, 2023).	19
Figure 2.3: U-Net symmetric encoder-decoder with skip connections (Ronneberger et al., 2015).	23
Figure 2.4: ENet architecture modules. (a) Initial block with parallel convolution and pooling. (b) Bottleneck module with factorised convolutions. 355K parameters, 93.73% mIoU (Paszke et al., 2016).	23
Figure 2.5: Multi-sensor fusion architecture combining depth camera and LiDAR through synchronisation and memory modules for robust environmental mapping and obstacle detection (Jia et al., 2022).	24
Figure 2.6: Development lifecycle comparison. (a) Traditional real-world testing versus (b) simulation-based development with final hardware validation (Daza et al., 2023).	26
Figure 3.1: Development pipeline showing progression from simulation to hardware deployment across five stages.	29
Figure 3.2 illustrates the QLab's virtual environment used for baseline development, showing the simulated QCar navigating a road network with traffic infrastructure....	31
Figure 3.3 shows the QCar platform with the integrated sensor suite used for hardware validation and real-world testing.	32
Figure 3.4: Quanser QCar technical specifications showing sensor placement, computing module, and I/O interfaces (Quanser, n.d.a).	33
Figure 3.5: Distributed system architecture showing laptop-based perception processing (left) with multi-stream reception, deep learning inference, and sensor fusion, and QCar-based control execution (right) with sensor acquisition and motor control, connected via TCP/IP over WiFi 802.11n.	34
Figure 3.6: Simulation training data examples showing (a) traffic light detection (1,288 instances, 2 classes) and (b) stop sign detection (496 instances, 1 class) with YOLO bounding box annotations from QLab's virtual environment.	36
Figure 3.7: Hardware unified dataset training samples (3,802 instances, 5 classes) showing real-world environmental diversity, including varied lighting, indoor/outdoor	

scenes, motion blur, and challenging backgrounds with YOLO bounding box annotations.....	38
Figure 3.8: U-Net architecture with symmetric encoder-decoder design and skip connections. 31M parameters, 121 MB model size, 256×256 input/output, 3-class segmentation (background, yellow lanes, white lanes).	44
Figure 3.9: ENet Architecture Overview	45
Figure 3.10: AND-based sensor fusion logic requiring both RealSense depth camera and RPLidar agreement before triggering emergency braking.....	50
Figure 4.1: Traffic light detection training curves over 100 epochs showing loss convergence and performance metrics	56
Figure 4.2: Normalised confusion matrix for traffic light detection (red: 84%, green: 100%).....	57
Figure 4.3: Stop sign detection training curves over 82 epochs demonstrating rapid convergence.....	58
Figure 4.4: Normalised confusion matrix for stop sign detection (single-class model)	59
Figure 4.5: Unified 5-class detection model training progression over 100 epochs on real QCar hardware dataset	61
Figure 4.6: Normalised confusion matrix for unified 5-class detection model on real QCar hardware (validation set)	62
Figure 4.7 shows a red-light detection at 0.74 confidence score with the QCar approaching an intersection.	68
Figure 4.8: Stop sign detection in QLab simulation at confidence scores: 0.88 and 0.90, respectively.....	68
Figure 4.9 demonstrates straight road detection with 3046 yellow and 21958 white pixels.	69
Figure 4.10: Real-world green and red traffic light detection on physical QCar (confidence scores: 0.74 & 0.54, respectively) with integrated lane segmentation ..	70
Figure 4.11: Real-world stop sign (confidence: 0.68), no-right-turn sign detection (confidence scores: 0.85) on physical QCar, demonstrating robust recognition in an operational environment.....	70
Figure 4.12: Traffic light detection under progressive occlusion (25% and 75% at 0.6m distance).....	72

Figure 4.13: Traffic stop sign detection under progressive occlusion (50% and 75% at 0.3m distance).....	74
Figure 4.14: Traffic No-right-turn sign detection under progressive occlusion (25% and 50% at 0.3m distance).....	75
Figure 4.15: Traffic sign detection under dark lighting conditions (stop: 0.59, no-turn: 0.84) at 0.3m distance.....	76
Figure 4.16: U-Net Lane segmentation at 25% occlusion (left: segmentation output, right: camera view).....	77
Figure 4.17: Enet Lane segmentation at 90% occlusion (left: segmentation output, right: camera view).....	77
Figure 4.18: U-Net segmentation performance across bright and dark lighting conditions.....	78
Figure 4.19: Enet segmentation performance across bright and dark lighting conditions.....	79
Figure 4.20: RealSense depth camera successful obstacle detection at 0.4m maximum effective range (large object, active IR sensing).....	80
Figure 4.21: RealSense detection failure at 0.6m distance (same large object), demonstrating sharp performance cutoff beyond effective operating range.....	80
Figure 4.22: LiDAR obstacle detection of a medium sized image at 0.6m distance, demonstrating superior range compared to RealSense 0.4m maximum.....	81
Figure 4.23: LiDAR obstacle detection of a medium sized image at 1.2m tested distance showing 3× superior range compared to RealSense (0.4m).....	82
Figure 4.24: Sensor fusion at 0.2m showing successful dual-sensor detection of a small sized image with emergency brake trigger (left: camera, centre: RealSense depth, right: LiDAR 0.26m detection).....	83
Figure 4.25: Sensor fusion at 0.4m (small object) at RealSense maximum range, demonstrating consensus zone limit for emergency braking (left: camera, centre: RealSense depth, right: LiDAR 0.47m detection).....	83
Figure 4.26: U-Net Lane following performance across throttle settings (0.055-0.070) showing optimal speed 0.060 with 2.62m distance, 108px mean offset, perfect yellow detection (100%) and zero boundary violations.....	85
Figure 4.27: U-Net trajectory comparison across throttle settings, showing speed 0.060 (green) maintaining drivable lane boundaries for 2.62m with zero violations, versus degraded performance at 0.065 (148 violations) and failure at 0.070.....	85

Figure 4.28: ENet lane following performance across throttle settings, demonstrating superior detection robustness with 100% yellow lane detection maintained through speed 0.065 (vs U-Net's 98.9%) and optimal 2.09m distance at 0.060.	86
Figure 4.29: ENet trajectory comparison across throttle settings showing superior mid-speed robustness at 0.065 (58 violations vs U-Net's 148, 61% reduction), demonstrating efficient architecture advantages despite 25% shorter optimal distance.	87
Figure 4.30: U-Net Lane following performance across lighting conditions showing perfect yellow lane detection (100%) under Normal/Bright/Dim with complete dark failure (0m, 69.8% detection).	88
Figure 4.31: U-Net trajectory comparison across lighting conditions demonstrating <5% variation in Normal/Bright/Dim (2.50-2.62m) with complete dark failure (0m travelled).	89
Figure A.1: BasicStream client-server configuration for remote frame streaming...	112
Figure A.2: RealSense depth obstacle detection with ROI filtering.	114
Figure A.3: LiDAR obstacle detection with persistence filtering.....	115
Figure A.4: AND-logic sensor fusion decision.....	116
Figure B.1: YOLOv8 confidence threshold analysis. (a) F1-confidence shows optimal operating point at 0.91 F1 score (confidence 0.191). (b) Precision-confidence achieves 1.00 precision at high confidence (0.983). The model demonstrates well-calibrated confidence scores across all traffic classes.....	118
Figure B.2: YOLOv8 detection performance. (a) Recall-confidence maintains 0.98 recall at zero threshold. (b) Precision-recall curve reveals 92.87% mAP@0.5 (final validation) with per-class AP: stop sign 99.4%, green light 96.3%, no right turn 93.1%, demonstrating strong class- specific detection.....	118

ABSTRACT

Research on autonomous driving requires extensive real-world validation, but large-scale platforms are expensive, real-world testing poses safety risks, and replicating real-world conditions in a research setting becomes difficult. Simulation provides a cost-effective way to run simulations rapidly, safely, and repeatably, but models of perception and control that work well in a simulated environment tend to degrade when implemented on real-world hardware, a problem that has not been well characterised on educational platforms.

This dissertation addresses this need by developing a vision-based autonomous navigation system for the Quanser QCar 1:10-scale platform and measuring simulation-to-reality transfer using QLab. A two-stage methodology was implemented with explicit performance comparison between simulation and hardware. In the simulation, lane following used HSV-based detection, and traffic perception used YOLOv8. For hardware deployment, a distributed architecture was adopted: deep learning inference (ENet segmentation and unified YOLOv8) executed on a laptop via TCP/IP, while an NVIDIA Jetson TX2 handled sensor I/O and actuation. Multi-sensor fusion combined RealSense D435 and RPLidar A2 with AND-logic obstacle confirmation.

Results quantify the reality gap. Simulation achieved 99.44% mAP@0.5 for traffic lights and 99.50% for stop signs, while hardware achieved 92.87% mAP@0.5, resulting in a 6.57 percentage-point reduction. ENet achieved 93.73% mIoU for lane segmentation, and sensor fusion produced zero false positives. The distributed pipeline operated at 18–22 FPS with TCP/IP latency below 50ms, meeting real-time requirements. Key integration challenges, including CSI camera–RealSense conflicts and embedded resource constraints, were resolved through initialisation sequencing and efficient architecture choices. This work provides systematic sim-real metrics and a replicable framework for autonomous navigation on educational platforms.

ACKNOWLEDGEMENTS

First and foremost, I thank God Almighty for His blessings, guidance, and strength throughout this journey. Without His grace, this achievement would not have been possible.

I am deeply grateful to my supervisors, Dr Olga, for her invaluable guidance and support throughout this research. It has been a privilege to work under her supervision.

I would also like to thank Kingston University for providing access to the facilities and resources essential for this research. The environment and support have been invaluable throughout my studies.

I am deeply thankful to my family for their unconditional love, support, and encouragement. Their understanding and belief in me have been a constant source of strength and motivation. Also, I am grateful to my friends and colleagues for their encouragement, valuable discussions, and support, which have made this journey more enjoyable and memorable.

Finally, I acknowledge and deeply appreciate everyone who contributed directly or indirectly to the completion of this dissertation.

1. INTRODUCTION

1.1. Background and Motivation

Camera-based autonomous navigation is a cost-effective way to give robots mobility by providing a stream of rich visual information about their surroundings. Recent advances in Deep Learning have revolutionised the state of the art in autonomous vehicles. CNNs can detect vehicles, pedestrians, road signs, and lane markings in real time with unprecedented accuracy. One-stage detectors, specifically YOLO, run at 20-30 FPS on an Embedded GPU, while semantic segmentation networks provide pixel-wise classification of drivable areas and lanes (Janai et al., 2020).

There are three essential tasks autonomous cars must excel at: they must perceive the environment around them using sensors (perception), predict how this environment will evolve over time (prediction), and decide on safe, optimal paths to follow using this knowledge (planning and control). This requires an imaging stack using complementary views from cameras, LiDAR, and ultrasonic sensors. Of these, computer vision has been remarkable for its level and cost-effectiveness. Vision alone may struggle to adapt to changing lighting conditions and adverse weather, but this has driven smart learning and judicious ways to combine different modalities into a single system (Janai et al., 2020; Badue et al., 2021).

Systems are often modular, where perception, prediction, planning, and control stages are separated for interpretability and real-time tuning. Alternatively, end-to-end learning approaches map images directly to vehicle commands, though these typically require larger datasets and more computational resources. Many practical implementations combine both approaches. Cameras are often combined with IMU, wheel encoders, or depth/LiDAR sensors to cope with difficult lighting and occlusions. Performance is normally judged not only by detector accuracy but also through closed-loop measures such as route completion, collisions or infractions, lateral tracking error, and processing latency (Badue et al., 2021).

However, autonomous vehicles face a major real-world challenge in their research: testing them on the road is expensive and challenging. This is even more challenging in varying weather and lighting conditions. If the algorithms developed posed risks to human life, it would be unwise to continue experimenting, and this issue directly affects

the development environment of autonomous vehicles. There would be large developments in simulators before further hardware tests (Daza et al., 2023)

Simulation environments promote rapid prototyping, risk-free testing, and repeatable testing. The QLABS Virtual QCar offered by Quanser is a simulation tool specifically designed for the QCar platform, with identical sensing Interfaces and code compatibility that allows it to run Python code in simulation as well as on hardware. Simulation features repeatable scenario testing with controllable conditions, synchronised dataset creation with automated annotation, and risk-free testing of potentially unstable algorithms (Quanser, n.d.b).

However, simulation also poses a problem of its own: the “reality gap,” or the loss of system performance as a simulated system is migrated from an idealised simulation environment to real hardware with noise in sensors, lighting variation, motion blur, timing constraints, and resource limitations. The reality gap is a well-recognised but inadequately measured problem. Much of the available research has presented results from either simulation alone, awaiting hardware validation in follow-up work, or hardware validation alone, with simulation data excluded from the comparative analysis. It has recently been proposed that it is necessary to directly model and evaluate the reality gap in terms of the amount of system performance lost going from simulators to hardware, because simulators cannot be directly repeatedly validated (Zhao et al., 2020; Daza et al., 2023).

This dissertation presents a simulation-to-reality challenge by systematically developing and validating a vision-based autonomous navigation system on the Quanser QCar platform. The work progresses from simulation development in QLABS through hardware deployment on a physical QCar, explicitly measuring performance differences, documenting integration challenges, and validating practical solutions. Unlike simulation studies that only simulate hardware transferability or hardware demos that do not involve methodology during the course of development, the contributions to the proposed work involve documenting the entire pipeline including the establishment of the baseline in simulation, the integration of hardware with documented challenges and solutions, the comparison of the performances on the

same scenarios, and the analysis of the reality gap for the respective system components.

1.2. Research Aim and Objectives

Project Aim

To develop and validate a real-time vision-based autonomous navigation system on QLab simulation and QCar hardware that detects traffic elements, follows lanes, avoids obstacles, and maintains stable closed-loop control, while quantifying sim-to-real performance through comprehensive metrics and documented workflows.

Objectives

O1. Dataset Preparation

To enable model training by collecting datasets from QLab simulation and QCar hardware, annotating traffic elements and lane markings, and applying augmentation for improved generalisation.

O2. Model Training

To achieve accurate perception by training YOLOv8 for traffic detection and comparing U-Net/ENet for lane segmentation, with systematic hyperparameter tuning and sensor calibration.

O3. Perception and Control System Development

To enable autonomous navigation by integrating YOLO detection, semantic segmentation, AND-logic sensor fusion with robustness evaluation, and Pure Pursuit control with Kalman filtering.

O4. Simulation Validation

To establish performance baselines by deploying in QLab, measuring detection accuracy, lane tracking error, processing latency, and documenting protocols for repeatability.

O5. Hardware Deployment and Sim-to-Real Analysis

To quantify the reality gap by deploying on QCar, resolving integration challenges through documented root-cause analysis, and providing a replicable framework for educational platforms.

2. LITERATURE REVIEW

2.1. Introduction

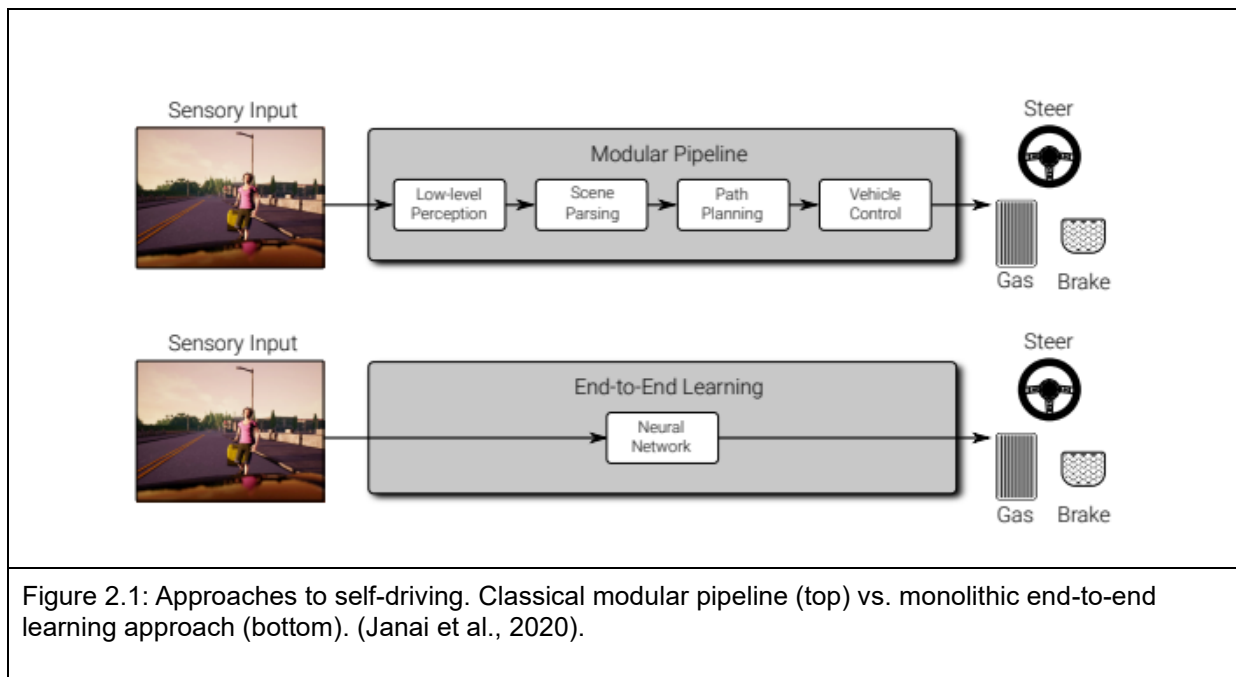
Autonomous vehicle navigation depends on the integration of computer vision, deep learning, sensor fusion, and control systems, ideally operating in real time on resource-constrained platforms. However, there is a substantial difference between the simulations carried out on computer algorithms and actual implementation on real platforms, especially those related to educational platforms such as those found on the Quanser QCar. This literature review discusses research on vision-based autonomous navigation systems for educational platforms, covering perception modules, sensor fusion techniques, control techniques, and reality transfer methods for resource-constrained autonomous systems.

2.2. Computer Vision for Autonomous Perception

Autonomous perception relies on sensor data for the extraction of scene information, including object detection (cars, pedestrians, traffic signs), boundary detection (lanes, road areas), and distance detection. Traditionally, techniques began with computer vision-based feature design, which has advanced to deep learning models based on scene representations learned through data (Janai et al., 2020).

Classical Computer Vision and Limitations

Classical approaches handle images in deterministic processing chains: preprocessing to remove noise, colour masking in the HSV colour space to select the lane markings, edge extraction using Canny operators, line extraction in the Hough transformation, and Kalman filters to maintain consistency in the sequence of images. Despite their interpretability and computational efficiency, these approaches are brittle, sensitive to lighting, limited in generalisation, and require manual threshold tuning (Janai et al., 2020).



Deep Learning and Object Detection

Modern autonomous perception uses deep neural networks that learn useful visual features directly from data. Models are typically pretrained on large datasets, then fine-tuned on project data to match the operating domain and exported to efficient runtimes for embedded execution.

Neural Network Architectures

CNNs (Convolutional Neural Networks)

CNNs learn visual features with stacked convolutions (edges → textures → parts → objects). They are fast, efficient, and naturally translation aware. Most classic and real-time detectors are CNN-based (e.g., Faster R-CNN, YOLO, RetinaNet). They map cleanly to GPUs/NPUs and prune/quantise well, making them straightforward to deploy in real-time systems (Grigorescu et al., 2020).

RNNs (Recurrent Neural Networks)

RNNs (LSTM/GRU) model sequence information. In vision, they are rarely the core image encoder; instead, they sit on top of frame-wise features to smooth predictions

over time, improve tracking, or predict motion/intent. Useful for temporal consistency, but not the primary engine for per-frame object detection (Grigorescu et al., 2020).

Transformers

Transformers use self-attention to relate any region to any other, capturing long-range context and complex layouts. In detection, they appear as: (1) end-to-end heads (DETR, Deformable DETR) that replace hand-crafted anchors/NMS, and (2) transformer backbones or hybrid CNN-Transformer models. They can reach excellent accuracy and simpler pipelines but often need more data/compute and careful optimisation to hit real-time on small devices (Carion et al., 2020).

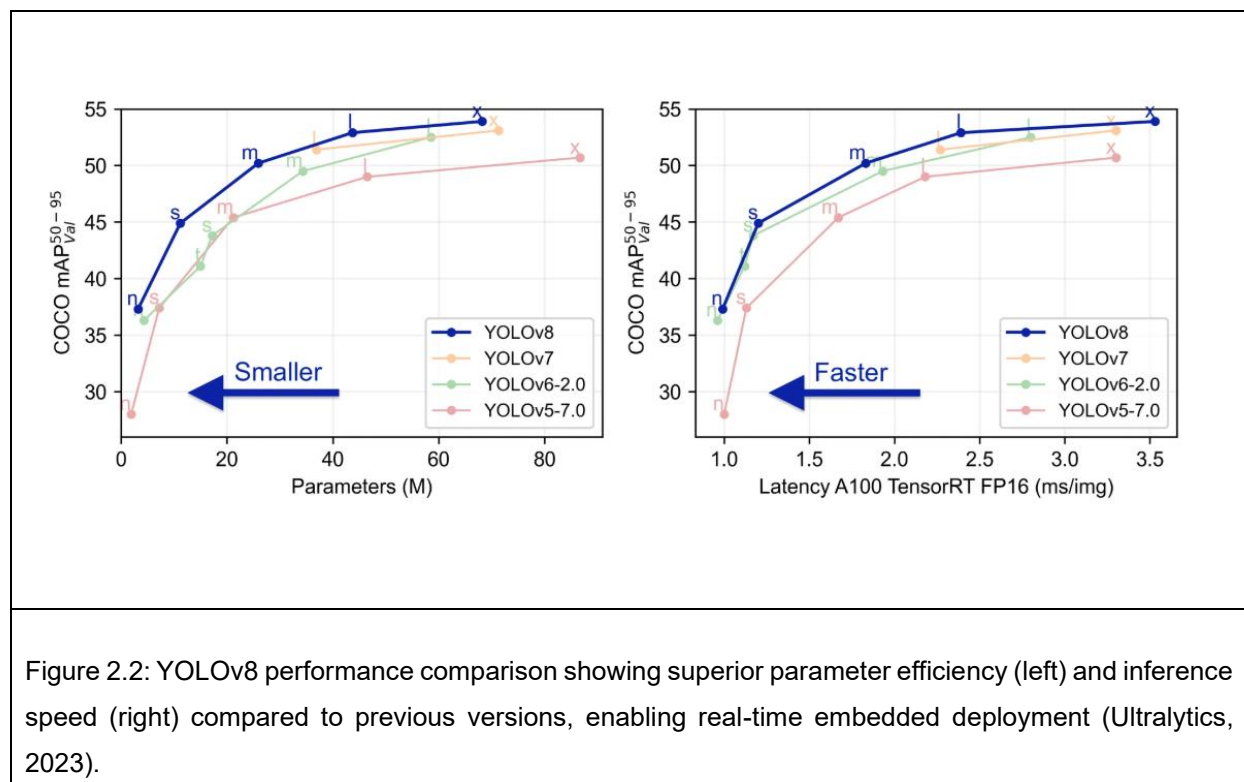
Criterion	CNNs	RNNs	Transformers
Typical role	Real-time detection	Temporal smoothing	Global context modelling
Deployment	Simple	Moderate	Complex
Pretrained models	Extensive	Limited	Growing
Context scope	Local spatial	Sequential	Global spatial
Embedded feasibility	High	Auxiliary	Challenging

Table 2.1: Comparison of Deep Learning Neural Network Architectures (Grigorescu et al., 2020; Carion et al., 2020).

For object detection, CNNs are the most practical: they are fast, lightweight, easy to deploy (e.g., YOLO/RetinaNet), and have plenty of pretrained options, though very small CNNs can miss tiny or heavily occluded objects. RNNs are not primary detectors; they add temporal memory on top of detections to smooth jitter and handle short occlusions, but they increase complexity and do not replace a strong per-frame model. Transformers capture global context and can reach higher accuracy with simpler end-to-end heads (e.g., DETR), yet they usually need more data/compute and are harder to run at high FPS on small devices. For QCar, CNNs are preferred: fast, lightweight, easy to deploy, with extensive pretrained options (Grigorescu et al., 2020; Carion et al., 2020).

Object detectors are commonly grouped into two-stage and one-stage families. Two-stage detectors generate region proposals and then refine localisation and classification, which can provide high accuracy but typically increases computational cost and inference latency on embedded platforms (Zou et al., 2023). One-stage detectors predict bounding boxes and classes in a single forward pass, making them more suitable for real-time control loops where consistent throughput is essential.

YOLOv8 was selected because it provides a strong speed–accuracy trade-off for real-time navigation while remaining practical to integrate into a Python-based robotics pipeline. First, its single-stage design supports low-latency perception, which is critical for closed-loop steering and traffic-rule compliance. Second, the Ultralytics framework provides pretrained weights, a simple training workflow, and straightforward deployment/export options, reducing engineering overhead during simulation-to-hardware transfer (Ultralytics, 2023). Third, YOLO-style detectors have demonstrated real-time performance on Jetson-class embedded hardware, aligning with the deployment constraints of this work (Behrendt & Novak, 2017). Together, these factors make YOLOv8 a suitable choice for the Qcar platform, where reliable inference speed and integration simplicity are as important as detection accuracy.



Classical Vision Challenges and DL based Mitigations

Classical computer vision built on hand-crafted features and geometric rules (edges, corners, thresholds, template matching) is brittle: changes in lighting, weather, texture, motion blur, glare, shadows, and partial occlusions often break detection and lane extraction, while geometry modules like SLAM are sensitive to calibration and time-sync errors, leading to drift and unstable tracks that hurt downstream planning and control (Yurtsever et al., 2020). Deep learning mitigates these issues by learning features directly from data, making perception more tolerant to appearance changes; one-stage and transformer-based detectors improve object recognition in clutter, encoder-decoder networks yield stronger lane/drivable-area masks than edge/threshold methods, and learned depth helps when classic matching fails and also with standard acceleration toolchains, these models can run in real time on embedded GPUs and provide planner-friendly outputs that are more reliable than brittle hand-crafted pipelines (Minaee et al., 2021; Yurtsever et al., 2020).

Embedded Platform Deployment

Embedded platform model deployments come with unique considerations. The Jetson TX2 offers much lower computational capabilities than desktop GPUs. The performance is 1.3 TFLOPS (FP16), compared with 11+ TFLOPS (Chen & Ran, 2019), which significantly worsens inference times on embedded platforms. The optimisation methods include architecture (suitably scaled YOLO models), TensorRT optimisations that achieve 2-4 times faster speeds (Wong et al., 2019), lower-precision inference (FP16 or INT8), or simply decreasing input resolution. The current scope of work addresses the gap in concurrent execution of multiple models (detection, segmentation, depth estimation) on the TX2 platform.

Semantic Segmentation for Lane Detection

Semantic segmentation classifies each pixel into categories such as lanes, drivable surface, and background. Traditional techniques using colour thresholding and Hough transforms are brittle under varying conditions (MathWorks, n.d.). Deep learning

segmentation learns optimal representations that adapt to appearance variation (Minaee et al., 2021).

Segmentation is dominated by encoder-decoder architectures: encoders downsample features while extracting them while decoders upsample them, to recover the spatial details. Skip connections preserve fine-grained information lost during downsampling (Minaee et al., 2021).

Architecture	Parameters	Approach	Strengths	Limitations
U-Net	~31M	Skip connections	Precise localisation	Slow inference, high memory
ENet	~355K	Factorised convolutions	Real-time capable	Reduced accuracy on complex scenes
ERFNet	~2M	Residual factorised blocks	Speed-accuracy balance	Moderate complexity
SegNet	~29M	Pooling indices reuse	Memory-efficient decoding	Moderate accuracy

Table 2.2: Comparison of Segmentation Architectures (Minaee et al., 2021).

U-Net

U-Net (Ronneberger et al., 2015) employs a symmetric encoder-decoder with skip connections that concatenate encoder features directly to decoder layers. This design preserves fine spatial details, making it effective for tasks requiring precise boundary delineation, such as lane detection. However, the U-Net has a large parameter count of more than 31 million, which challenges real-time embedded inference.

ENet

ENet (Paszke et al., 2016) focuses on efficiency, addressing it via an asymmetric architecture, factorised convolutions, and aggressive downsampling, achieving real-time speed with only 355K parameters, 87 times fewer than U-Net. ENet targets real-

time embedded applications while maintaining competitive accuracy on structured scenes such as road segmentation.

ERFNet

ERFNet employs residual connections with factorised convolutions, decomposing standard convolutions into 1D horizontal and vertical operations. This design reduces computational cost while maintaining accuracy, achieving a balance between the U-Net's precision and the ENet's efficiency. With approximately 2M parameters, ERFNet targets real-time intelligent transportation applications but introduces moderate architectural complexity (Minaee et al., 2021).

SegNet

SegNet stores pooling indices during encoding and reuses them for upsampling, thereby reducing memory requirements compared to full-feature-map storage. While effective for general scene understanding, its 29M parameters and moderate segmentation accuracy make it less suitable for resource-constrained embedded platforms requiring both speed and precision (Minaee et al., 2021).

Rationale for U-Net and ENet Selection

U-Net and ENet represent two ends of the accuracy-efficiency spectrum, making them suitable candidates for comparative evaluation on embedded platforms:

1. Accuracy baseline: U-Net provides a high-accuracy reference due to its skip connections, which preserve spatial detail, enabling assessment of the segmentation quality achievable with sufficient resources.
2. Efficiency baseline: ENet demonstrates that comparable accuracy is achievable with drastically reduced parameter count (87× fewer), critical for embedded deployment where memory and inference speed are constrained.
3. Structured scene suitability: Both architectures are well-suited to road scenes with geometric regularity. ENet's early downsampling and efficient bottleneck design support real-time segmentation, while U-Net's skip connections preserve boundary detail for precise localisation (Paszke et al., 2016; Ronneberger et al., 2015).

4. Deployment practicality: ENet's compact size enables concurrent execution with object detection models on resource-limited hardware, while U-Net serves as an upper-bound accuracy benchmark (Minaee et al., 2021).

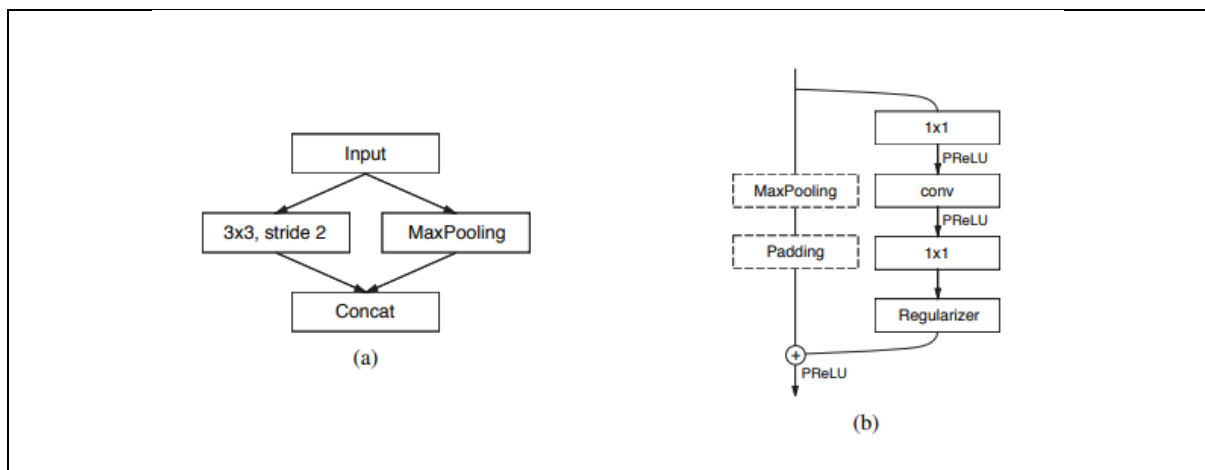
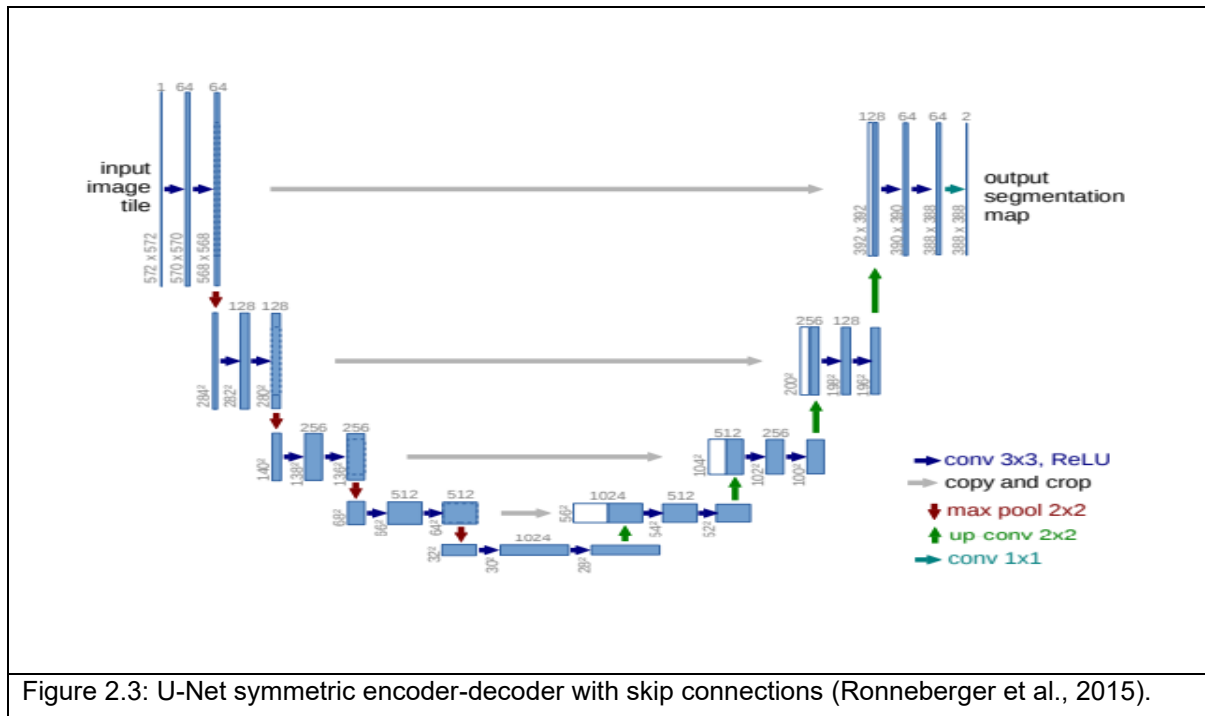
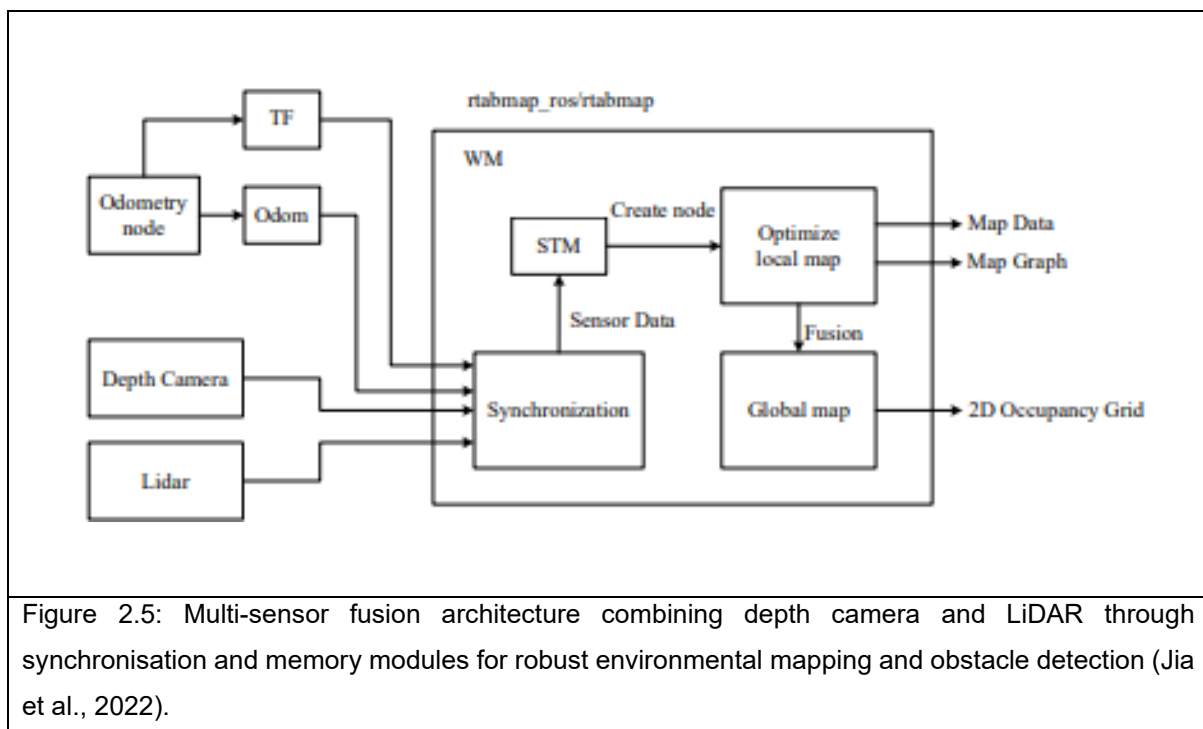


Figure 2.4: ENet architecture modules. (a) Initial block with parallel convolution and pooling. (b) Bottleneck module with factorised convolutions. 355K parameters, 93.73% mIoU (Paszke et al., 2016).

2.3. Multi-Sensor Fusion for Robust Perception

Multi-modal perception with complementary sensors that overcome these individual sensor limitations. While cameras generally provide rich semantic information, their capabilities for precise distance measurement are limited. Depth sensors, on the other hand, provide rather accurate near-range measurements but fail with many material types and have a very limited range. LiDAR provides reliable extended ranging with sparse sampling. Therefore, combining modalities would enhance robustness (Yeong et al., 2021).

Fusion methods range from early fusion (raw sensor data) to late fusion (decision-level), balancing computational efficiency against reasoning capability. Most research focuses on automotive sensors using sophisticated methods such as Extended Kalman Filters and Particle Filters. Simple approaches based on AND/OR logic for embedded educational platforms remain poorly validated (Yeong et al., 2021).



2.4. Control Systems for Autonomous Navigation

Path tracking involves translating perception outputs into actuator commands while satisfying the system's dynamical model. A geometric control method, Pure Pursuit, uses look-ahead points along the reference trajectories to calculate the control command, the steering angle. The advantage of using Pure Pursuit is the simple need

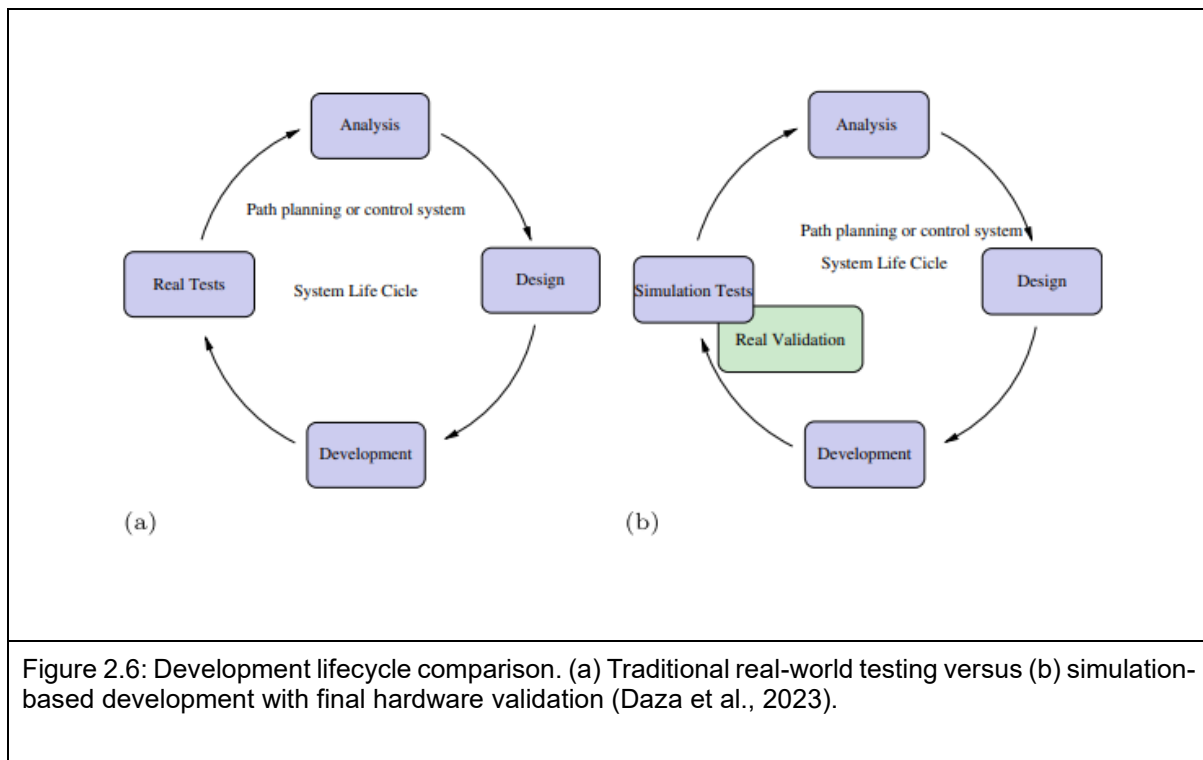
for position, geometric path information, and a look-ahead distance, which ensures a stable Ackermann-steered car (Rokonuzzaman et al., 2021).

Model Predictive Control provides optimal control while satisfying constraints, but requires computations that cannot be performed by an embedded system. For educational platforms running at low speed and in structured environment settings, Pure Pursuit provides optimal results in terms of complexity. Alongside Kalman filtering for state estimation and smoothing steering, this strategy provides optimal results in terms of efficiency, simplicity, and effectiveness (Rokonuzzaman et al., 2021).

2.5. Simulation Environments and the Reality Gap

High-fidelity simulators such as CARLA (Dosovitskiy et al., 2017) support photorealistic images and diverse scenarios but are meant for full-size cars. For a platform such as 'QCars,' Quanser's 'QLabs' supports physics-based simulators with identical sensor interfaces, so that code written in the simulators can be directly used with hardware (Quanser, n.d.b).

The common issue is that transferring from simulation to hardware usually results in performance degradation, so-called "reality gap." Zhao et al. (2020) attribute this to inaccurate physics, simplified sensor models, and visual domain shift. Visions are particularly sensitive because of camera noise, lighting variation, and appearance differences. Peng et al. (2018) reported that even for the most photo-realistic simulators, performance would degrade by 5 to 10 percentage points on real robots.



Daza et al. (2023) illustrate how explicit reality gap modelling supports robust design, albeit with a focus on full-scale systems. When it comes to QCar, there is no measurement of perception degradation and latency increase, with a focus on integration, and most of these studies fail to offer a direct simulation-to-hardware comparison.

2.6. Hardware Integration and Embedded Deployment

Deploying deep learning on embedded platforms presents distinct challenges: limited memory, computational constraints, real-time requirements, and thermal management.

The TX2 in QCar provides significantly fewer resources than desktop GPUs: 256 CUDA cores vs. 2,000+ CUDA Cores, 8GB of shared vs. 24GB of dedicated memory, and 1.3 TFLOPS vs. 10+ TFLOPS. Wong et al. (2019) have shown that there is a 2-4× speedup using TensorRT with an accuracy loss of less than 1%. However, documenting the parallel execution of multiple models on TX2 remains sparse.

The many sensors pose practical problems that are rarely mentioned: USB bandwidth limitations, driver conflicts, and timing synchronisation. Again, for resource-

constrained platforms, the practical solution is a distributed architecture separating perception from actuation, introducing 10-50ms of additional latency in communication. Further, the gap between algorithm design and its integration into hardware creates barriers for researchers transitioning from simulation to working systems.

2.7. Critical Evaluation and Research Gaps

Most autonomous vehicle research presents only simulation results, deferring hardware validation to 'future work'. While simulation enables fast experimentation, idealised sensor models and controlled environments produce optimistic bounds rather than realistic expectations (Zhao et al., 2020).

Publications presenting real-world results often lack simulation baseline comparisons. Common omissions include hardware integration challenges, quantitative performance degradation, resource utilisation, and failed approaches—hindering reproducibility.

A vast majority of studies are conducted on the whole vehicle or on advanced robots. Robotic platforms such as QCar are underrepresented in the literature that has adopted them, leaving graduates without appropriate approaches, evaluation standards, and repair strategies. Research platforms with specialised components and hefty budgets versus improved approaches on cost-effective platforms are still underexplored.

This study explores five research gaps.

Gap 1: Quantified Sim-to-Real Transfer—A lack of a comprehensive quantitative study of the transfer process between Q Labs and Q Cars. How much degradation in perception is to be expected? To what extent are the latencies elevated on embedded platforms?

Gap 2: Documented Hardware Integration—Hardware-related deployment issues, such as sensor conflicts, USB bandwidth constraints, or the need for optimisation, are rarely documented. There will be a need to provide solutions to these issues through root cause analysis.

Gap 3: Multi-Sensor Fusion on Constrained Platforms—Although multi-sensor fusion is a mature research area, the usefulness of methods such as AND-logic fusion on educational embedded systems has yet to be established. Simulations versus hardware experiments: how are computing costs involved?

Gap 4: QCar/QLabs Platform Validation—Although QCar/QLabs is well adopted within education circles, there is a lack of performance metrics, validated procedures, and troubleshooting

Gap 5: Documentation of the Whole Pipeline—The detailing in published works encompassing challenges, failures, iterations, and learnings is very limited.

This work fills gaps in the quantitative QLABs and QCar comparison, hardware integration with root cause analysis, AND-logic fusion validation on hardware, QCar methods validation with performance analysis, and a complete pipeline description, including problems and iterations.

2.8. Conclusion

This literature review provided the technical foundation for vision-based autonomous navigation systems and identified key research gaps. Studies based solely on simulation, a lack of documentation of practical system integration, and the absence of validation of the education platform are current obstacles in developing autonomous systems in education.

The move from simulation development to embedded hardware integration is a challenge that has not been adequately treated in the literature. This paper offers a gap-filled contribution to the educational autonomous systems literature by documenting the quantitative sim-to-real process, a full list of integration challenges, and effective solutions for the transition process at QCar/QLabs.

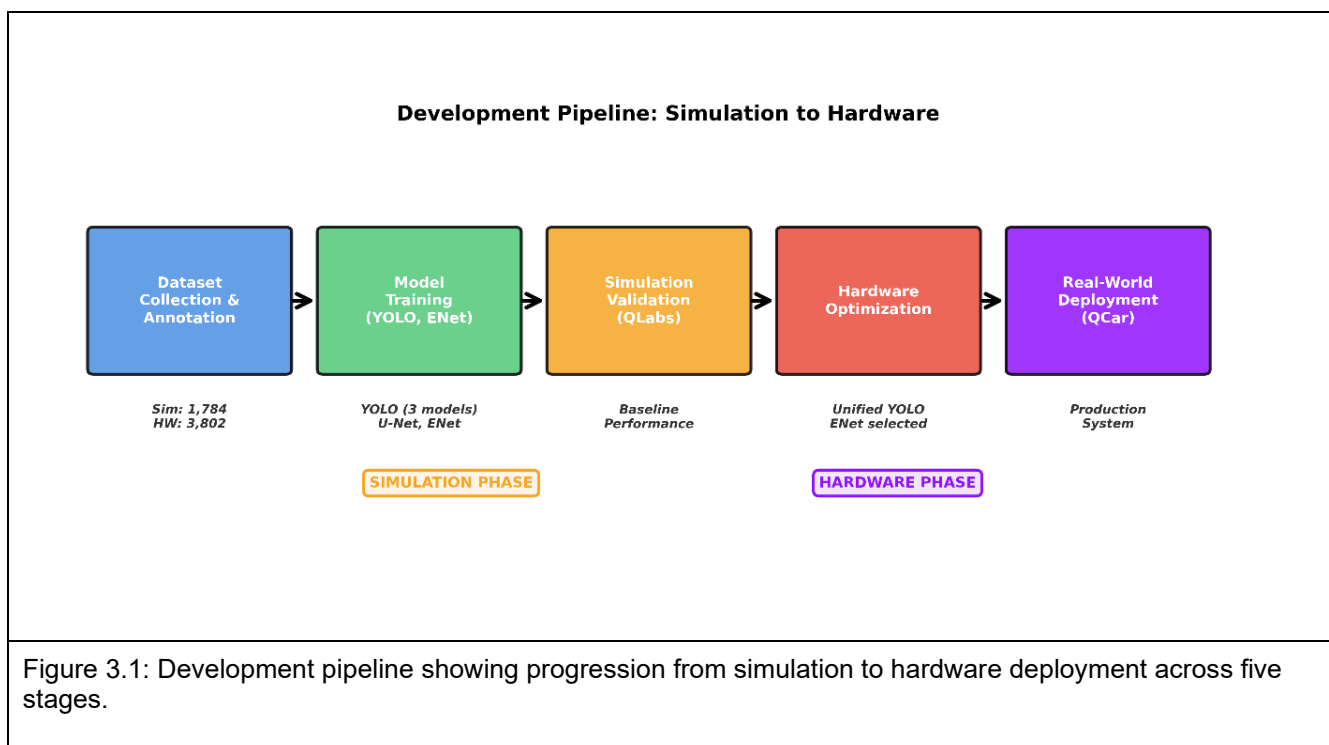
The next chapter explains the methodology used to fill all these knowledge gaps, covering the entire system development phase, from simulation to hardware implementation and, finally, validation.

3. METHODOLOGY

Overview

This chapter describes how the autonomous navigation system was developed for the Quanser QCar platform, from QLab simulation through to hardware deployment and validation.

The system combines three perception components: YOLOv8 for traffic element detection, E-net/Unet for lane-following segmentation, and sensor fusion using an Intel RealSense depth camera and an RPLidar for obstacle detection. Also, Pure Pursuit tracking with Kalman filtering handles vehicle control.



3.1.1 Development Pipeline

This development process was divided into five successive steps: establishing a classical baseline with HSV-based vision algorithms, training deep learning models with transfer learning and domain adaptation, validating in simulation in QLab with detailed performance metrics, implementing the distributed architecture for hardware

deployment, and conducting comparative validations to measure sim-to-real transfer degradation.

3.1.2 Rationale for Simulation-to-Hardware Approach

Simulation allows rapid iteration without risking hardware damage, and QLABs provides repeatable testing conditions. It enables testing scenarios in a controlled, stable environment, automatically generating datasets with proper annotations, and safely testing new algorithms. It is crucial to test the baseline's performance in simulation.

However, it has been understood that it is difficult to reproduce sensor noise, lighting, blur, and time constraints in a simulation. To address this difference, systematic validation is needed. The progressive method involves setting up a simulation framework, then integrating hardware. This enables quantitative assessment of system performance degradation.

3.1. Development Platforms

3.2.1 QLABs Virtual Environment (Simulation Platform)

3.2.1.1 Platform Description

QLabs (Quanser Interactive Labs) offers a physics-based simulation service for QCar, featuring an identical sensor API that enables seamless code transfer from simulation to actual implementation. The main features of this simulator include realistic rendering for vision development, customizable track configurations with traffic features, simultaneous multi-sensor data streams, and Python-based programming support. The identical API makes it easy to switch from simulation implementation to actual implementation without modification (Quanser, n.d.b).

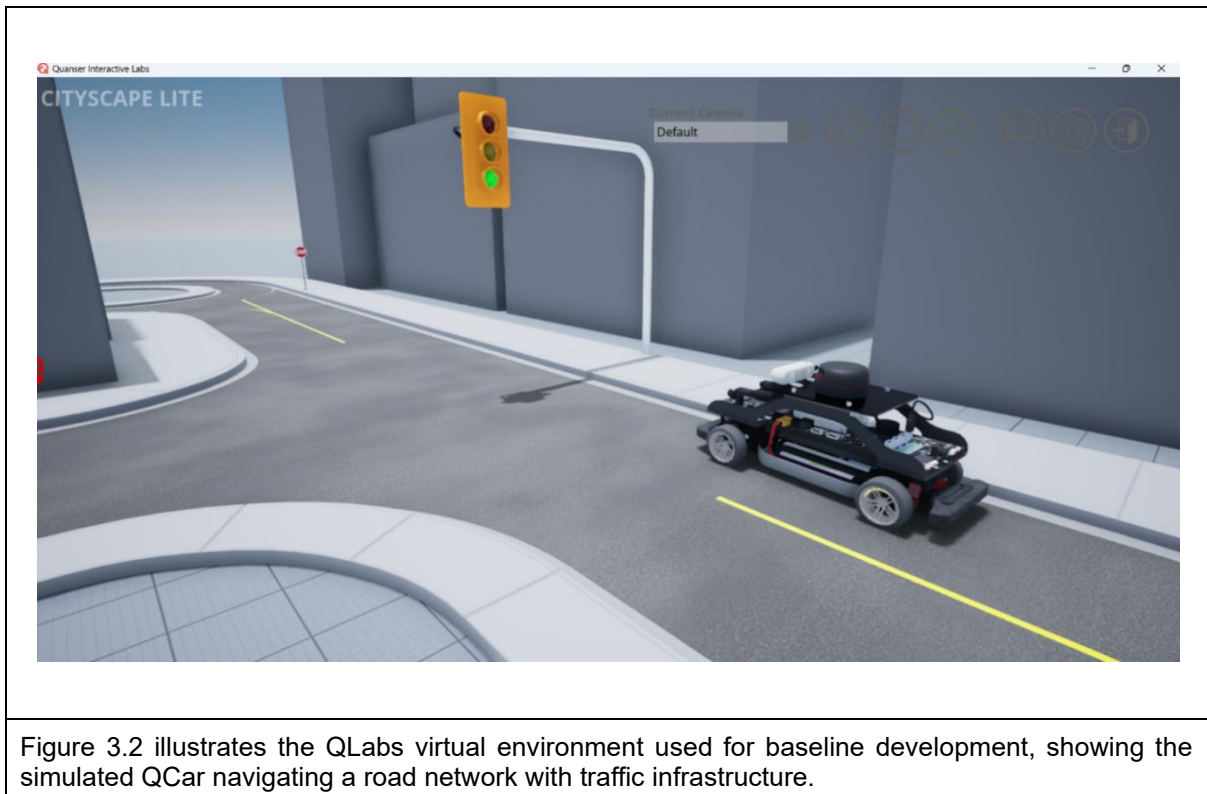


Figure 3.2 illustrates the QLab virtual environment used for baseline development, showing the simulated QCar navigating a road network with traffic infrastructure.

3.2.1.2 Virtual QCar Specifications

The simulated QCar replicates physical platform characteristics:

Component	Specification
Vehicle Dynamics	Ackermann steering geometry with validated friction and inertia models
Sensors	CSI camera (1640×1232 pixels, 30 FPS), simulated RealSense depth camera, simulated RPLidar 2D LiDAR
Control Interface	Throttle: [-1, 1] (Normalised), Steering: [-1, 1] ($\pm 23^\circ$ physical range)
Physics Simulation	100 Hz update rate, kinematic and dynamic validation against hardware

Table 3.1: QLab Virtual QCar Platform Specifications

3.2.2 QCar Physical Platform (Hardware Deployment)

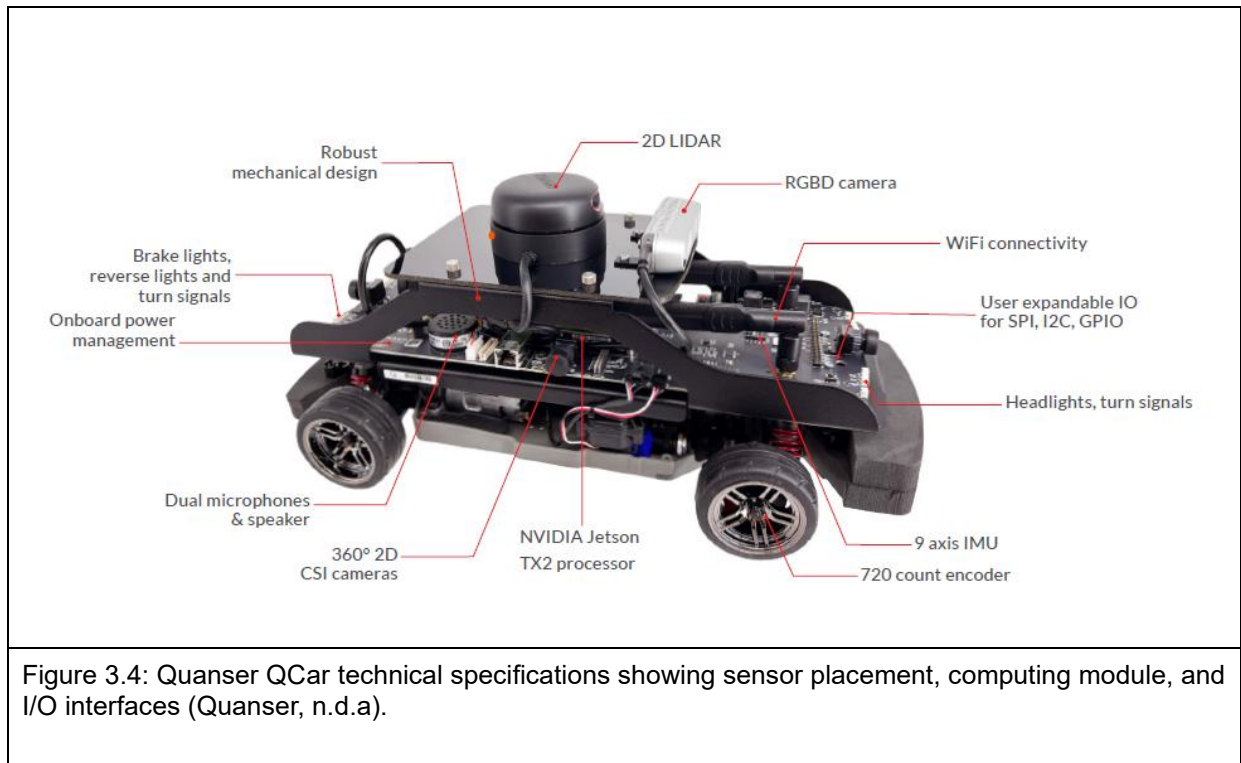
3.2.2.1 Platform Specifications

The QCar autonomous vehicle platform (1:10 scale) serves as the target hardware deployment environment. The computing hardware consists of NVIDIA Jetson TX2 (256 CUDA cores, 8GB RAM, 1.3 TFLOPS FP16) running Ubuntu 18.04 with CUDA 10.2.



Figure 3.3 shows the QCar platform with the integrated sensor suite used for hardware validation and real-world testing.

The sensor suite comprises three sensors that deliver visual and ranging information to the system. The CSI RGB camera can capture 640x480-pixel colour images at 30 frames per second and is designed to provide visual information to the system. The Intel RealSense D435 Depth Camera delivers 640x480 stereo image capture, with a resolution capable of calculating distances between objects and obstacles in meters. The RPLiDAR A2 360 Degree Laser Scanning Lidar delivers 8000 pulses a second with a maximum range of 12 meters.



The actuation processes are offered by two motor systems. The DC motor is used for throttle control via PWM speed control and allows for both forward and reverse movement. The steering is facilitated by a Servo motor via PWM angle control and allows orientation within the limits of the Ackermann steering geometry. The wheelbase is set to the value used in the simulation for $L = 0.256$ meters. The electrical components are powered by a 12V LiPo battery, a motor controller, and a 5V regulator for electronics.

3.2.2.2 Distributed Computing Architecture

The computational demands of concurrent deep learning models necessitate a distributed architecture: perception processing on a laptop workstation (NVIDIA RTX 3060, 12GB VRAM) and control execution on a QCar Jetson TX2.

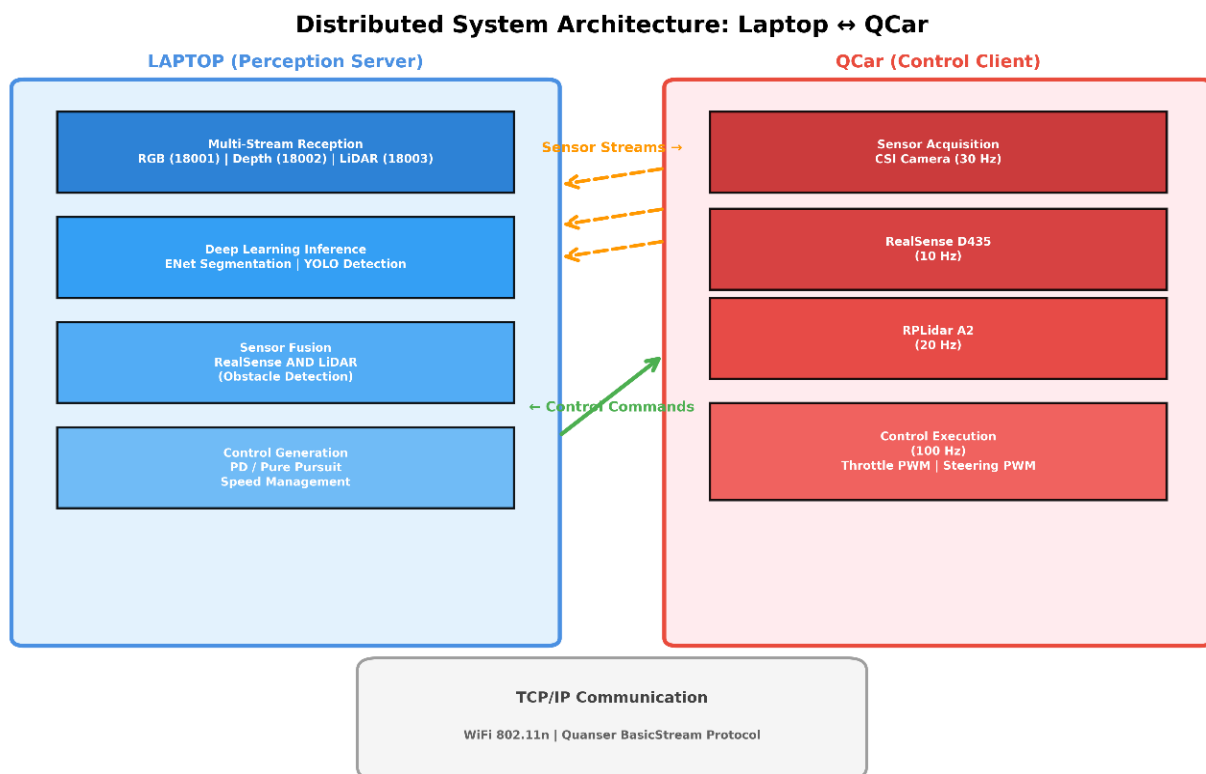


Figure 3.5: Distributed system architecture showing laptop-based perception processing (left) with multi-stream reception, deep learning inference, and sensor fusion, and QCar-based control execution (right) with sensor acquisition and motor control, connected via TCP/IP over WiFi 802.11n.

The laptop server executes YOLO detection, ENet segmentation, and depth/LIDAR processing, and transmits control commands via TCP/IP to the QCar client for trajectory tracking, motor control, and data acquisition. The BasicStream protocol from Quanser on 5GHz WiFi is used for transmission, requiring an end-to-end delay of 50ms or less for JPEG-compressed images of 100KB/frame resolution and compressed control commands of 1KB or less. Non-blocking sensor reads with 50ms timeouts, 5-command buffers, and emergency stop after 200ms of lost communication enable the system to run at 20+ FPS while maintaining control.

Execution Mode Clarification:

Two modes were tested: (1) Jetson TX2-only benchmarks for model selection, timing and (2) a distributed system with laptop inference and Jetson handling sensors and control. All closed-loop tests and results (18–22 FPS) used the distributed approach.

3.2. Dataset Preparation and Annotation

3.3.1 Traffic Detection Datasets

3.3.1.1 Simulation Datasets (QLabs)

Two datasets were prepared for QLABs traffic detection:

Traffic Light Dataset:

- Source: QLABs virtual environment recordings
- Scenes: Straight roads, intersections, roundabouts
- Classes: RED_LIGHT, GREEN_LIGHT (binary classification)
- Annotations: 1,247 bounding boxes using Labellmg
- Characteristics: Consistent lighting, perfect geometry, no occlusion

Stop Sign Dataset:

- Source: QLABs scenario recordings
- Class: STOP_SIGN (single class)
- Annotations: 496 instances
- Characteristics: Controlled placement, ideal visibility



Figure 3.6: Simulation training data examples showing (a) traffic light detection (1,288 instances, 2 classes) and (b) stop sign detection (496 instances, 1 class) with YOLO bounding box annotations from QLab's virtual environment.

Separate datasets reflected the initial development strategy; later unified for hardware deployment. Simulation data provides a clean baseline with consistent lighting and perfect annotations, but lacks real-world variability.

3.3.1.2 Hardware Dataset (QCar Unified)

The hardware unified dataset addresses real-world deployment requirements through comprehensive data collection from the physical QCar platform.

Collection Strategy:

- Multiple physical environments: lab, hallways, outdoor areas
- Varied lighting: natural daylight, fluorescent, mixed sources
- Different distances: 0.5m to 5m
- Occlusion levels: none, partial, significant
- Active traffic signal states only: red, green, yellow.

Dataset Composition:

The hardware unified dataset comprises 3,802 annotated instances across 5 traffic element classes required for autonomous navigation. Only active traffic signals and relevant road signs were annotated to match operational decision-making requirements.

Class	Instances	Percentage	Train / Val / Test
Red Light	865	23%	606 / 130 / 129
Yellow Light	814	21%	570 / 122 / 122
Green Light	818	22%	573 / 123 / 122
Stop Sign	975	26%	683 / 146 / 146
No-Right-Turn	330	9%	231 / 50 / 49
Total Dataset	3,802	100%	2,663 / 571 / 568

Table 3.2: Hardware Unified Dataset Composition

Justification of Dataset Split and Class Distribution

A 70/15/15 train/validation/test split was adopted following established dataset preparation methodology (Roh et al., 2021). The training set (70%) provides sufficient samples for robust feature learning, the validation set (15%) supports model selection and early stopping without leaking test information, and the held-out test set (15%) provides an independent estimate of generalisation performance. Stratified sampling was applied to maintain consistent class distributions across all subsets.

Traffic lights comprise the majority of instances (66%), with red (23%), green (22%), and yellow (21%) distributed according to typical operational frequency. Regulatory signs account for the remaining 35%, with stop signs (26%) and no-right-turn signs (9%).

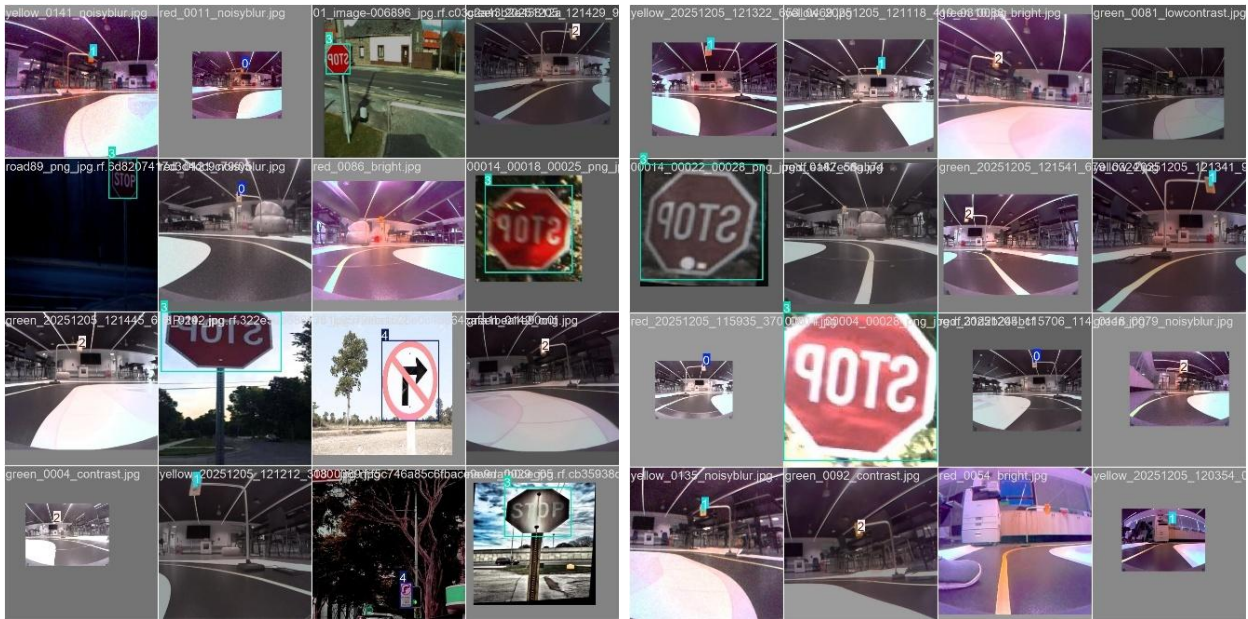


Figure 3.7: Hardware unified dataset training samples (3,802 instances, 5 classes) showing real-world environmental diversity, including varied lighting, indoor/outdoor scenes, motion blur, and challenging backgrounds with YOLO bounding box annotations.

Five classes of red, yellow, green, stop, and no_right_turn were created by using the open-source annotation tool Labellmg. In the above process, only active signals were labelled, excluding OFF states, as those are irrelevant for navigation. Variation in lighting, perspective distortion, occlusion, and sensor noise is included in the hardware dataset.

3.3.2 Lane Segmentation Dataset

3.3.2.1 Data Collection

Source images: QCar CSI camera during test drives on a custom 1:10 scale track with yellow/white lane markings. Collection scenarios: straight sections, curves (radius 0.5 -2m), intersections, varying lighting conditions. Total: 1,847 frames (1920 × 1080) from 15+ minutes of driving.

3.3.2.2 Annotation Process

Pixel-wise annotation was performed using the Computer Vision Annotation Tool (CVAT). Three semantic classes were defined: background (0), yellow lane markings (1), and white lane markings, including roundabout regions (2). Lane boundaries were manually traced and verified for consistency and edge precision. Final masks were exported as single-channel PNG files with pixel values corresponding to each class.

3.3.2.3 Data Augmentation

To enhance model robustness and generalisation, real-time data augmentation was applied during training.

Augmentation Technique	Parameters	Purpose
Horizontal Flip	50% probability	Bidirectional driving capability
Brightness Adjustment	$\pm 20\%$ variation	Illumination robustness
Rotation	$\pm 10^\circ$ range	Camera misalignment tolerance
Gaussian Blur	$\sigma = 0-1.5$	Motion blur simulation
Gaussian Noise	$\sigma = 0-0.05$	Sensor noise modelling

Table 3.3: Data Augmentation Strategy for Lane Segmentation Training.

Augmentations were applied stochastically during training only. The augmented dataset comprised 4,395 images derived from 1,847 base captures.

3.3. Model Training and Optimisation

3.4.1 Traffic Detection (YOLOv8)

3.4.1.1 Model Architecture Selection

YOLOv8 variants were evaluated for distributed deployment architecture.

Architecture comparison:

Model	Parameters	Speed (TX2)	mAP@0.5
YOLOv8n	3.2M	45 FPS	92.1%
YOLOv8s	11.2M	28 FPS	95.8%
YOLOv8m	25.9M	15 FPS	97.2%

Table 3.4: YOLOv8 Architecture Benchmarks (Ultralytics, 2023). Jetson TX2 benchmarks for model selection; final deployment used distributed laptop inference.

YOLOv8s selected for 95.8% accuracy at 28 FPS, meeting real-time requirements (>20 FPS) while maintaining high detection precision.

3.4.1.2 Simulation Models (Baseline Development)

Separate YOLO models were trained for traffic light and stop sign detection in QLABs simulation to establish baseline performance under controlled conditions.

Training Configuration:

Hyperparameter	Value	Rationale
Architecture	YOLOv8s	Optimal accuracy-speed balance
Input Resolution	640×640 pixels	Standard YOLO input size
Batch Size	16	GPU memory constraint (12GB VRAM)
Training Epochs	100	Empirical convergence observation
Initial Learning Rate	0.01(initial)	SGD (YOLOv8 default)
Augmentation	Mosaic, MixUp, HSV	YOLO default pipeline
Transfer Learning	COCO pretrained weights	Accelerated convergence

Table 3.5: YOLOv8 Training Configuration for Simulation Baseline Models

Hyperparameter Justification

Architecture Selection: YOLOv8s (11.2M parameters) was selected for its practical balance between detection accuracy and inference speed. YOLOv8n offers faster inference but reduced accuracy, while YOLOv8m provides marginal accuracy gains at significantly higher computational cost (Ultralytics, 2023).

Input Resolution: The 640×640-pixel resolution is the standard input size for YOLOv8, balancing detection accuracy with computational efficiency. Higher resolutions improve small object detection but increase memory requirements and reduce inference speed (Ultralytics, 2023).

Batch Size: A batch size of 16 was selected to maximise GPU memory utilisation within the 12GB VRAM constraint while maintaining stable gradient estimates during training (Goyal et al., 2017).

Training Epochs: Training was conducted for 100 epochs based on empirical observation. Validation metrics plateaued after approximately 80 epochs, and further epochs provided minimal improvement while risking overfitting.

Learning Rate: The initial learning rate of 0.01 follows YOLOv8's default SGD configuration, which has demonstrated stable convergence across diverse object detection tasks (Ultralytics, 2023).

Augmentation: Mosaic, MixUp, and HSV augmentations are part of YOLOv8's default pipeline. Mosaic combines four training images, thereby improving small-object detection and batch diversity. MixUp blends image pairs for regularisation. HSV augmentation enhances colour robustness (Ultralytics, 2023).

Transfer Learning: COCO pretrained weights provide robust initial feature representations, accelerating convergence and improving final accuracy compared to random initialisation, particularly beneficial for limited dataset sizes (Zou et al., 2023).

These baseline models serve as a reference for subsequent sim-to-real transfer analysis.

3.4.1.3 Hardware Unified Model (Production Deployment)

For real-world deployment, a unified YOLOv8s model was trained on the hardware dataset (3,802 instances, 5 classes) using transfer learning from COCO-pretrained weights.

Different datasets were used for the simulation and hardware stages. Two different YOLO models, each validated in QLABs: one for traffic lights (red/green) and one for stop signs. For hardware deployment, labels were unified into a single 5-class set: red, yellow and green traffic lights, a stop sign, and a no-right-turn sign. Dataset-specific labels outside of this set were either combined or removed during pre-processing to maintain consistency.

Stage	Model Type	Classes
Simulation – Traffic Lights	YOLO (separate)	2 (Red, Green)
Simulation – Stop Signs	YOLO (separate)	1 (Stop)
Hardware – Unified	YOLO (unified)	5 (Red, Yellow, Green, Stop, No-right-turn)

Table 3.6: Model Architecture Evolution - Simulation (separate detectors) to Hardware (unified 5-class detector)

Parameter	Value	Rationale
Base Model	YOLOv8s (COCO pretrained)	Transfer learning foundation
Dataset	Hardware unified	Real-world variability
Split	70% train / 15% val / 15% test	Standard ML practice
Total Epochs	100	Extended for convergence
Learning Rate	0.01 → 0.0001 (cosine decay)	Gradual convergence
Batch Size	16	Same as simulation baseline
Augmentation	Mosaic, MixUp, HSV	YOLO default pipeline

Table 3.7: Hardware Unified Model Training Configuration

3.4.2 Lane Detection Approaches

Lane detection employed a progressive methodology: classical computer vision for simulation baseline, followed by deep learning for robust hardware deployment.

3.4.2.1 Classical Computer Vision Approach (Simulation Baseline)

The classical HSV lane detection method (MathWorks, n.d.) served as the baseline implementation upon which the simulation baseline was established. In the HSV colour space, lane boundary isolation via HSV segmentation relies on colour thresholds to isolate yellow and white lanes. Sensitivity to lighting and occlusion led to the implementation of the learned approach.

Processing Stage	Method	Parameters
Colour Space	RGB → HSV conversion	Colour constancy enhancement
Yellow Lane Threshold	HSV filtering	H: 80-100, S: 100-255, V: 100-255
White Lane Threshold	HSV filtering	H: 0-180, S: 0-50, V: 200-255
Morphological Operations	Opening + Closing	5×5 kernel (noise removal, gap filling)
Centroid Extraction	Pixel averaging	Lane centre computation
Path Computation	Midpoint calculation	Target = midpoint (yellow, white)

Table 3.8: Classical Lane Detection Pipeline Configuration (QLabs Simulation)

Adaptive Strategy:

- Both lanes detected → midpoint following.
- Single lane detected → offset from detected lane.
- No lanes detected → maintain previous command.

Rationale: This classical approach enabled rapid simulation development and algorithm validation before investing in deep learning dataset preparation. However, anticipated limitations (lighting sensitivity, fixed thresholds, poor occlusion handling) motivated deep learning adoption for hardware deployment.

3.4.2.2 Deep Learning Segmentation Models (Hardware Deployment)

Hardware deployment required deep learning segmentation to handle real-world environmental variation beyond the capabilities of classical methods. This section describes the approaches used for lane detection.

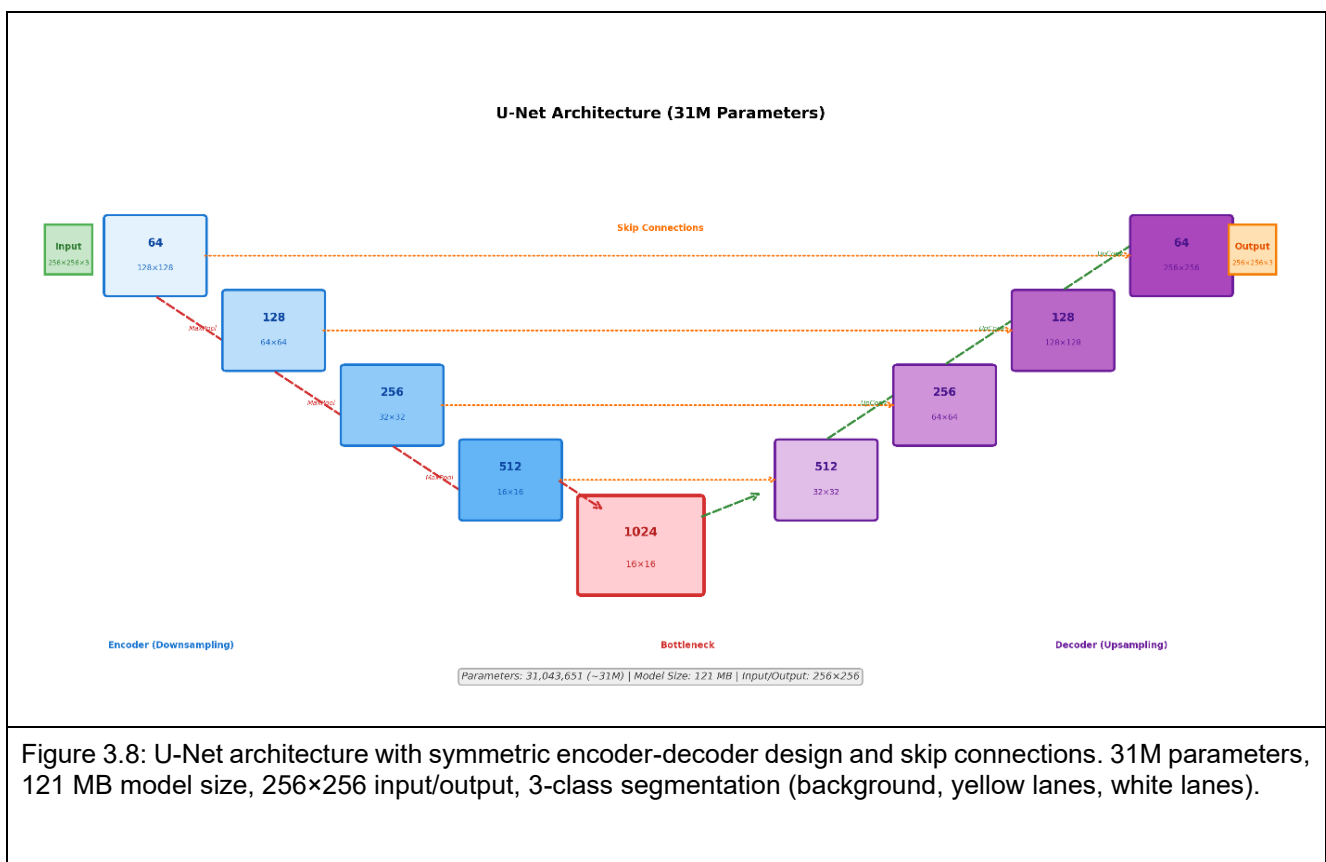
3.4.2.3 Architecture Comparison and Selection

Two semantic segmentation architectures were evaluated for lane detection: U-Net (accuracy-focused) and ENet (efficiency-focused).

Component	U-Net (Accuracy)	ENet (Efficiency)
Parameters	31,043,651 (31.0M)	355,249 (0.355M)
Model Size	121 MB	1.4 MB
Architecture	Symmetric (4+4) with skip connections	Asymmetric (3+2) without skip
I/O Resolution	256×256×3 → 3-class segmentation	Same
Memory Footprint	High	87× lower
Embedded Suitability	Limited	Optimized

Table 3.9: Architecture Specifications - U-Net vs Enet

Key Insight: ENet's 87× parameter reduction (31.0M → 0.355M) and 86× model size reduction (121 MB → 1.4 MB) enables real-time deployment on memory-constrained Jetson TX2.



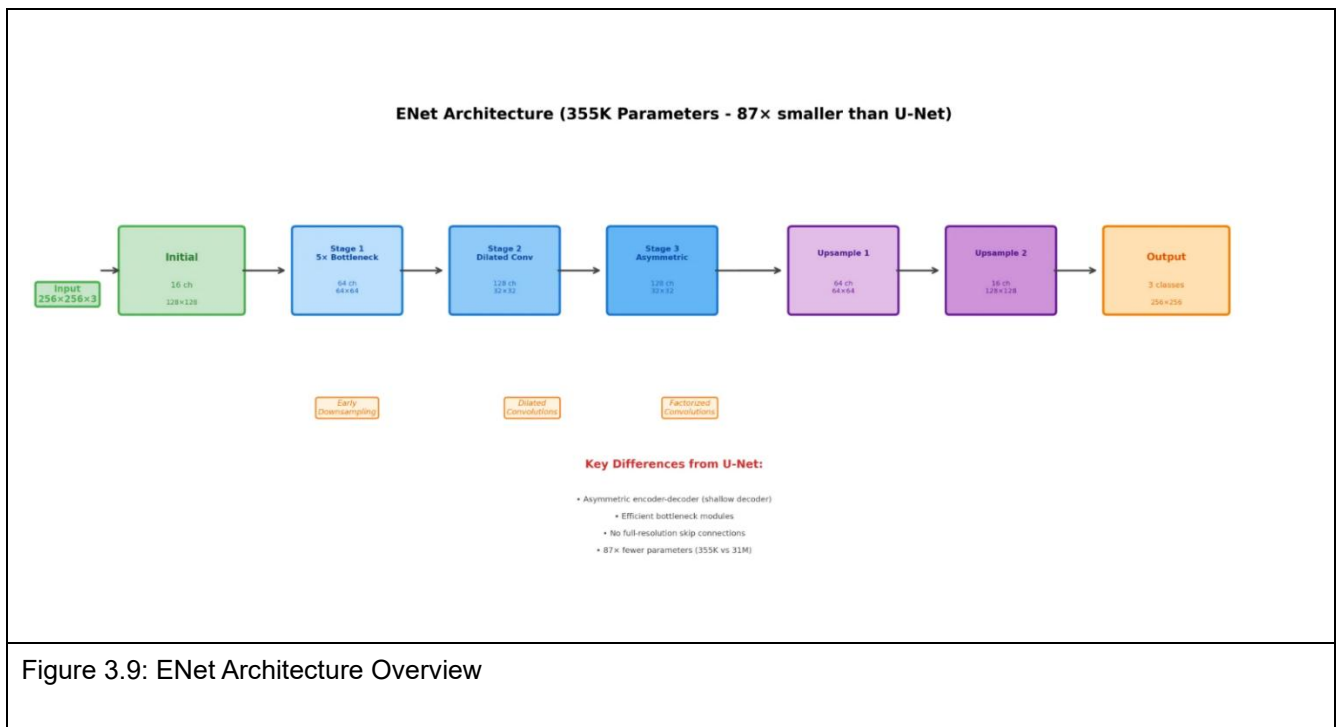


Figure 3.9: ENet Architecture Overview

3.4.2.4 Segmentation Model Training

These two architectures were trained and tested on the same set of images (4,395 images, with a training: validation split of 90:10). Three classes were defined: background (0), yellow lines (1), and white lines (including roundabouts) (2). The U-Net was trained with a mini-batch size of 8 (limited by the model's 31M parameters), the Adam optimiser with a learning rate of 0.001, the ReduceLROnPlateau learning rate scheduler, and early stopping with a patience of 10 epochs. The ENet used its tiny model size (0.355M params), a mini-batch size of 16, the Adam optimiser with a learning rate of 0.0005 that decays to 0.00025 after epoch 95, and weight decay of 1e-4. Both networks used the cross-entropy loss function, random start, and the same set of data augmentation operations: brightness, contrast, horizontal flip, rotation, and crop. After training, the ENet model was deployed on the laptop server for distributed computing. In evaluation, improvements were noted in feature annotation for roundabout boundaries and white lines, with no loss to navigation functionality.

3.4. Control System Design

The control system translates perception outputs into vehicle motion commands, implementing lane following, traffic signal compliance, and obstacle avoidance. The modular architecture separates low-level control from high-level decision making.

3.5.1 Lane Following Control

Two different controllers were designed. PD was used for basic development, and Pure Pursuit for complicated shapes.

3.5.1.1 PD Controller

The Proportional-Derivative controller provided baseline lane following for straight roads and gentle curves. The control law computed steering commands based on lateral error (deviation from the lane centre) and the error rate of change:

$$\delta = K_p \cdot e + K_d \cdot \dot{e} \quad (\text{Eq. 1})$$

where e represents the lateral error extracted from the segmentation mask centroids.

Parameter Configuration: $K_p = 0.5$ was tuned empirically to balance responsiveness and oscillation; values below 0.3 led to sluggish response, while values above 0.7 induced instability. $K_d = 0.1$ provides derivative damping to reduce overshoot and oscillation (Rokonuzzaman et al., 2021). The 30 Hz update rate matches the camera frame rate, ensuring synchronised control and exceeding the minimum 10 Hz recommended for low-speed lateral control (Badue et al., 2021). Saturation limits of ± 0.5 radians respect QCar's physical steering constraints (Quanser, n.d.a).

3.5.1.2 Pure Pursuit Controller

Pure Pursuit enabled geometric path following for roundabouts and sharp curves through lookahead-based steering. The Ackermann steering relationship computes the steering angle as

$$\delta = \text{atan} (2L \cdot l_d / L_d^2) \quad (\text{Eq. 2})$$

where L = wheelbase (0.256m), l_d = lateral distance to lookahead point, and L_d = lookahead distance.

Parameter Configuration: The wheelbase $L = 0.256\text{m}$ matches QCar's physical specification (Quanser, n.d.a). The lookahead distance $L_d = 0.8\text{m}$ was tuned empirically in simulation; shorter distances ($< 0.5\text{m}$) caused aggressive oscillatory tracking, while longer distances ($> 1.2\text{m}$) reduced path accuracy on tight curves. The selected value balances trajectory smoothness against tracking precision for the test track geometry (Rokonuzzaman et al., 2021).

The algorithm extracted the lane centreline from segmentation masks and selected lookahead points along this path. This lookahead-based approach provides inherent smoothing while maintaining geometric accuracy for curved trajectories (Rokonuzzaman et al., 2021).

3.5.2 Traffic Signal Compliance

YOLO detections were used to drive a set of simple, rule-based behaviours for traffic-signal compliance. At traffic lights, a red light triggered the vehicle to slow down and stop. A yellow light was treated more cautiously: if the car was already close to the junction, it continued through; otherwise, it slowed and stopped. A green light allowed normal driving to continue. If no signal was detected, the controller kept the current state as a basic fail-safe rather than switching behaviour unexpectedly. Signal proximity was estimated using the bounding-box height, as larger boxes were assumed to indicate a closer light, which triggered braking earlier.

Stop-sign handling followed a fixed four-step routine. Once a stop sign was detected, the vehicle slowed and came to a full stop when the bounding box exceeded a predefined size threshold (used as a proxy for distance). The vehicle then held the stop for 3 seconds and resumed motion only when the path ahead was confirmed to be clear.

3.5.3 Obstacle Detection and Emergency Braking

Sensor fusion between the RealSense depth camera and RPLidar implemented obstacle detection for emergency braking. The fusion strategy employed AND-logic, requiring both sensors to agree before triggering emergency stops to minimise false positives.

3.5.3.1 RealSense Depth Processing

The RealSense depth camera detected obstacles in the vehicle's path. Depth information was extracted from a region of interest centred on the expected travel path.

Depth processing pipeline:

1. Capture a depth frame with a resolution of 640 x 480.
2. Definition of a region of interest (ROI)—Driving corridor: define ROI on driving corridor, position at centre 60% of image width & 30–60% of image height.
3. Range filter for retaining a valid range of depths from 0.5 m to 3.5 m.
4. Calculate how many pixels are within the region of interest that are below a given distance threshold.
5. Mark the obstacle when the pixel count surpasses the minimum threshold.

The ROI was selected to focus on what is relevant for the navigation: objects directly ahead, and to limit at the same time interferences from peripheral regions that are often irrelevantly structured. The range limits helped to reject spurious readings, either caused by the vehicle body at very short distances or by distant surfaces that are not relevant for immediate collision avoidance.

The RealSense depth camera was configured with a critical detection distance of 1.2 metres and a minimum pixel threshold of 100 pixels. The region of interest covered the centre 60% of the frame's width and the 30-60% of its height, focusing on the road ahead while avoiding edge distortion.

3.5.3.2 LiDAR Processing

The RPLidar provided 360-degree ranging data, serving as the initial stage of the obstacle detection process and offering comprehensive coverage, unlike the depth camera, which is prone to failure modes. The data was then filtered, and points within a critical proximity zone were counted.

LiDAR processing pipeline:

Processing steps:

- a) Acquire 360° scan data
- b) Filter points by distance: <0.8 meters

- c) Count filtered points
- d) Classify as strong/weak detection
- e) Apply temporal persistence for weak detections

RPLidar A2 was set up with a critical range of 0.8 metres, scanning frequency of around 20 Hz, and angular resolution of 0.5 degrees. For strongly detected lines (more than 20 points), response occurred in real time, while for weak lines (5 to 19 points), response occurred based on temporal persistency.

3.5.3.3 AND-Based Sensor Fusion

The fusion strategy required both sensors to detect obstacles before triggering emergency braking, implementing a conservative approach that prioritised reducing false negatives over preventing false positives. This AND-logic reflected the complementary failure modes of the two sensors.

Sensor Fusion Logic: AND-Based Obstacle Detection

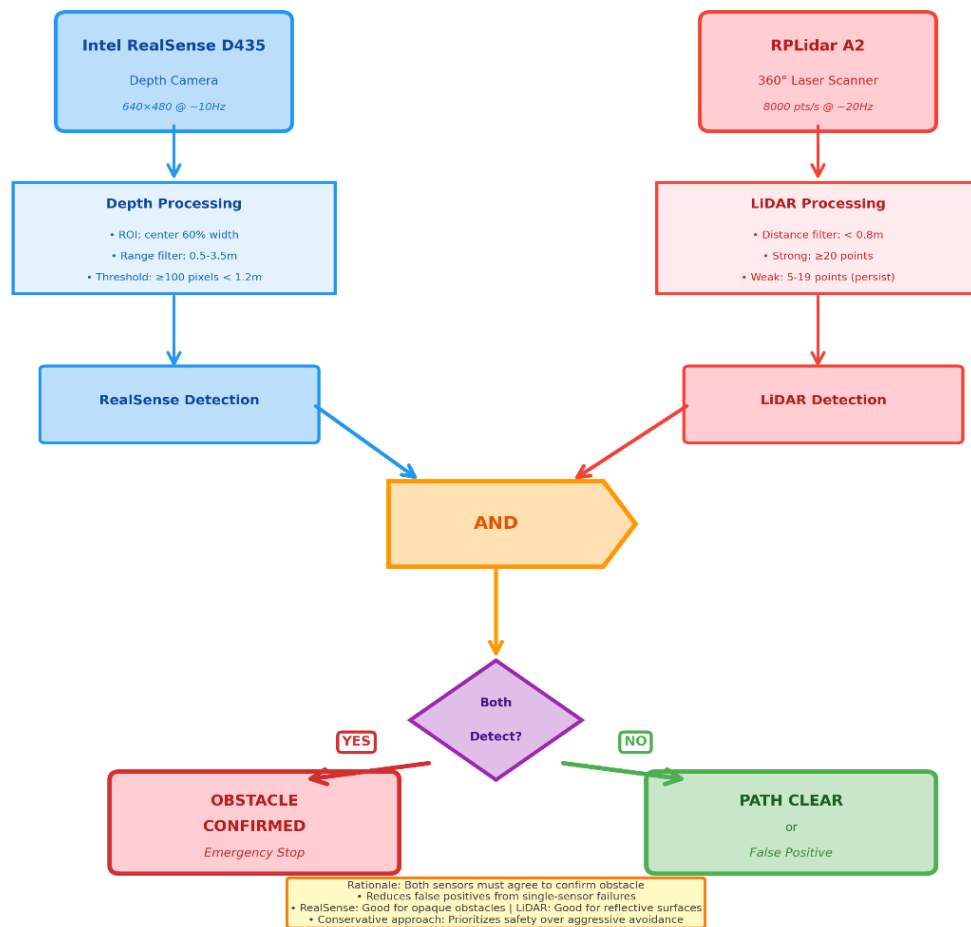


Figure 3.10: AND-based sensor fusion logic requiring both RealSense depth camera and RPLidar agreement before triggering emergency braking.

Sensor Fusion Logic:

The AND-logic fusion needed simultaneous detection by the two sensors: an obstacle was declared only when the RealSense and LiDAR detected it; in that case, an emergency braking was initiated. In any other case, normal operation resumed. This inherent redundancy prevented the weaknesses of individual sensors: transparent or reflective surfaces will cause RealSense to fail, while LiDAR mostly fails on small objects. Passive braking-zero throttle, coast-to-stop-proofed sufficient during low-speed indoor operations, and there was no need to implement any active deceleration control.

3.5. System Integration Architecture

3.6.1 Simulation Architecture (Monolithic)

Simulation used a monolithic architecture with all processing in a single Python process on the laptop. The control loop ran at approximately 20 Hz. Each cycle captured an image from QLABs, processed it through HSV-based lane detection for lane following and two separate YOLO models for traffic light and stop sign detection, then sent control commands to the simulator. This setup enabled rapid prototyping and simplified debugging.

3.6.2 Hardware Architecture (Distributed)

The distributed architecture was required by the hardware to separate perception on the laptop from control on the QCar, due to the computational limitations of the Jetson TX2, as explained before. TCP/IP enabled bidirectional communication, and its processing flow was divided between the laptop for perception and decision-making, while sensor and motor control were handled by QCar.

Model selection differed from the simulation: ENet and U-Net were used for segmentation, while a unified YOLO model handled all traffic detection. By reducing latency and memory usage, it preserves accuracy.

3.6. Testing and Validation Methodology

3.7.1 Simulation Testing

Simulation testing has been used to establish a baseline level of performance in a controlled QLABs setting. The test environments involved were lane following, a curved road, a roundabout, traffic lights, stop signs, and complex multi-element situations. The test scenarios were run multiple times to ensure consistency in logging paths, commands, and perception results.

3.7.2 Hardware Testing

Hardware testing, which validated real-world performance by quantifying the sim-to-real gap, was done progressively from simple to complex scenarios. The safety measures included the following: a physical barrier defining an operating

area, an emergency stop button, a maximum speed limit of 0.070 throttle setting, continuous human supervision, and sensor-fusion emergency braking.

Testing protocol:

1. System initialisation and sensor calibration
2. Ensure that the emergency stop works.
3. Perform scenarios in an incremental manner: straight → curves → roundabouts → traffic objects → obstacle avoidance
4. Log all metrics and document failures

3.7.3 Performance Metrics

Quantitative results were used to compare the simulation and hardware environment's perception and control subsystems. Table 3.10 below summarises the evaluation metrics used in this study (Zou et al., 2023; Minaee et al., 2021; Rokonzaman et al., 2021).

Category	Metric	Definition
Object Detection	mAP@0.5	Mean Average Precision at 0.5 IoU threshold
	Precision	True positives / all positive predictions
	Recall	True positives / all ground truth instances
	Inference Time	Processing time per frame (ms)
Segmentation	mIoU	Mean Intersection over Union across all classes
	Per-class IoU	IoU for yellow and white lane classes
	Pixel Accuracy	Percentage of correctly classified pixels
	Inference Time	Processing time per frame (ms)
Control	Lateral Error	Deviation from lane centre (m)
	Tracking Smoothness	Steering command variance
	Speed Consistency	Throttle variation during navigation
	Stopping Accuracy	Distance from intended stop position (m)

Table 3.10: Performance Evaluation Metrics.

3.7.4 Matched Scenario Testing

Test scenarios were designed to enable direct comparison across both platforms. Matched scenario pairs included straight lanes, curved roads, roundabouts, traffic lights, and stop signs. Identical metrics were collected on both platforms to quantify perception and control degradation in real-world conditions.

3.7.5 Component Validation and Integration Testing

Full system integration of individual components was performed after validation of each component. Lane-following testing was conducted to confirm stable tracking and boundary detection. Standalone mode traffic element detection was performed to validate YOLO inference accuracy. Obstacle detection used the RealSense and RPLidar sensors separately before implementing fusion logic.

This validation process highlighted some important integration needs: conflicts in sensor initialisation, constraints on communication timing, and resource allocation, which were used to inform the final architecture. Full system performance in realistic conditions is reported in Chapter 4.

3.7.6 Robustness Evaluation

Robustness testing was conducted to evaluate the system's response to challenging scenarios, including occlusions, varying lighting conditions, and sensor noise. Testing of occlusions was conducted differently based on the distance the obstacles were placed. Lighting test was executed under the varying lighting conditions. Tests based on distance were done based on the distance between the traffic objects and the Qcar.

3.7. Summary

This chapter presented a systematic methodology for vision-based autonomous navigation from simulation through hardware deployment on the QCar platform.

The approach included dataset preparation (1,784 simulations and 3,802 hardware detection instances, 4,395 segmentation images), architecture evaluation on a balance between accuracy and efficiency (YOLO detection, U-Net vs. ENet segmentation algorithms), distributed architecture design separating perception from

control, and integrated control combining geometric path tracking with rule-based logic and multi-sensor fusion.

Matched simulation and hardware testing scenarios enable quantitative sim-to-real assessment. The following chapter presents results and performance analysis validating this methodology.

4. RESULTS

This chapter covers the results of experiments with the autonomous vehicle navigation system, both in simulation in the QLABS environment and on the actual QCar vehicle for simulation-to-reality testing. These results include performance on object detection and lane segmentation, achieving the best possible training performance across various environments in sim-to-reality testing.

4.1. Object Detection Training Results

4.1.1 Simulation Environment Training

The initial model development phase utilised the QLABS simulation environment to train separate YOLO models for traffic light and stop sign detection. This approach enabled rapid prototyping and model validation before hardware deployment.

4.1.1.1 Traffic Light Detection Model

The traffic light detector was trained using the simulation dataset (1,288 instances, 80/20 split). Training used the YOLOv8 architecture, initialised with COCO-pretrained weights. The model converged after 100 epochs of training, completing in 2,277 seconds (approximately 38 minutes). Stochastic Gradient Descent (SGD) optimisation with linear learning rate decay was used, with the learning rate decreasing from 0.0016 to 0.000033 over the training period, representing a 49-fold reduction.

Final Performance Metrics (Epoch 100):

- Mean Average Precision (mAP@0.5): 99.44%
- Mean Average Precision (mAP@0.5-0.95): 51.46%
- Precision: 99.27%
- Recall: 99.43%

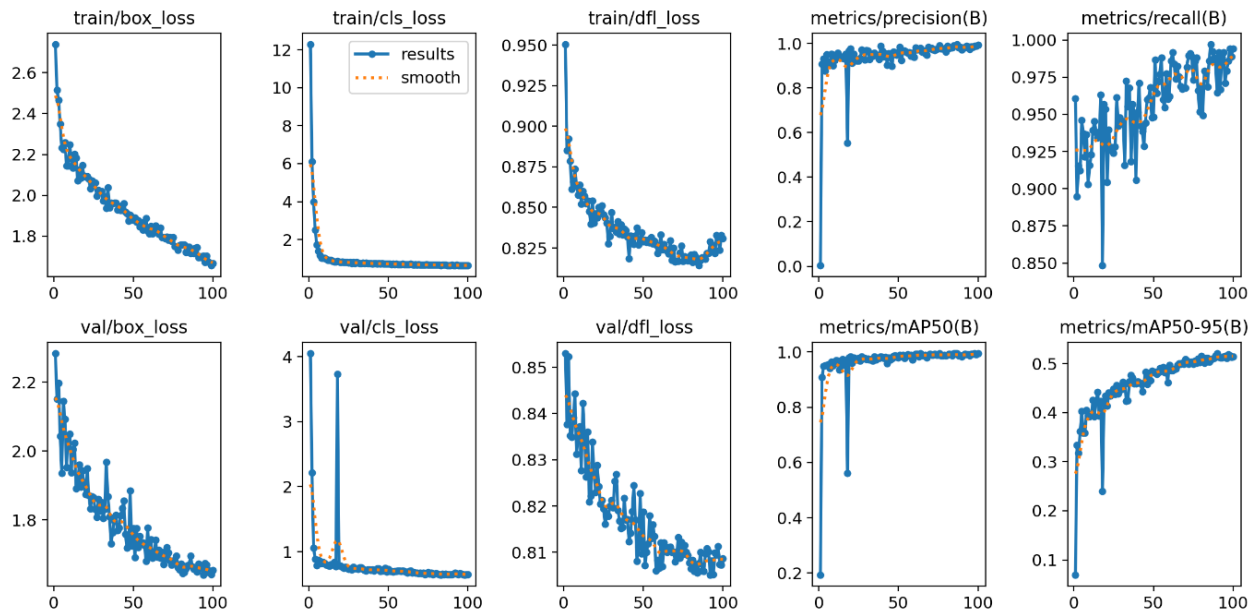


Figure 4.1: Traffic light detection training curves over 100 epochs showing loss convergence and performance metrics

The training curves showed smooth convergence for all the metrics (Figure 4.5). The box regression loss decreased from 2.74 to 1.67, the classification loss decreased from 12.30 to 0.64, whereas the Distribution Focal Loss (DFL) loss remained unchanged at 0.95. The precision and recall values reached 0.99 at around 10 epochs for both, while mAP@0.5 steadily increased to 0.99. The value for mAP@0.5-0.95

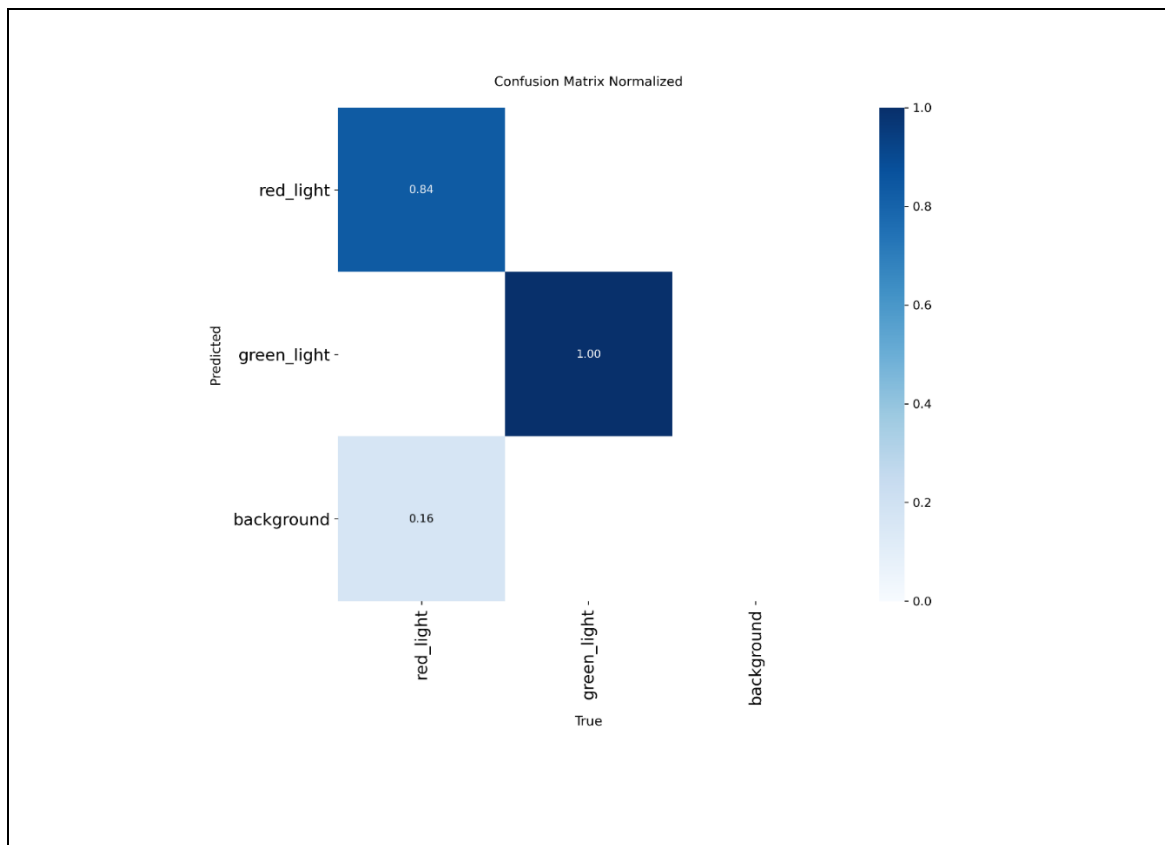


Figure 4.2: Normalised confusion matrix for traffic light detection (red: 84%, green: 100%)

The confusion matrix (Figure 4.6) showed strong class discrimination: red lights 84% correct, green lights 100% correct. Primary error source: 29 background false positives (16%) for red lights. No inter-class confusion occurred.

4.1.1.2 Stop Sign Detection Model

The stop sign detector was also trained on 496 labelled instances from the QLABS simulation, using the same 80/20 train/validation split. The single-class detection problem proved easier than multi-class traffic light detection, and convergence was faster.

Training used the same architecture and hyperparameters as the traffic light model, which converged in 82 epochs via early stopping, finishing training in just 742 seconds, or about 12 minutes. This reflects both the smaller dataset and a much simpler single-class detection task.

Final Performance Metrics (Epoch 82):

- Mean Average Precision (mAP@0.5): 99.50%
- Mean Average Precision (mAP@0.5-0.95): 86.12%
- Precision: 99.95%
- Recall: 100.00%

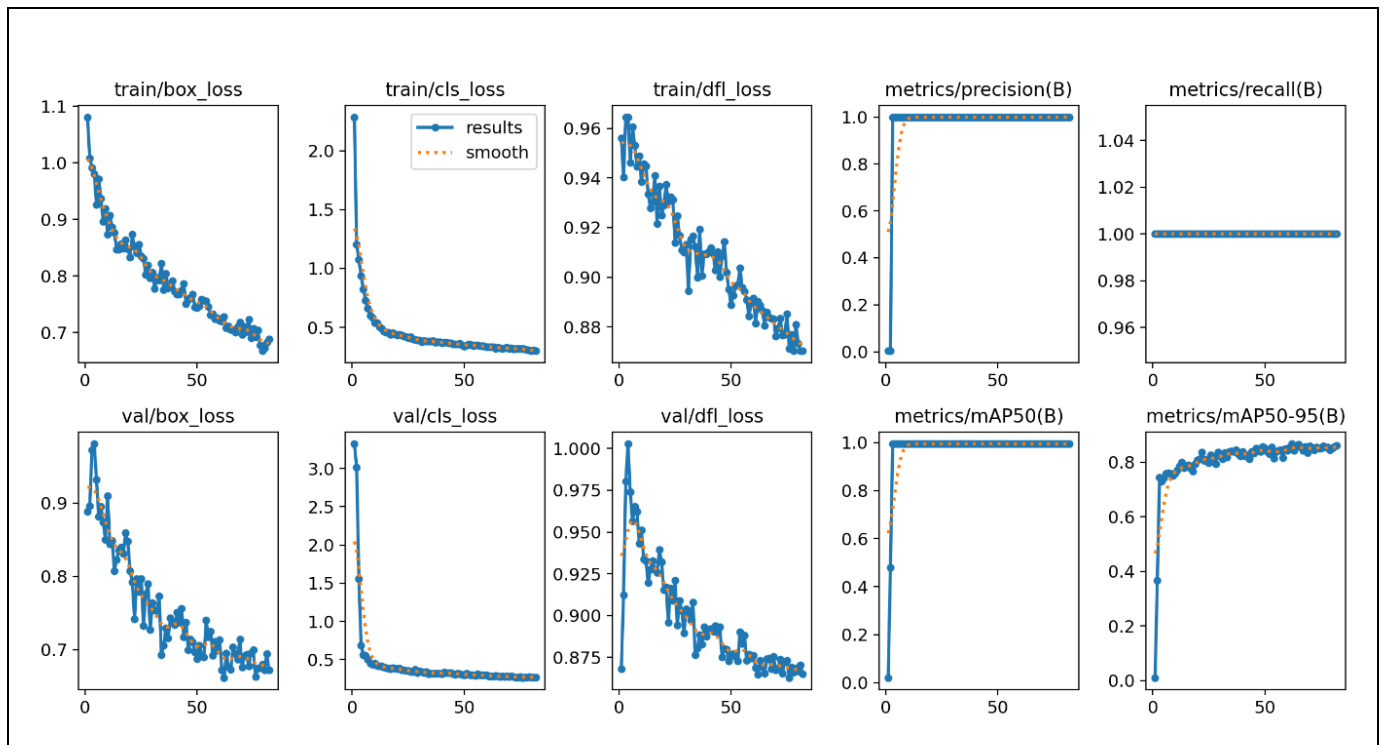


Figure 4.3: Stop sign detection training curves over 82 epochs demonstrating rapid convergence

Training curves showed rapid convergence, with precision and recall reaching 1.0 by epoch 10. mAP@0.5-0.95 reached 86.12% versus traffic lights' 51.46% (67% improvement in localisation quality).

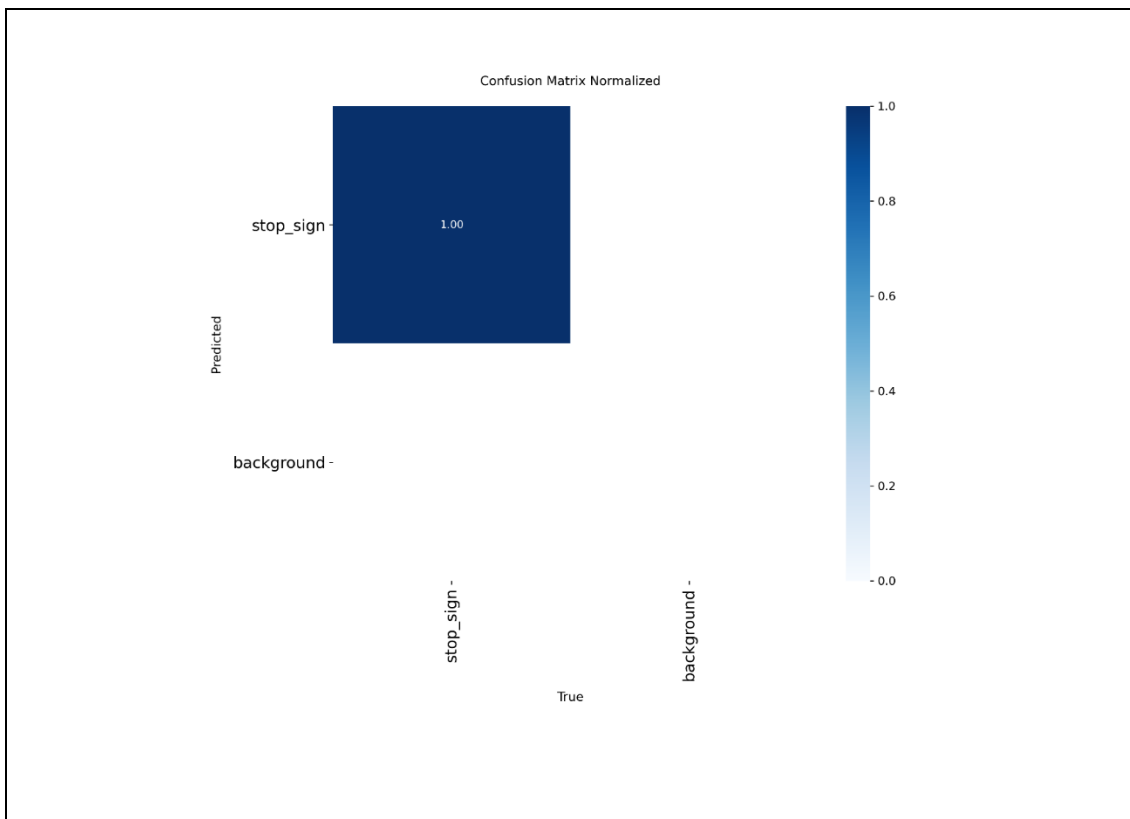


Figure 4.4: Normalised confusion matrix for stop sign detection (single-class model)

The confusion matrix (Figure 4.4) confirmed 100% correct classification across all 124 validation instances.

4.1.1.3 Simulation Results Analysis

Comparison of the two simulation models reveals task complexity effects on detection performance:

Metric	Traffic Lights	Stop Signs	Observation
Dataset instances	1,288	496	2.6× more data
Training epochs	100	82	Earlier convergence for stops
Training time	38 min	12 min	3.2× faster (smaller dataset)

mAP@0.5	99.44%	99.50%	Near perfect value for both
Recall	99.43%	100%	Perfect stop sign detection
mAP@0.5-0.95	51.46%	86.12%	67% better localisation

Table 4.1: Simulation Training Results Comparison

Stop sign detection turned out to be much easier than traffic light detection as the geometric features are more salient due to the octagonal shape, the high contrast of red against most backgrounds allows robust colour-based discrimination, and the larger apparent size in camera frames simplifies localisation. In contrast, traffic lights feature smaller target objects, smaller appearance differences between red and green states, and less distinctive shape features. Both have demonstrated excellent detection performance in the controlled simulation environment and have validated the model architectures and training procedures before hardware deployment.

4.1.2 Hardware Platform Training

For hardware deployment on the physical QCar, a unified detection model was trained to simultaneously detect all traffic objects in a single forward pass, improving computational efficiency and simplifying the detection pipeline.

4.1.2.1 Unified Multi-Class Detection Model

The unified model architecture used a YOLOv8 model trained to detect five classes: red lights, yellow lights, green lights, stop signs, and no-right-turn signs. This consolidation eliminated the need for multiple model inference passes and enabled efficient real-time detection on embedded hardware.

The training dataset comprised 3,802 labelled instances collected from the physical QCar camera across diverse real-world conditions. Training followed the standard YOLOv8 procedure with COCO pre-trained initialisation. The model ran for 100 epochs, taking 11,206 seconds (3.1 hours, 186 minutes). Longer training compared to simulation models was indicative of a larger dataset, more complex classes, and real-world image variations that proved more difficult for the model.

Final Performance Metrics (Epoch 100):

- Mean Average Precision (mAP@0.5): 92.87%
- Mean Average Precision (mAP@0.5-0.95): 58.27%
- Precision: 86.22%
- Recall: 95.13%

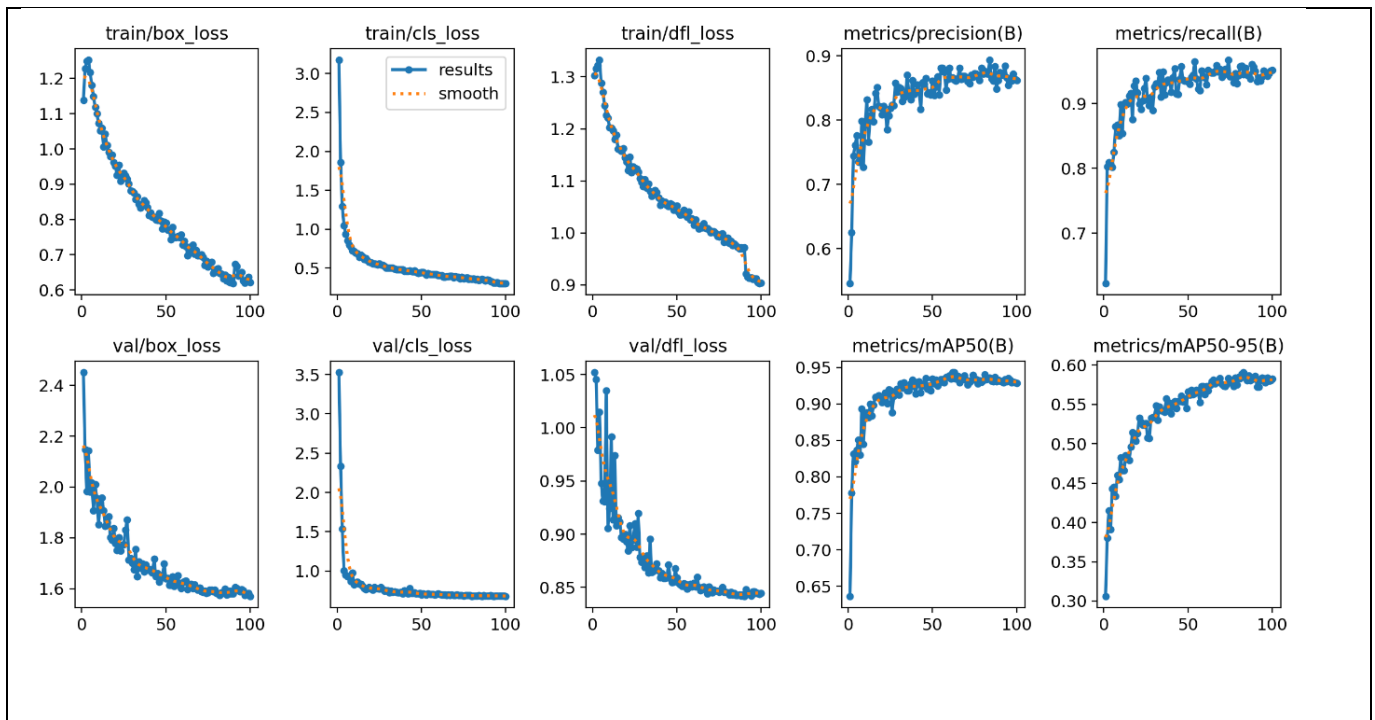


Figure 4.5: Unified 5-class detection model training progression over 100 epochs on real QCar hardware dataset

Training curves demonstrated steady convergence. Box loss decreased from 1.14 to 0.62, classification loss improved from 3.18 to 0.30, and Distribution Focal Loss (DFL) loss reduced from 1.30 to 0.90. The mAP@0.5 reached approximately 93% by epoch 100, with precision and recall stabilising above 86% and 95%, respectively.

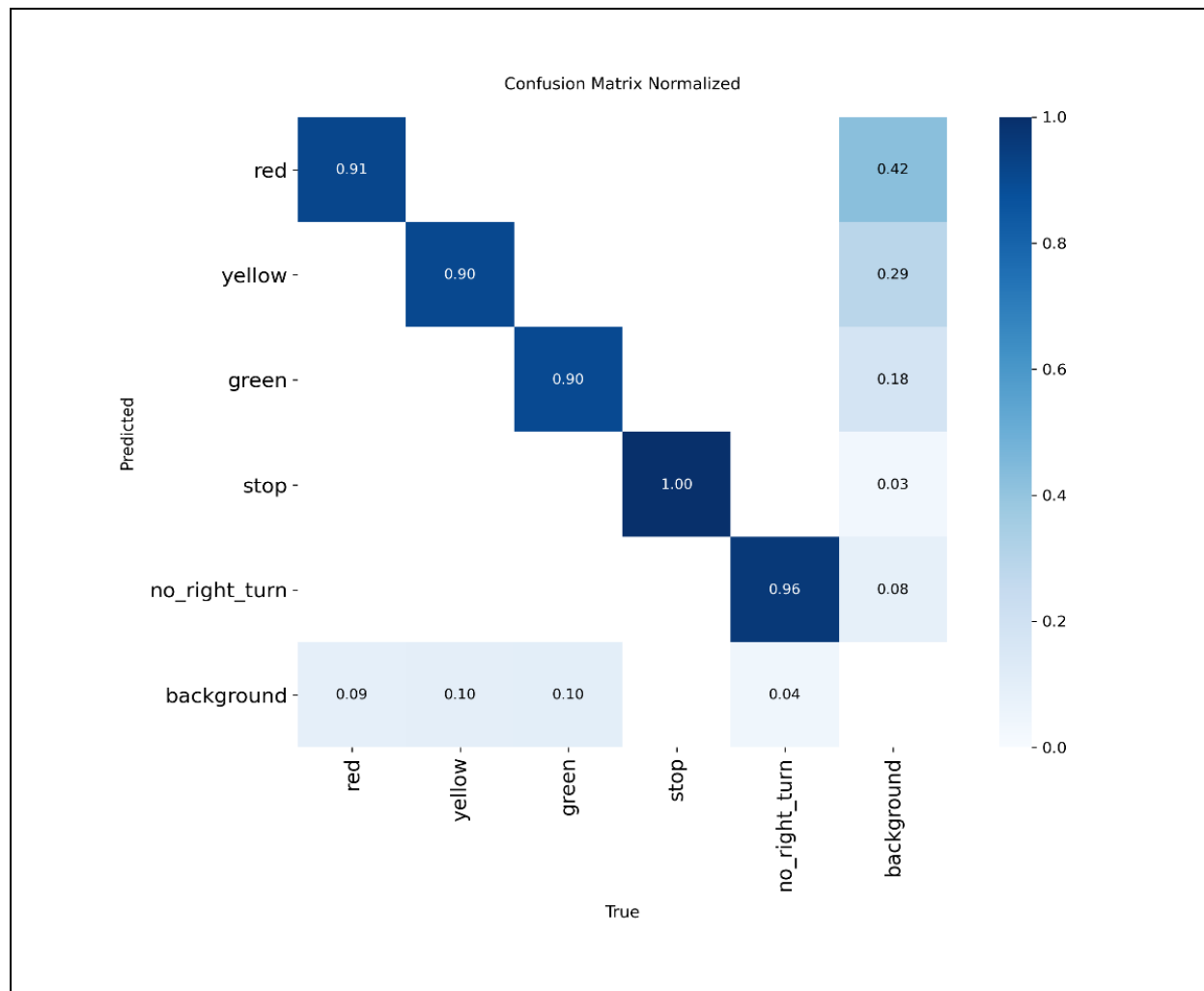


Figure 4.6: Normalised confusion matrix for unified 5-class detection model on real QCar hardware (validation set)

Confusion matrix analysis (Normalised) revealed per-class performance characteristics:

- Red lights: 91% recall with 31 background false positives
- Yellow lights: 90% recall with 21 background false positives
- Green lights: 90% recall with 13 background false positives
- Stop signs: 100% recall with 2 background false positives
- No-right-turn signs: 96% recall with 6 background false positives

The major source of error was background false positives, especially true for traffic light images. The red colour had the maximum number of background false positives,

with 31 images. The traffic sign images yielded excellent results, with a stop sign achieving a perfect classification and no-right-turn signs achieving 96% accuracy.

4.1.2.2 Sim-to-Real Performance Comparison

Direct comparison between simulation and hardware performance quantifies the reality gap:

Metric	Simulation (Separate)	Hardware (Unified)	Change
mAP@0.5	99.44% (lights), 99.50% (signs)	92.87%	-6.57%
Total classes	3 (red, green, stop)	5 (all objects)	+2 classes
Training time	50 mins combined	186 min	3.7× longer
Environment	Controlled simulation	Real-world	Increased complexity

Table 4.2: Sim-to-Real Performance Comparison

The 6.57 percentage-point drop is due to difficulties in sim-to-real transfer, including lighting variations, occlusions, background clutter, and camera noise. However, the 92.87% mAP@0.5 is remarkable in a real-world scenario, where the combined model can classify five classes simultaneously with an accuracy above 90%.

The additional training time (3.7×) was due to the increased dataset size (3,802 vs 1,784 instances), the complexity of the classes (5 vs 2+1 classes), and variations in the images. The result was a production-grade detector that enabled real-time inference.

4.2. Lane Segmentation Training Results

Lane segmentation models were evaluated to identify optimal architectures that balance segmentation accuracy against computational efficiency for embedded deployment.

4.2.1 Dataset Characteristics

Lane segmentation training used 293 manually annotated images from QCar operations. Data augmentation expanded this to 4,395 images through rotation ($\pm 5^\circ$),

scaling (0.5–1.5×), horizontal flipping, brightness/contrast adjustment, HSV augmentation, and random cropping. The dataset was split 90/10 into training (3,956 images) and validation (439 images). Three semantic classes were defined: background, yellow centrelines, and white markings, including roundabout edges.

4.2.2 U-Net Architecture Training

U-Net served as the baseline segmentation architecture, providing reference accuracy without computational constraints.

4.2.2.1 Model Configuration

In the U-Net implementation, the architecture was symmetric, consisting of an encoder and a decoder, with skip connections. Architectural parameters were set to 31,043,651 parameters (31M), a model size of 121 MB, input and output image sizes of 256×256×3 RGB, 4 encoder blocks, a bottleneck, and 4 decoder blocks.

4.2.2.2 Training Procedure

Training configuration:

- Epochs: 100
- Batch size: 8 (GPU memory limited)
- Optimiser: Adam
- Learning rate: 0.001 with ReduceLROnPlateau scheduling
- Loss function: Cross-Entropy Loss
- Early stopping: 10 epoch patience
- Training hardware: NVIDIA GPU with 8GB memory
- Training time: Approximately 6 hours

4.2.2.3 Performance Results

Final Validation Performance (Epoch 100):

- Mean IoU (lanes only): 93.56%

- Mean IoU (all classes): 95.88%
- Per-class IoU:
 - Background: 99.76%
 - Yellow lanes: 91.07%
 - White lanes: 96.31%

The U-Net model achieved 93.56% mIoU on validation, matching training accuracy and indicating good generalisation. The learning rate decreased from 0.001 to 0.00025 by epoch 100, enabling fine-tuning near convergence. Per-class performance: background achieved 99.76% IoU, white lanes reached 96.31% IoU due to high contrast, and yellow lanes attained 91.07% IoU on the more challenging centre line detection task.

4.2.3 ENet Architecture Training

ENet provided an efficient alternative architecture designed for real-time embedded deployment.

4.2.3.1 Model Configuration

The ENet implementation used an asymmetric encoder-decoder architecture. The architecture specifications were as follows: 355,249 parameters (0.36M), 2.4MB model size, 256×256×3 RGB input/output resolution, 3 encoder stages, 2 decoder stages, and skip connections were removed in favour of factorised convolutions.

4.2.3.2 Training Procedure

Training configuration:

- Epochs: 100
- Batch size: 16 (doubled from U-Net due to smaller memory footprint)
- Optimiser: Adam
- Learning rate: 0.0005 with step decay
 - Initial: 0.0005

- Decay at epoch 95: 0.00025 (2× reduction)

- Loss function: Cross-Entropy Loss
- Weight decay: 1e-4 (L2 regularisation)
- Training hardware: Same NVIDIA GPU
- Training time: Approximately 1.4 hours (85 minutes)

4.2.3.3 Performance Results

Validation Performance: ENet resulted in 93.73% mIoU (lanes only) and 95.90% mIoU (all classes) at epoch 100. Per-class IoU: Background: 99.76%,

Notably, ENet performed equally well in terms of accuracy (93.73% vs 93.56% mIoU) with the same number of parameters as the U-Net, despite having 87 times fewer parameters. The performance of the white lane was

The larger batch size of 16 stabilised training through better gradient estimates. Step decay at epoch 95 enabled fine-tuning near convergence, while L2 regularisation prevented overfitting.

The training time was 85 minutes, a 76% reduction from U-Net's 6 hours, thanks to lower computational requirements enabled by a lighter network architecture.

4.2.4 Architecture Comparison and Analysis

Comprehensive comparison reveals the efficiency-accuracy trade-off characteristics:

Metric	U-Net	ENet	ENet Advantage
Accuracy			
Val mIoU (lanes)	93.56%	93.73%	+0.17% (better)
Yellow lane IoU	91.07%	91.07%	Equal
White lane IoU	96.31%	96.40%	+0.09% (better)
Background IoU	99.76%	99.76%	Equal
Efficiency			
Parameters	31,043,651	355,249	87× smaller

Model size	121 MB	2.4 MB	50× smaller
Training time	6.0 hours	1.4 hours	4.2× faster
Deployment			
Memory footprint	High	Low	Embedded-ready
Real-time capable	No	Yes	Suitable for distributed deployment

Table 4.3: U-Net vs ENet Architecture Comparison

ENet matched U-Net's accuracy (93.73% vs 93.56% mIoU) despite 87× fewer parameters. This resulted from the large, augmented dataset (4,395 images) preventing overfitting, dilated convolutions suited to structured lane patterns, implicit regularisation from reduced capacity, and improved gradient stability from larger batch size.

Factorised convolutions and early downsampling reduced computations without losing spatial details, while the asymmetric encoder-decoder balanced feature extraction and reconstruction.

ENet's compact size (2.4 MB vs 121 MB) enabled concurrent execution with YOLO and sensor fusion on the laptop server. Faster training (85 minutes vs 6 hours) facilitated rapid development. The system achieved 18–22 FPS end-to-end throughput for autonomous navigation.

4.3. Qualitative Detection Performance

Visual analysis of detection outputs provides insight into model behaviour across operational scenarios.

4.3.1 Simulation Environment Detection Examples

Detection examples from QLab simulation demonstrate system performance in controlled conditions.

Traffic Light Detection: The system successfully detected red and green traffic lights with confidence scores ranging from 0.60 to 0.74. Confidence scores represent the model's certainty in its detection predictions, ranging from 0 to 1, where higher values

indicate greater reliability. A threshold of 0.5 is typically applied to filter valid detections (Zou et al., 2023).



Figure 4.7 shows a red-light detection at 0.74 confidence score with the QCar approaching an intersection.

The simulation environment's consistent lighting and clean backgrounds enabled reliable detection across varying distances.

Stop Sign Detection: Stop sign detection achieved higher confidence scores ranging from 0.88 to 0.90.

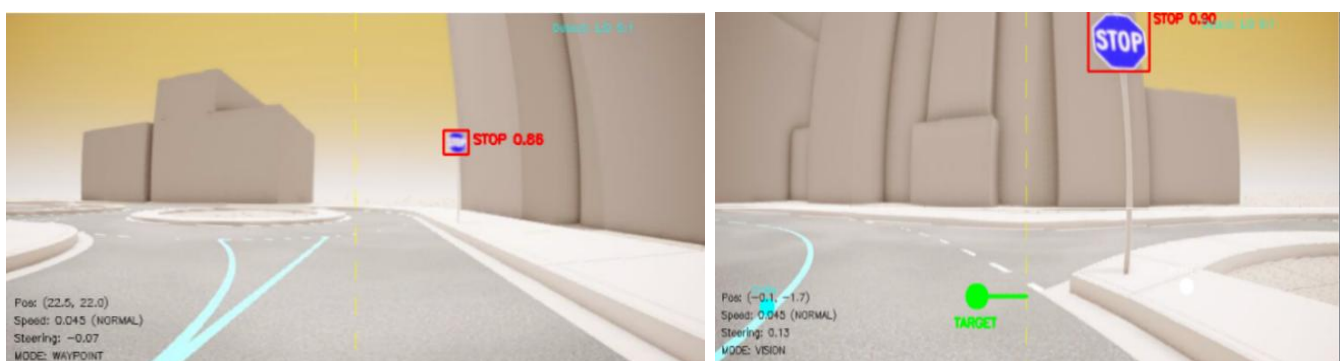


Figure 4.8: Stop sign detection in QLab simulation at confidence scores: 0.88 and 0.90, respectively.

Lane Following (HSV-based): Classical computer vision lane detection successfully identified yellow (cyan) and white lane markings.



Figure 4.9 demonstrates straight road detection with 3046 yellow and 21958 white pixels.

The consistent simulation colours enabled reliable HSV-based segmentation with the computed lane centre (green circle) positioned correctly between detected boundaries.

4.3.2 Hardware Platform Detection Examples

Real-world detection on the physical QCar demonstrates the unified 5-class model's performance under operational conditions.

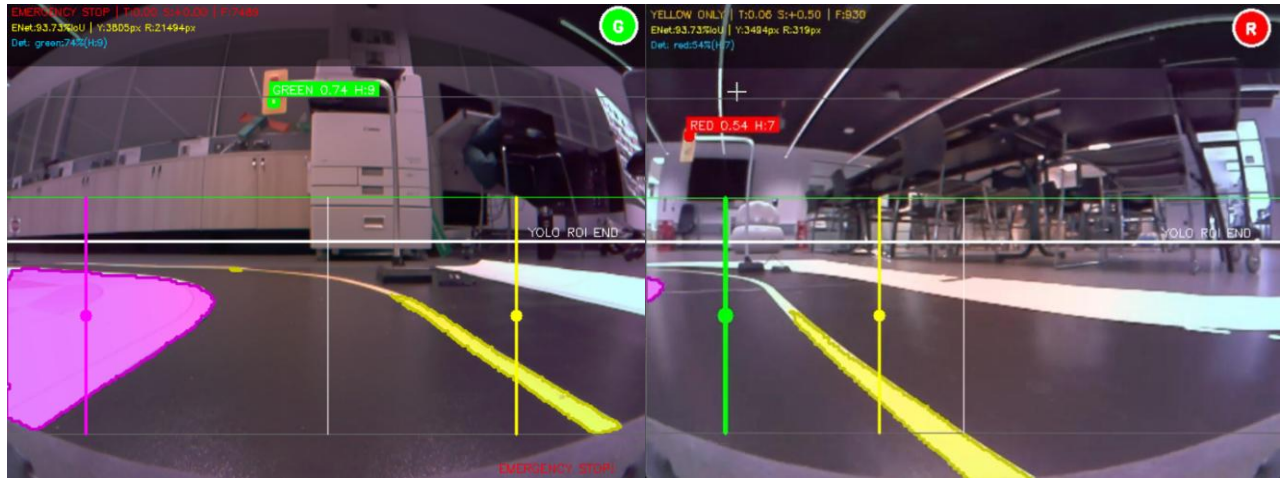


Figure 4.10: Real-world green and red traffic light detection on physical QCar (confidence scores: 0.74 & 0.54, respectively) with integrated lane segmentation

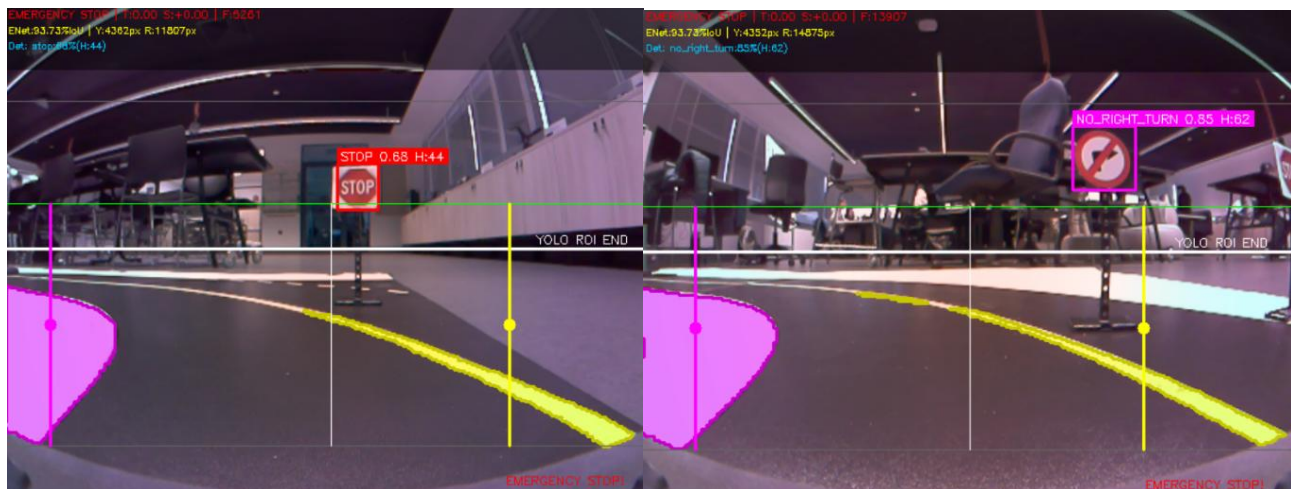


Figure 4.11: Real-world stop sign (confidence: 0.68), no-right-turn sign detection (confidence scores: 0.85) on physical QCar, demonstrating robust recognition in an operational environment

Unified model achieved 0.74-0.85 confidence scores on real QCar (Figures 4.10-4.11), successfully detecting traffic elements with simultaneous lane segmentation despite real-world challenges (lighting variation, motion blur, environmental complexity).

4.4. Robustness Testing Results

Comprehensive robustness testing quantifies system performance across environmental variations and challenging conditions.

4.4.1 Traffic Light Detection Robustness

4.4.1.1 Occlusion Testing

Traffic light detection was evaluated under progressive occlusion at a distance of 0.6m under normal lighting conditions.

Traffic Light Class	0% Occlusion	25% Occlusion	50% Occlusion	75% Occlusion
Red Light	0.74	0.42	0.39	Failed
Green Light	0.73	0.44	Failed	Failed
Yellow Light	0.65	0.36	Failed	Failed

Table 4.4: Traffic Light Detection - Occlusion Robustness (at 0.6m). Values represent YOLO confidence scores (0-1 range). "Failed" indicates no detection.

Red lights demonstrated the highest occlusion robustness, maintaining detection (albeit with reduced confidence) up to 50% occlusion. Green and yellow lights failed at 50% occlusion, likely due to their smaller apparent size and reduced colour distinctiveness when partially obscured. All traffic light classes failed at 75% occlusion, representing the practical detection limit under severe partial visibility.

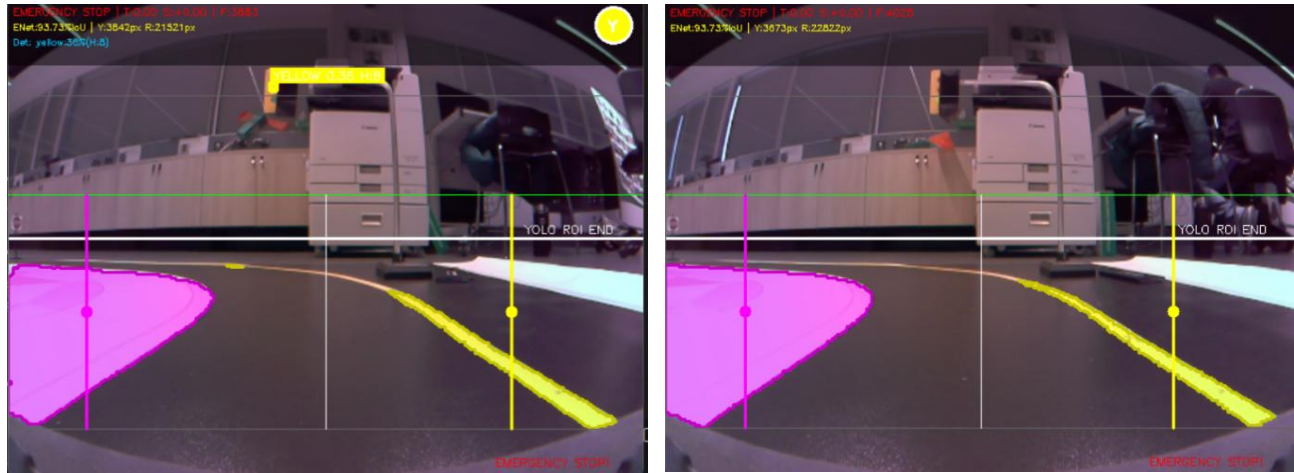


Figure 4.12: Traffic light detection under progressive occlusion (25% and 75% at 0.6m distance)

The confidence degradation pattern revealed exponential decay rather than linear reduction. From 0% to 25% occlusion, confidence dropped 43% for red lights, 40% for green lights, and 45% for yellow lights. The relatively consistent degradation across classes suggests occlusion affects detection through reduced visible area rather than class-specific features.

4.4.1.2 Distance Robustness Testing

Detection performance was evaluated across distances from 0.4m to 1.0m to characterise the operational range.

Traffic Light	0.4m	0.5m	0.6m	0.7m	0.8m	0.9m	1.0m
Red Light	0.74	0.79	0.52	0.47	0.51	0.72	0.42
Green Light	0.82	0.74	0.48	0.54	0.54	0.62	0.57
Yellow Light	0.75	0.76	0.47	0.43	0.63	0.65	0.43

Table 4.5: Traffic Light Detection - Distance Robustness. Values represent YOLO confidence scores (0-1 range) at different distances from the traffic light.

Detection remained reliable across the entire 0.4-1.0m range, though confidence varied significantly. Green lights showed the highest close-range confidence at 0.82 (0.4m), while all classes exhibited a performance dip in the mid-range (0.6-0.7m). This

non-monotonic pattern likely reflects the training data distribution, with more examples at very close and moderate distances than at mid-range positions.

Recovery at 0.9m suggests an optimal detection window where traffic lights occupy favourable proportions of the image frame. The final confidence at 1.0m remained above 0.40 for all classes, indicating viable detection for autonomous navigation decision-making.

4.4.1.3 Lighting Condition Robustness

Traffic light detection was tested under four lighting conditions at fixed 0.6m distance.

The four circumstances include Normal (standard fluorescent lighting), Bright (enhanced LED lighting), Dim (about 30% intensity), and Dark (less than 5% intensity). Tests were done on the full circuit.

Traffic Light Class	Bright (120%)	Normal (100%)	Dim (30%)	Dark (<5%)
Red Light	0.80	0.81	0.79	0.48
Green Light	0.72	0.77	0.76	0.38
Yellow Light	0.76	0.74	0.76	0.32

Table 4.6: Traffic Light Detection under lighting conditions tested at a fixed 0.6m distance. Values represent YOLO confidence scores (0-1 range)

Performance remained excellent (confidence >0.72) across bright, normal, and dim conditions, validating HSV augmentation. Dark conditions caused degradation (0.32-0.48), with red lights most robust (0.48) due to high-intensity illumination and yellow lights weakest (0.32).

4.4.2 Traffic Sign Detection Robustness

4.4.2.1 Stop Sign and No-Right-Turn Distance Testing

Traffic sign detection performance was evaluated across the operational distance range.

Sign Type	0.2m	0.4m	0.6m	0.8m
Stop Sign	0.71	0.54	0.37	0.31

No-Right-Turn	0.88	0.84	0.38	Failed
---------------	------	------	------	--------

Table 4.7: Traffic Sign Detection - Distance Robustness. Values represent YOLO confidence scores (0-1 range) at different distances.

The stop signs were detected regularly across the 0.2-0.8m range, with confidence steadily decreasing. Strong detection at short distances (0.71 at 0.2m) allowed for quick spotting, while detection was still possible at 0.8m with a confidence of 0.31.

No-right-turn signs were very strong in the close-distance subset, with a value of 0.88 at 0.2 meters, but rapidly degraded beyond 0.4 meters. Incomplete detection at 0.8 meters is probably related to the reduced physical dimensions and lack of distinctive circular marking in comparison to the octagonal stop sign. The degree of confidence is 0.38 at 0.6 meters.

4.4.2.2 Occlusion Robustness

Traffic signs were tested under progressive occlusion at 0.3m distance.

Sign Type	0% Occlusion	25% Occlusion	50% Occlusion	75% Occlusion
Stop Sign	0.78	0.63	0.33	Failed
No-Right-Turn	0.85	0.69	0.51	Failed

Table 4.8: Stop Sign & No-Right-Turn - Occlusion Robustness tested at 0.3m distance. Values represent YOLO confidence scores (0-1 range).

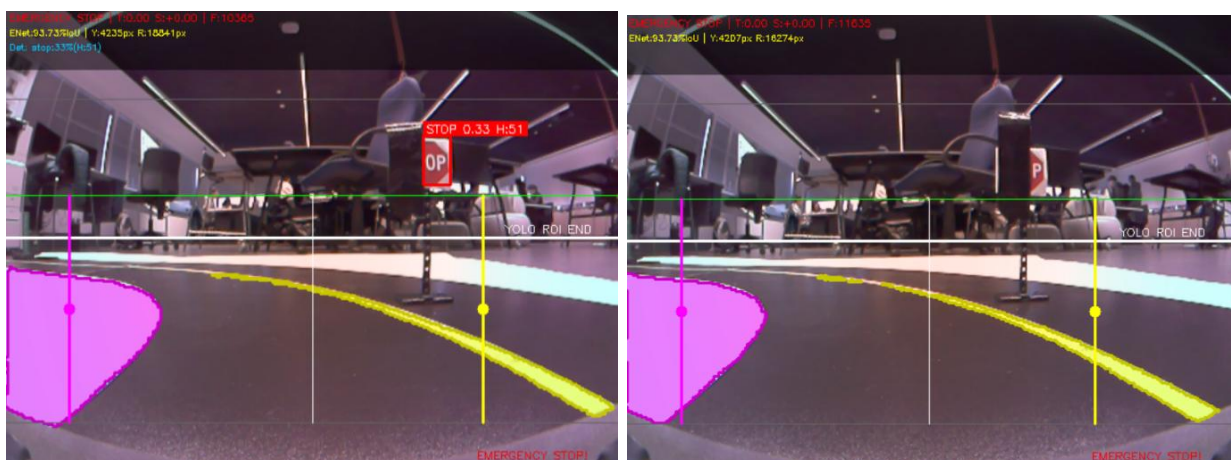


Figure 4.13: Traffic stop sign detection under progressive occlusion (50% and 75% at 0.3m distance)

The no-right-turn signs were found to be more resistant to occlusions than the stop signs, retaining 0.51 confidence levels even when 50% of the sign was occluded, compared with 0.33 for the stop signs. This may be because the circle still has more distinctive curves than the octagon's sharp edges.

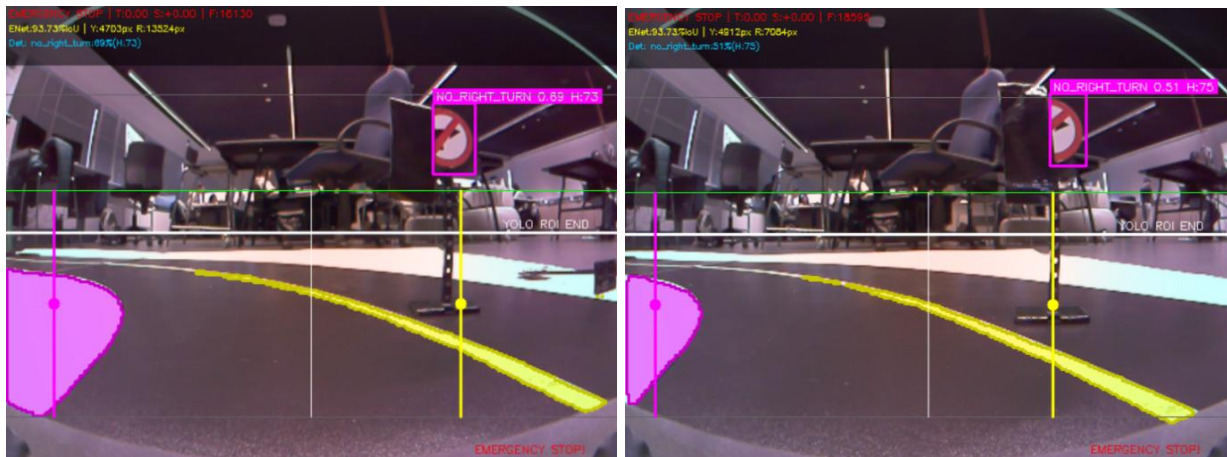


Figure 4.14: Traffic No-right-turn sign detection under progressive occlusion (25% and 50% at 0.3m distance)

Both sign types failed at 75% occlusion, which matched the traffic light result and thus represented the practical limit of visual recognition. Confidence degradation followed roughly the exponential decay, with a 19% drop for stop and 19% drop for no-turn from 0% to 25% occlusion, accelerating to 48% and 26%, respectively, from 25% to 50% occlusion.

4.4.2.3 Lighting Condition Robustness

Traffic signs were evaluated across four lighting conditions at a distance of 0.3m.

Sign Type	Bright (120%)	Normal (100%)	Dim (30%)	Dark (<5%)
Stop Sign	0.77	0.79	0.59	0.76
No-Right-Turn	0.87	0.88	0.83	0.78

Table 4.9: Stop Sign & No-Right-Turn - Lighting Conditions Lighting conditions tested at 0.3m distance. Values represent YOLO confidence scores (0-1 range)



Figure 4.15: Traffic sign detection under dark lighting conditions (stop: 0.59, no-turn: 0.84) at 0.3m distance

Stop signs maintained robust performance across all lighting conditions (0.59-0.79), with dark performance (0.76) likely enhanced by retroreflective materials under vehicle headlight illumination. No-right-turn signs showed consistent performance (0.78-0.88) with minimal dark degradation (0.78).

4.4.3 Lane Segmentation Robustness

4.4.3.1 Occlusion Impact on Segmentation

Lane segmentation performance was evaluated under progressive occlusion with the vehicle stationary.

Model	0% Occlusion	25% Occlusion	50% Occlusion	75% Occlusion	90% Occlusion	100% Occlusion
U-Net	5031 / 11058	4476 / 8311	6150 / 523	3577 / 0	1856 / 0	0 / 0
ENet	4951 / 10818	4166 / 7897	3470 / 3842	3026 / 512	2495 / 0	0 / 0

Table 4.10: Lane Segmentation - Occlusion Robustness (Stationary Vehicle). Values shown as yellow lane pixels / Roundabout (white) pixels detected.

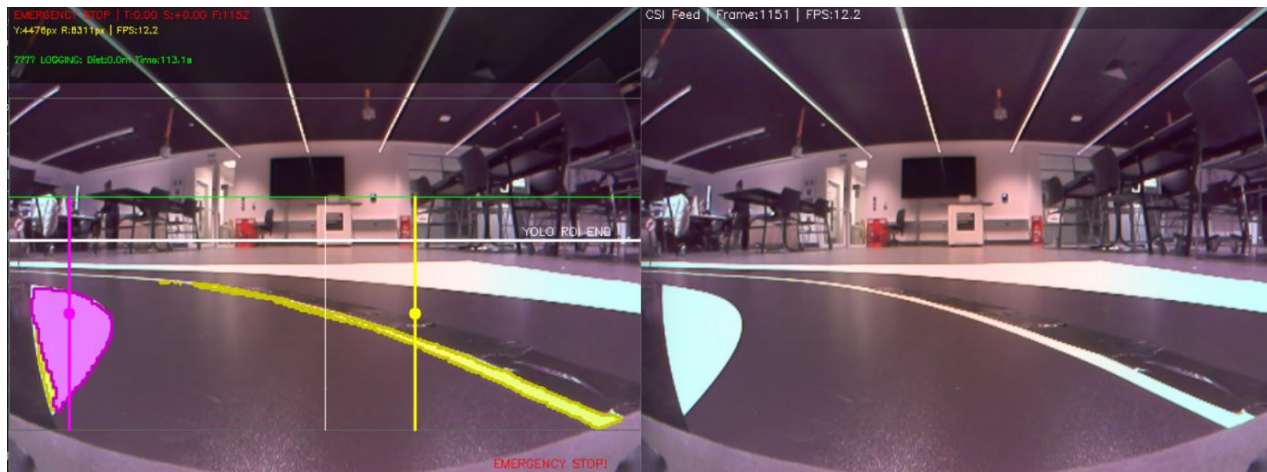


Figure 4.16: U-Net Lane segmentation at 25% occlusion (left: segmentation output, right: camera view)



Figure 4.17: Enet Lane segmentation at 90% occlusion (left: segmentation output, right: camera view)

At 0% occlusion, both models detected approximately 5,000 yellow-lane pixels and 10,000–11,000 roundabout pixels. Roundabout detection degraded faster than yellow lanes, U-Net lost all roundabout pixels by 75% occlusion, whereas ENet retained 512. Yellow-lane pixels persisted to 90% occlusion for both models. ENet showed stronger mid-level retention: 3,842 roundabout pixels at 50% occlusion compared with 523 pixels for U-Net. Both models failed at 100% occlusion.

4.4.3.2 Lighting Condition Impact

Segmentation performance was tested across lighting conditions with a stationary vehicle. Pixel counts for yellow and white (roundabout) lane detection for both U-Net and Enet are stated below,

U-Net Performance:

Lighting Condition	Yellow Lane Pixels	Roundabout Pixels
Bright (120%)	5235	10191
Normal (100%)	5338	10122
Dim (30%)	4675	2559
Dark (<5%)	2795	54

Enet Performance:

Lighting Condition	Yellow Lane Pixels	Roundabout Pixels
Bright (120%)	5280	7471
Normal (100%)	5287	7485
Dim (30%)	4896	7441
Dark (<5%)	3287	728

Table 4.11: Lane Segmentation performance values under different lighting Conditions.

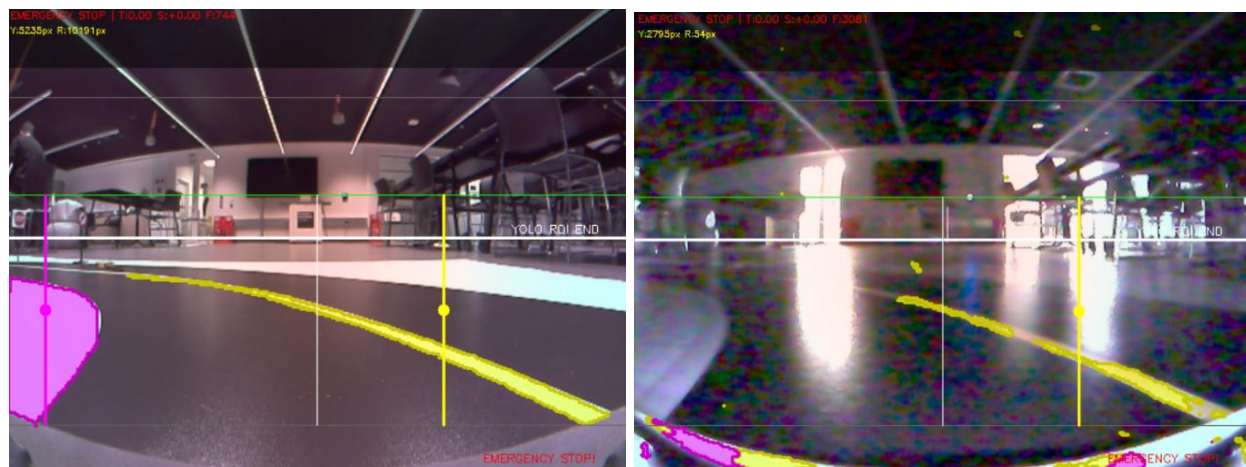


Figure 4.18: U-Net segmentation performance across bright and dark lighting conditions.

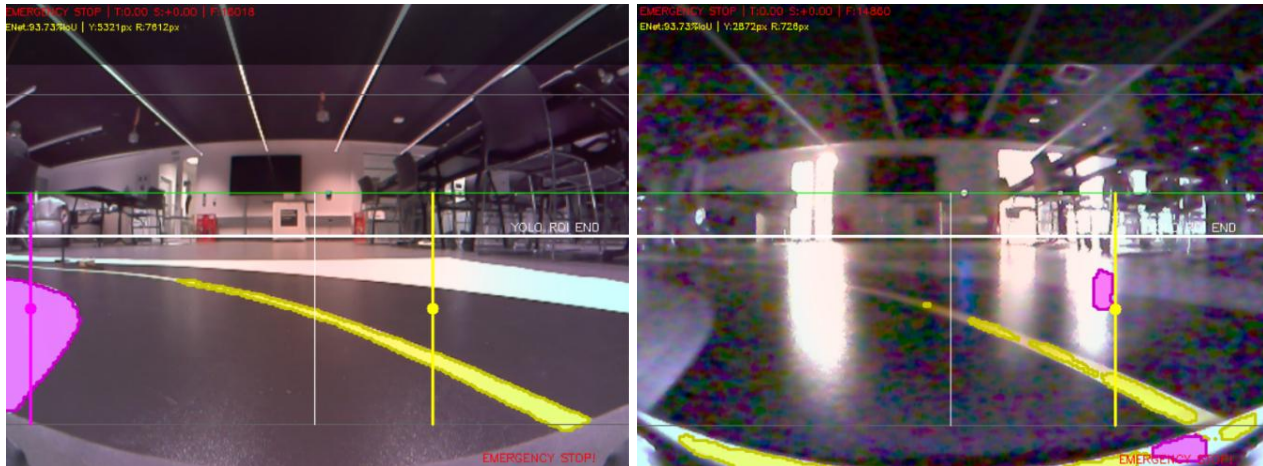


Figure 4.19: Enet segmentation performance across bright and dark lighting conditions.

Both models achieved 5200-5300 yellow lane pixels in bright/normal conditions. U-Net detected more roundabout pixels (~10000 vs ~7400). Dark conditions caused major degradation. U-Net's roundabout detection dropped to 54 pixels (99.5% reduction) and yellow lanes 48%. ENet proved more robust: dim lighting (7441 vs 2559 roundabout pixels), dark conditions (728 vs 54), yellow lanes (38% vs 48% degradation).

4.4.4 Obstacle Detection System Robustness

The obstacle detection system employed sensor fusion between Intel RealSense depth camera and RPLidar for comprehensive environment monitoring.

4.4.4.1 RealSense Distance and Size Performance

RealSense depth camera detection was evaluated across object sizes and distances, with consistent performance across all lighting conditions.

Object Size	0.2m	0.4m	0.6m	0.8m	1.0m	1.2m	Lighting Independent
Large	✓	✓	x	x	x	x	✓
Medium	✓	✓	x	x	x	x	✓
Small	✓	✓	x	x	x	x	✓
Extra Small	x	✓	x	x	x	x	✓

Table 4.12: Obstacle Detection (RealSense) – Distance & Size Performance.” P” = Successfully detected, “X” = Failed detection. “Lighting Independent” indicates identical performance across Bright, Normal, Dim, and Dark conditions.

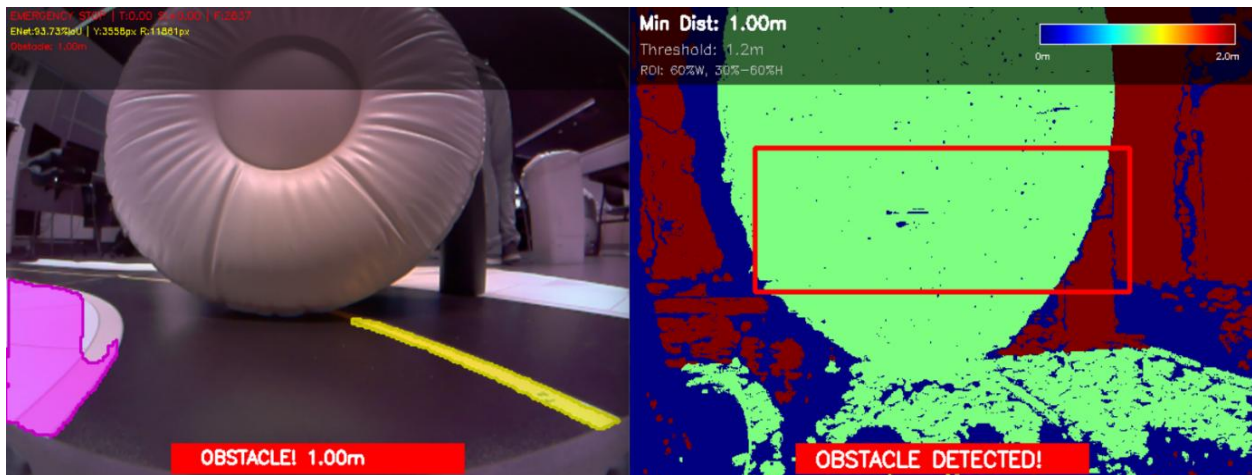


Figure 4.20: RealSense depth camera successful obstacle detection at 0.4m maximum effective range (large object, active IR sensing)

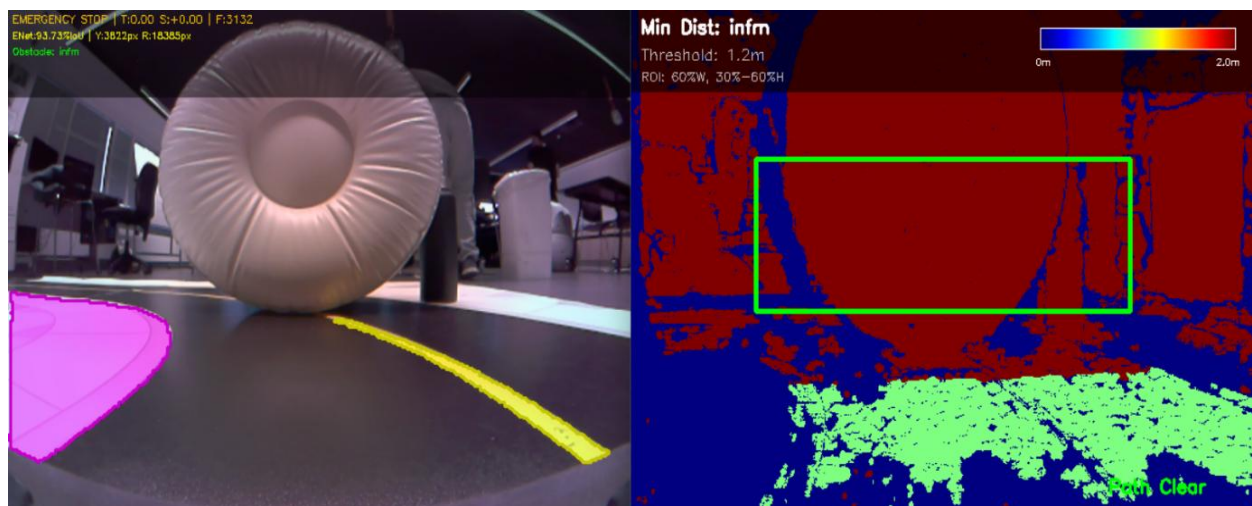


Figure 4.21: RealSense detection failure at 0.6m distance (same large object), demonstrating sharp performance cutoff beyond effective operating range.

RealSense showed no lighting dependence, achieving the same level of detection across bright, general, dim, and dark lighting conditions. This verifies the underlying active sensing technology: it emits IR light; it does not depend on ambient lighting,

unlike vision sensing. The fact that the range is 0.2-0.4m in all lighting conditions is evidence that the limitations are due to range, not lighting conditions.

4.4.4.2 LiDAR Distance and Size Performance

LiDAR sensor detection was evaluated using identical test protocols, with consistent performance across all lighting conditions.

Object Size	0.2m	0.4m	0.6m	0.8m	1.0m	1.2m	Lighting Independent
Large	✓	✓	✓	✓	✓	✓	✓
Medium	✓	✓	✓	✓	✓	✓	✓
Small	✓	✓	✓	✓	✓	✓	✓
Extra Small	x	x	x	x	x	x	✓

Table 4.13: Obstacle Detection (LiDAR) - Distance & Size Performance." P" = Successfully detected, " x " = Failed detection. "Lighting Independent" indicates identical performance across Bright, Normal, Dim, and Dark conditions.



Figure 4.22: LiDAR obstacle detection of a medium sized image at 0.6m distance, demonstrating superior range compared to RealSense 0.4m maximum



Figure 4.23: LiDAR obstacle detection of a medium sized image at 1.2m tested distance showing 3× superior range compared to RealSense (0.4m)

LiDAR also had flawless lighting independence for all scenarios. Large, medium, and small objects were detected 100% at all ranges (0.2-1.2m) irrespective of lighting levels, solidifying that LiDAR's limitations are related to the size and material properties of objects, not lighting.

4.4.4.3 Sensor Fusion Performance

The AND-logic sensor fusion strategy was validated through systematic testing across object sizes and distances.

Object Size	0.2m	0.4m	0.6m	0.8m	1.0m	1.2m
Large	Both	Both	Both	None	None	None
Medium	Both	Both	LiDAR Only	None	None	None
Small	Both	Both	LiDAR Only	None	None	None
Extra Small	None	RealSense Only	None	None	None	None

Table 4.14: Combined Sensor Fusion (RealSense + LiDAR) – Distance & Size. “Both” = Both sensors detected (AND-logic trigger), “LiDAR Only” = Only LiDAR detected (no trigger), “RealSense Only” = Only RealSense detected (no trigger), “None” = Neither sensor detected.

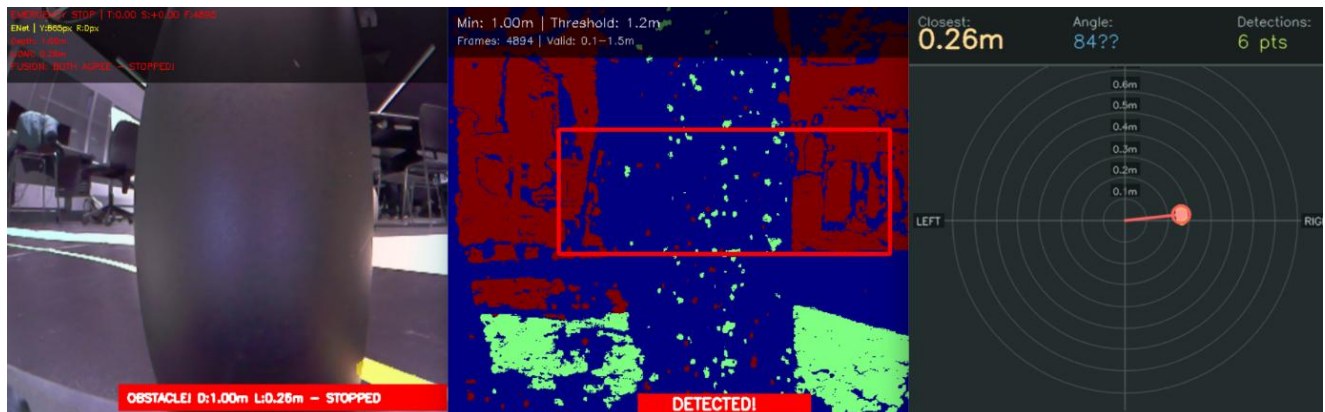


Figure 4.24: Sensor fusion at 0.2m showing successful dual-sensor detection of a small sized image with emergency brake trigger (left: camera, centre: RealSense depth, right: LiDAR 0.26m detection)

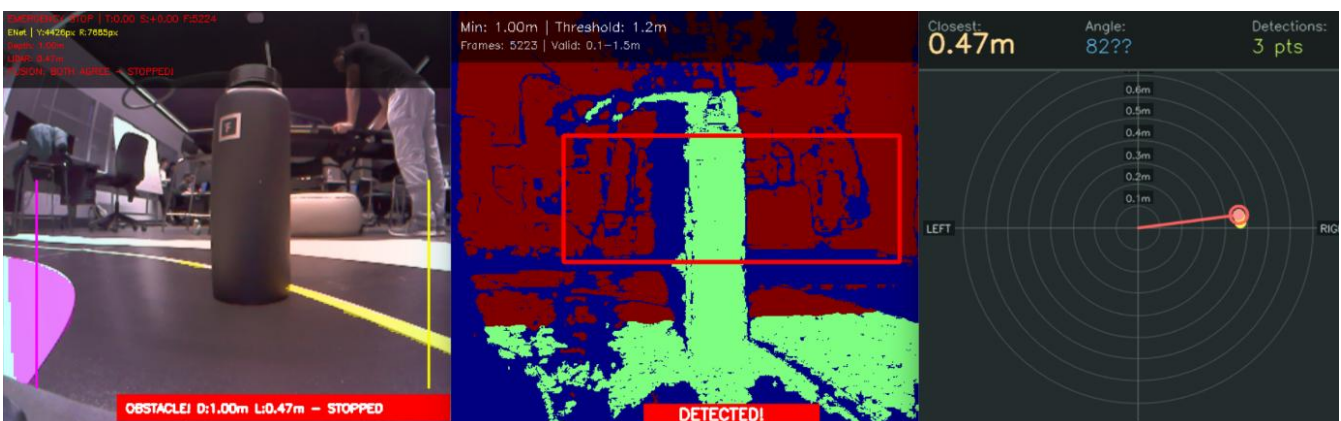


Figure 4.25: Sensor fusion at 0.4m (small object) at RealSense maximum range, demonstrating consensus zone limit for emergency braking (left: camera, centre: RealSense depth, right: LiDAR 0.47m detection).

Results demonstrate the consensus zone extended to 0.2–0.6m for large obstacles, where both RealSense and LiDAR achieved reliable detection. Medium and small obstacles exhibited narrower consensus (0.2–0.4m), constrained by RealSense's effective range. Beyond the consensus zone, LiDAR detected obstacles to its 0.8m threshold but could not trigger braking without RealSense confirmation.

The AND-logic approach achieved zero false positive emergency stops while maintaining reliable detection within the consensus range. Large objects benefited from extended consensus due to improved depth measurement on larger surfaces.

Extra-small objects presented a safety gap at 0.4m, where RealSense detected without LiDAR confirmation, preventing brake activation.

4.5. Lane Following Performance

Lane-following performance was systematically evaluated to identify optimal operating parameters and compare architectural performance between U-Net and ENet segmentation models.

4.5.1 Speed Optimisation Results

Both U-Net and ENet architectures were evaluated across four throttle settings (0.055, 0.060, 0.065, 0.070) under normal lighting conditions on the full circuit track. The QCar executed autonomous lane-following using Pure Pursuit control with Kalman filtering for smooth steering commands.

Model	Speed	Distance (m)	Mean Offset (px)	Max Offset (px)	Yellow Detection %	Violations	Status
U-Net	0.055	0.65	101	158	100.0	0	Stalled
	0.060	2.62	108	216	100.0	0	Optimal
	0.065	1.77	179	393	98.9	148	Degraded
	0.070	0.64	152	409	88.8	58	Failed
ENet	0.055	0.77	105	157	100.0	0	Stalled
	0.060	2.09	105	223	100.0	0	Optimal
	0.065	1.88	134	296	100.0	58	Degraded
	0.070	0.80	213	480	72.2	80	Failed

Table 4.15: Lane Following Performance - U-Net vs ENet Speed Comparison

Throttle setting 0.060 emerged as optimal for both architectures, balancing distance travelled against lane-keeping accuracy. U-Net achieved 2.62m distance (25% longer than ENet's 2.09m). However, ENet demonstrated superior mid-speed robustness with 61% fewer boundary violations at speed 0.065 (58 vs 148 violations).

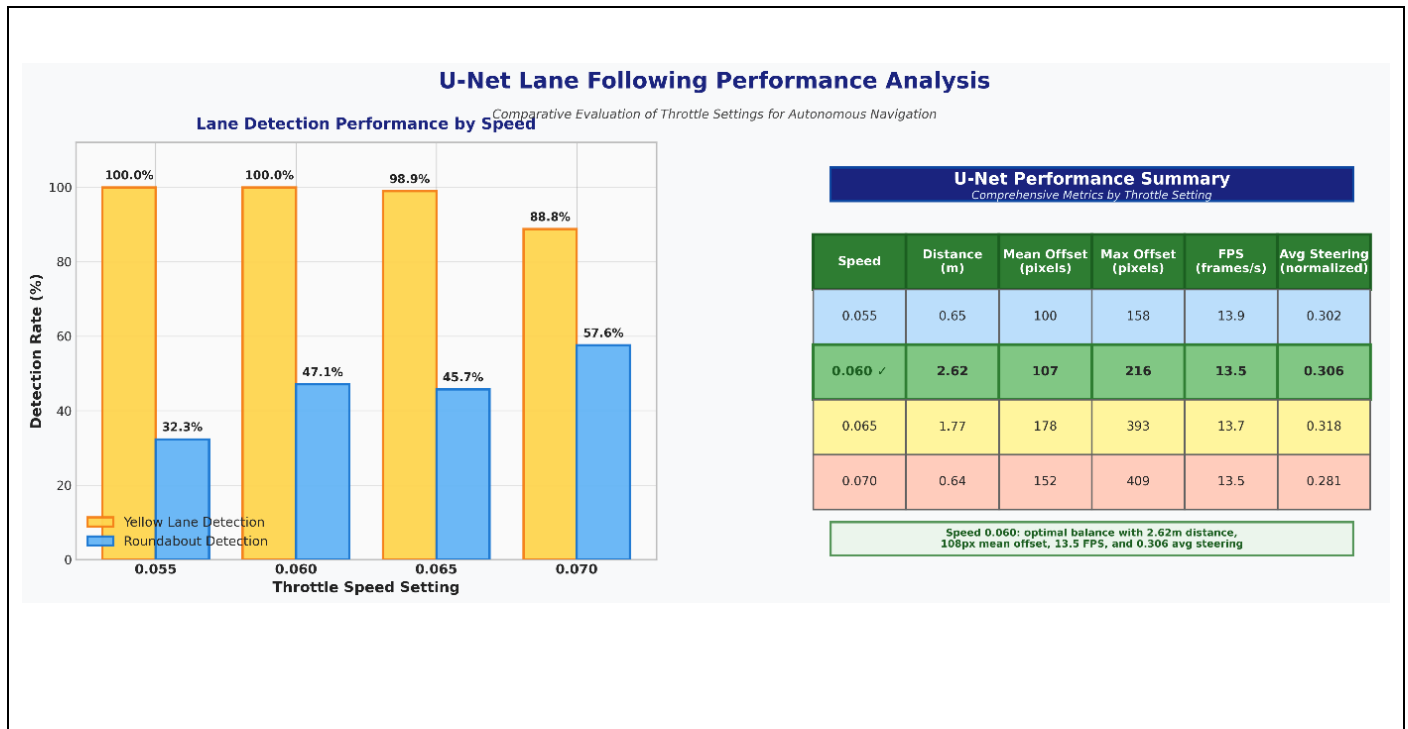


Figure 4.26: U-Net Lane following performance across throttle settings (0.055-0.070) showing optimal speed 0.060 with 2.62m distance, 108px mean offset, perfect yellow detection (100%) and zero boundary violations

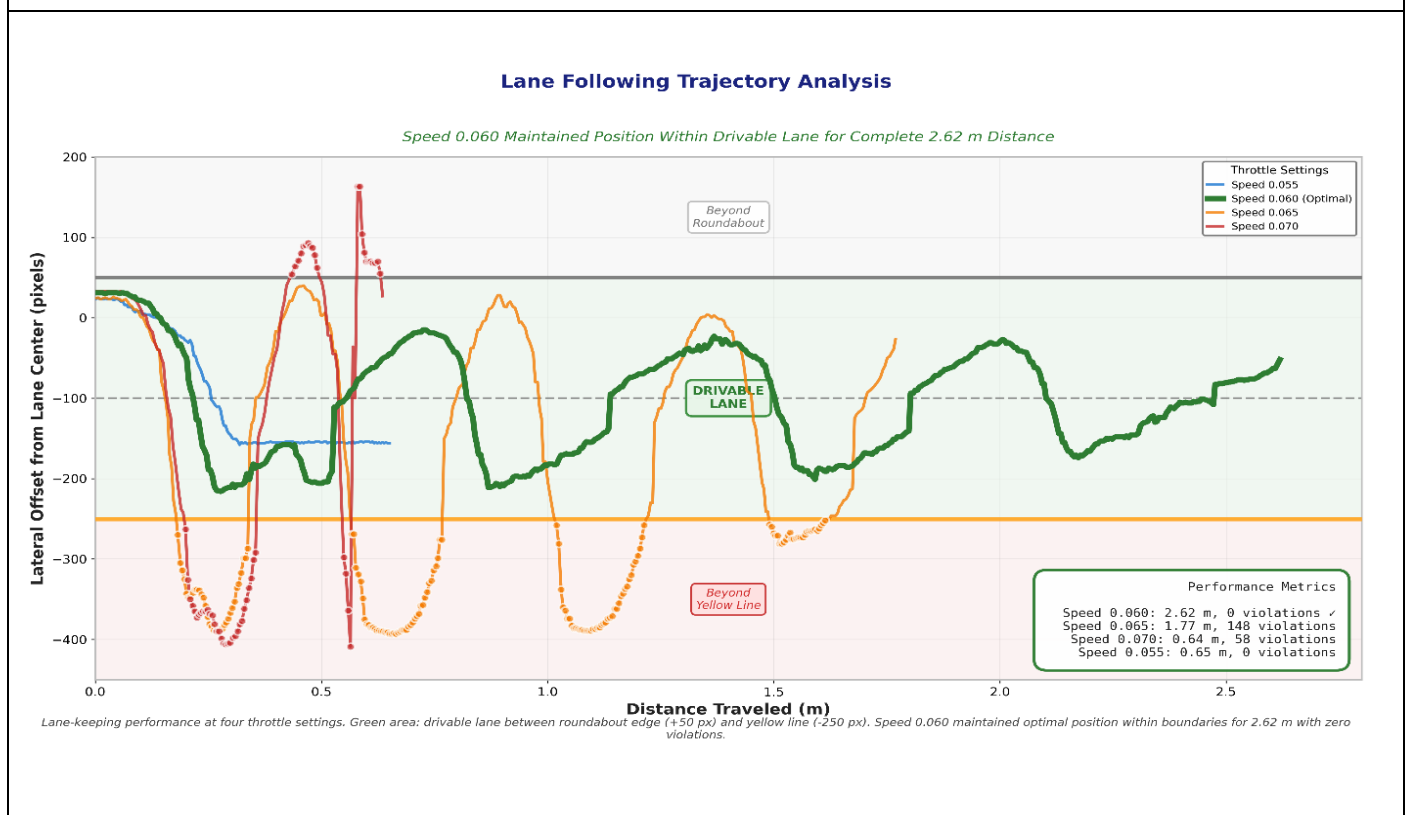


Figure 4.27: U-Net trajectory comparison across throttle settings, showing speed 0.060 (green) maintaining drivable lane boundaries for 2.62m with zero violations, versus degraded performance at 0.065 (148 violations) and failure at 0.070.

The U-Net achieved 100% yellow lane detection at 0.060, then degraded to 98.9% at 0.065, and finally to 88.8% at 0.070. The ENet maintained a detection rate of 100% over a wider speed range (0.055-0.065) before failing at 0.070, at which it detected only 72%. U-Net's 25% distance advantage (2.62m vs 2.09m) stems from superior temporal consistency enabled by skip connections, resulting in smoother control.

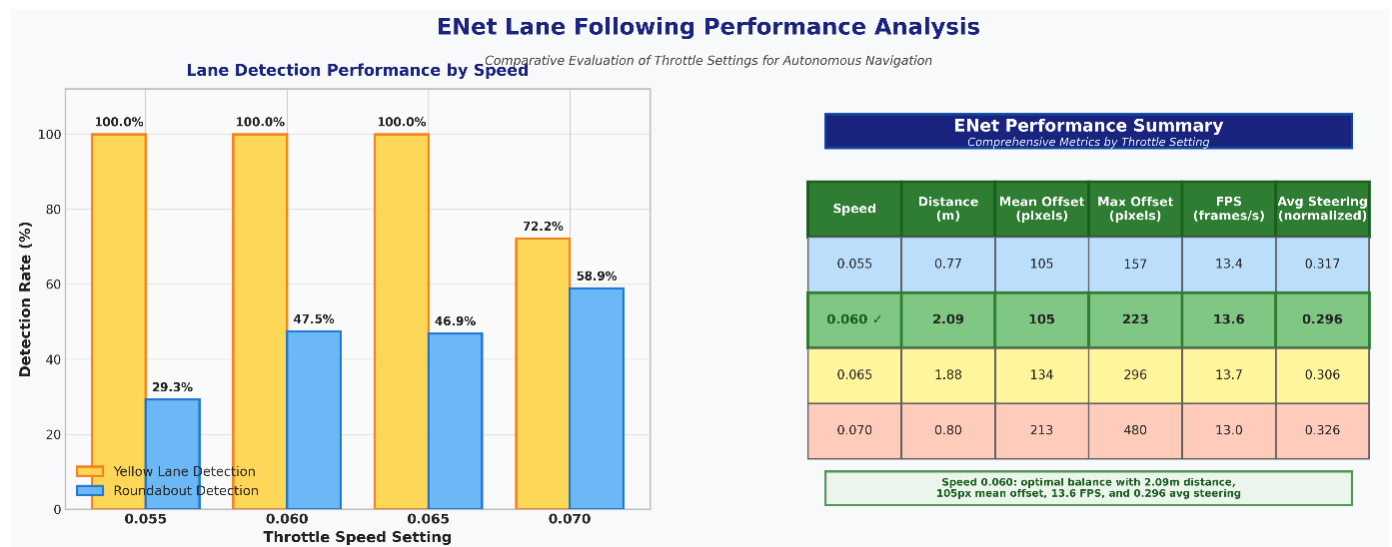
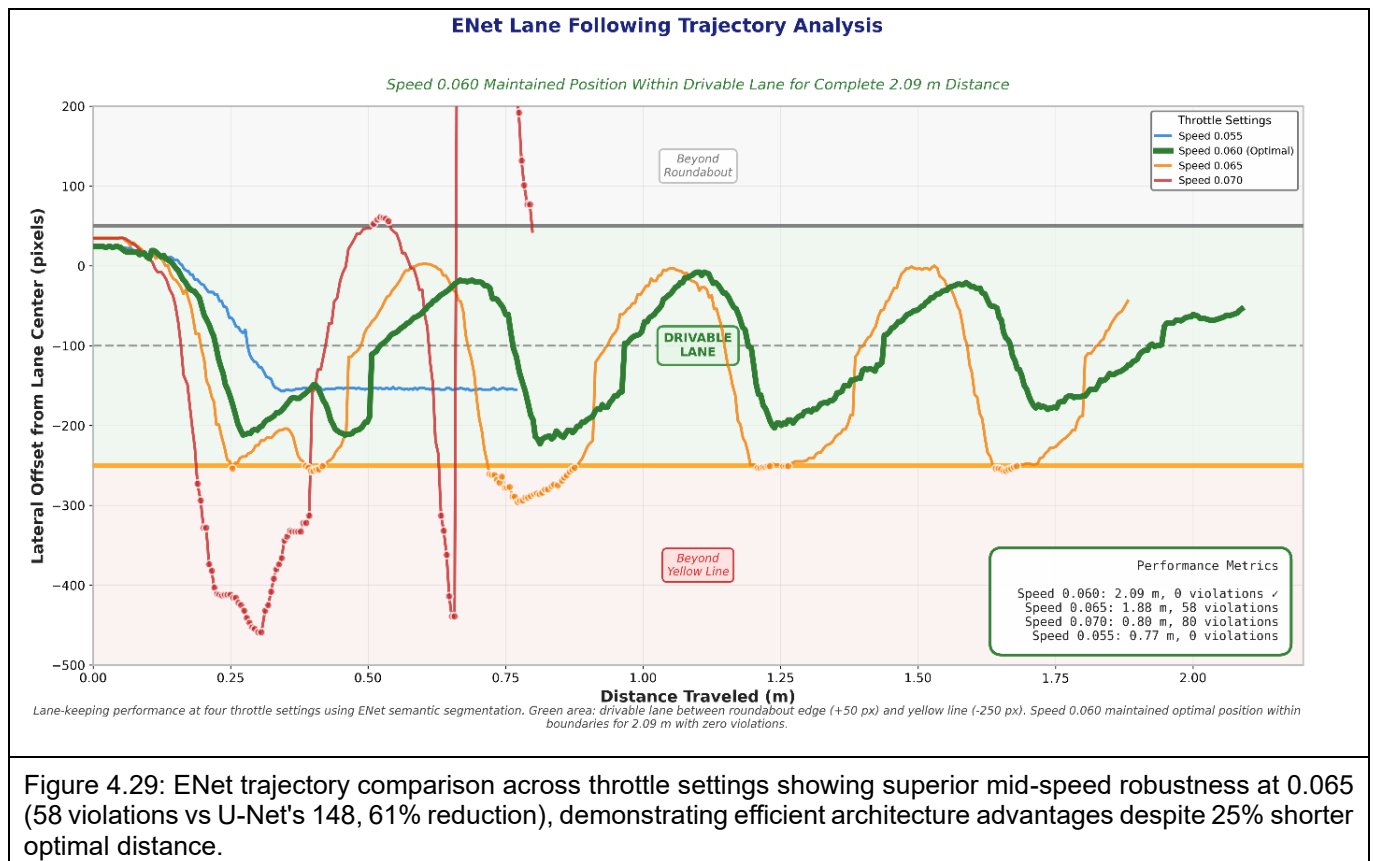


Figure 4.28: ENet lane following performance across throttle settings, demonstrating superior detection robustness with 100% yellow lane detection maintained through speed 0.065 (vs U-Net's 98.9%) and optimal 2.09m distance at 0.060.



The inefficacy of ENet at speed 0.070 indicates that the efficient model performs near-perfection with high blur (factorised convolutions and spatial reduction failing to provide sufficient capacity to preserve segmentation accuracy above a certain threshold). The lack of distance disadvantage in ENet's accuracy (2.09m, zero errors) confirms its reliability for autonomous control. The 87% parameter reduction normalises a 25% compromise on distance.

4.5.2 Lighting Condition Robustness

The robustness to lane-following was tested under four lighting conditions and at a fixed speed of 0.060 to assess sensitivity to lighting and to speed.

Condition	Distance (m)	Mean Offset (px)	Max Offset (px)	Yellow Detect %	Roundabout Detect %	Status
Normal	2.53	109	245	100.0	48.6	Success
Bright	2.50	118	247	100.0	47.4	Success
Dim	2.62	114	234	100.0	44.8	Success
Dark	0.00	0	0	69.8	36.9	Failed

Table 4.16: U-Net Lighting Condition Performance at speed 0.060.

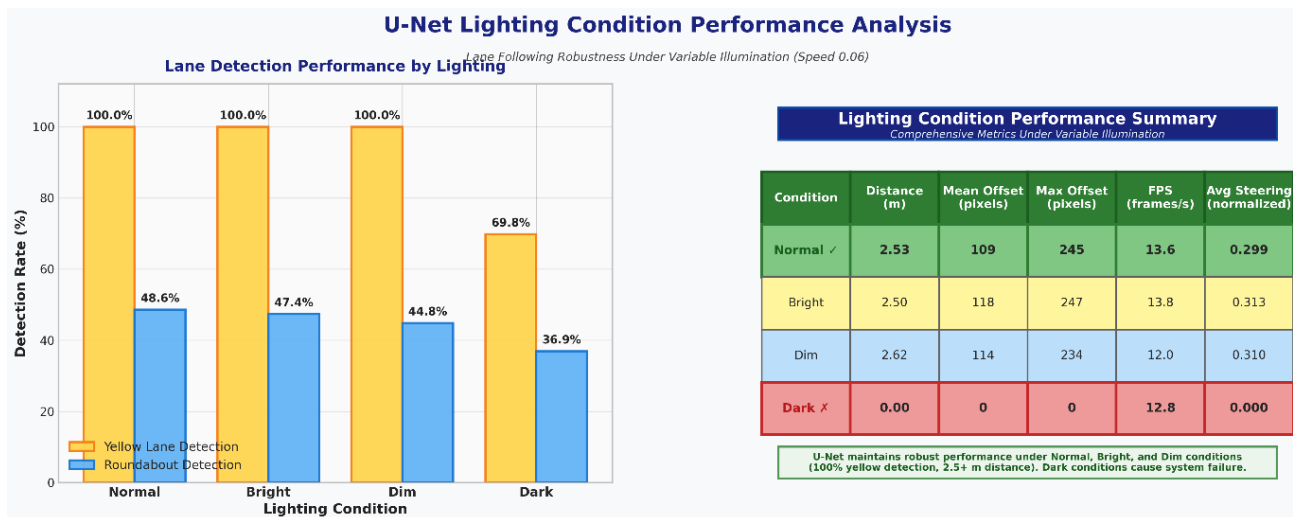


Figure 4.30: U-Net Lane following performance across lighting conditions showing perfect yellow lane detection (100%) under Normal/Bright/Dim with complete dark failure (0m, 69.8% detection).

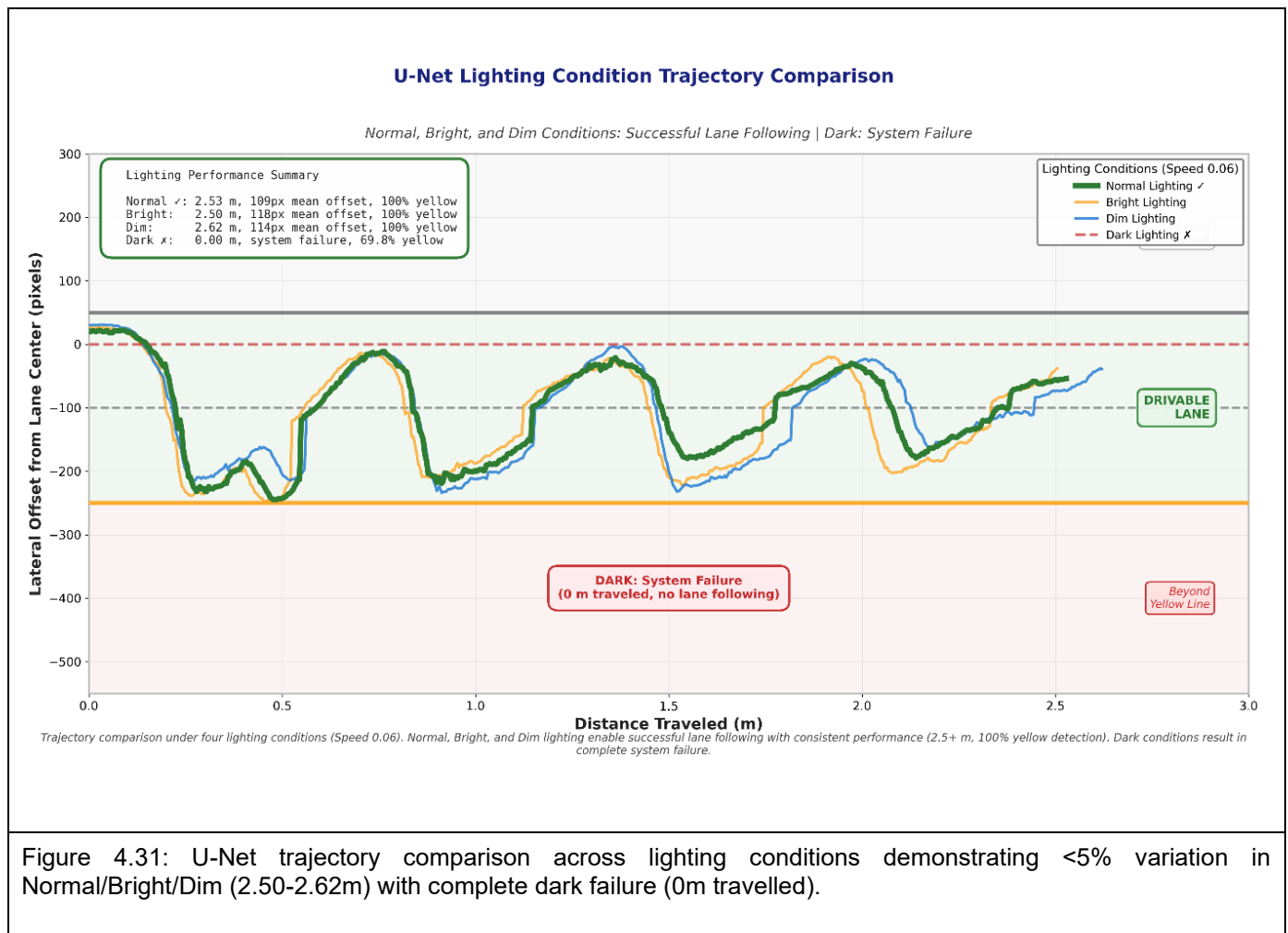


Figure 4.31: U-Net trajectory comparison across lighting conditions demonstrating <5% variation in Normal/Bright/Dim (2.50-2.62m) with complete dark failure (0m travelled).

Normal, Bright, and Dim lighting conditions had stable performances (2.50-2.62m, 5% variation). Dim lighting yielded a maximum distance of 2.62m with 8% improvement in mean offset error (114 vs 118 pixels). Yellow lane detection remained at 100% for all lighting conditions. The dark environment resulted in a total failure distance of 0m, and the yellow lane detection performances dropped to 69.8%, and roundabout to 36.9%

4.5.3 Lane Following Performance Summary

The optimal throttle value for both architectures is 0.060 with 2.09–2.62m and an offset of 105–108px with no violations. The U-Net resulted in a longer distance of 25% (2.62m) compared to 2.09m for optimal throttle values, with fewer violations, and ENet with a throttle value of 0.065 for mid-speed scenarios. Under Normal, Bright, and Dim lighting conditions, the U-Net performed well, but under Dark conditions, it performed poorly, whereas ENet performed better in low-light conditions.

4.6. Chapter Summary

Object detection training resulted in 99.44% mAP@0.5 on traffic lights and 99.50% on stop signs in simulation, compared to 92.87% mAP@0.5 on the hardware. A 6.57 percentage point sim-to-real gap was observed. In the case of lane segmentation, ENet provided the same accuracy as U-Net at 93.73% versus 93.56% m

Robustness tests showed that traffic detection was robust to 50-75% occlusion at 0.4-1.0m, active sensors (RealSense and LiDAR) remained lighting-independent, and ENet consistently outperformed in darkness. The AND-logic sensor fusion identified a 0.2-0.6m consensus.

Optimal performance in lane following was at a speed of 0.060 throttle setting, where U-Net increased distance by 25% (2.62m against 2.09m), while ENet was more robust in mid-range scenarios (61% fewer violations). Both Networks performed uniformly across all lighting conditions, including bright, dim, and normal, but failed in dark environments.

Results demonstrate the system's suitability for controlled environments with defined boundaries. Chapter 5 explores the results and compares them to those found in the literature.

5. DISCUSSION

This chapter presents the interpretation of experimental results, the analysis of the transfer from simulation to reality, and the evaluation of the practical implications. The discussion of key findings encompasses object detection, lane segmentation, and sensor fusion; quantitative work on reality gap; hardware integration challenges; and critical assessment of research limitations.

5.1. Interpretation of Key Findings

5.1.1 Object Detection Performance

Traffic light and stop sign detection achieved near-perfect simulation performance (99.44–99.50% mAP@0.5), establishing a robust baseline for sim-to-real comparison. The controlled QLab environment with consistent lighting and unoccluded targets enabled optimal performance.

Stop sign detection demonstrated 67% superior localisation quality compared to traffic lights (86.12% vs 51.46% mAP@0.5-0.95). This reflects three factors: distinctive octagonal geometry providing unique shape features, larger apparent size enabling precise bounding box regression, and high-contrast red colouring facilitating robust discrimination.

Hardware Deployment and Reality Gap

The unified five-class model achieved 92.87% mAP@0.5 on hardware, representing a 6.57 percentage point degradation from the simulation baseline (99.44% traffic lights, 99.50% stop signs; Table 4.2). This gap falls within published benchmarks where vision-based autonomous systems report 5-10% degradation when transitioning from simulation to physical deployment without domain adaptation techniques (Zhao et al., 2020; Peng et al., 2018).

The confusion matrix (Figure 4.6) revealed background misclassification as the primary error source, particularly affecting red lights (42% false positive rate). Real-world urban environments introduce visual complexity, which aligns with autonomous vehicle perception literature where urban visual clutter represents a fundamental detection challenge (Janai et al., 2020).

The obtained 92.87% mAP is enough for controlled educational environments, while for safety-critical deployment, it would be necessary to improve the results. Production systems use redundant sensing, extensive validation, and formal safety certification, which is outside the scope of this work. Any future safety-critical work would require domain adaptation, more diversity in training data across weather and lighting conditions, and multi-sensor redundancy.

5.1.2 Lane Segmentation Architecture Comparison

ENet achieves comparable validation accuracy (93.73% vs 93.56% mIoU, Table 4.3) despite 87× parameter reduction (355K vs 31M) challenging conventional assumptions equating model capacity with performance.

Explaining ENet's Superiority

Three mechanisms explain this result. First, capacity constraints forced learning of robust features rather than memorising patterns—the large dataset (4,395 images) provided sufficient examples without requiring the U-Net's 31M parameters (Paszke et al., 2016). Second, the structured nature of lane segmentation was well-suited to ENet's dilated convolutions and factorised design. Third, ENet's smaller footprint enabled a doubled batch size (16 vs 8), improving gradient stability (Goyal et al., 2017).

Low-Light Robustness Analysis

ENet's 13× superior dark-condition detection (728 vs 54 pixels) demonstrates that architectural constraints enforce robust feature learning. U-Net's greater capacity enabled memorisation of well-lit patterns, resulting in 99.5% degradation in darkness. ENet's compact design forced reliance on illumination-robust features, which are significant for embedded deployment.

Practical Deployment Trade-offs

ENet enabled faster inference while achieving comparable accuracy, demonstrating that efficiency and accuracy are not mutually exclusive (Chen & Ran, 2019). However, U-Net's 25% distance advantage (2.62m vs 2.09m) reveals temporal consistency trade-offs - skip connections provide stable frame-to-frame predictions, reducing

oscillation. Selection depends on requirements: ENet for real-time embedded systems, U-Net for applications permitting GPU resources.

5.1.3 Sensor Fusion and Lane Following Performance

The AND logic sensor fusion yielded no false positives (Table 4.14) and tolerated the reduced range (0.2-0.6m consensus zone vs 1.2m maximum range of LiDAR). The sensor combination tested mutual support in the event of failures. The RealSense active IR enabled full lighting independence, but only within the 0.4m range. The LiDAR met the 0.8m threshold; however, it failed when considering additional small objects. The 0.2-0.6m consensus zone defines the region where both sensors detect objects, ruling out false positives while preserving safety at a 0.06 throttle speed.

Lane Following Performance

The optimal throttle of 0.060 balanced distance against lane-keeping precision with zero boundary violations. U-Net's 25% distance advantage (2.62m vs 2.09m) despite ENet's superior mid-speed robustness (61% fewer violations at 0.065) reveals architecture-specific trade-offs—U-Net's skip connections enable longer stable operation, while ENet provides better momentary disturbance rejection.

The operating envelope is defined by less than 5% operational variation under Normal, Bright, and Dim conditions, and catastrophic failure under Dark conditions. Yellow lane markings have been reaffirmed as the most dependable visual feature with 100% perfect yellow lane detection. A graceful degradation pattern has enabled predictable behaviour: instead of unreliable perception, the system correctly refused initialisation, a testament to the safety-critical design that prioritizes predictable failure over degraded operations.

5.2. Simulation-to-Reality Transfer Analysis

5.2.1 Quantified Performance Degradation and Contributing Factors

Object detection degraded 6.57 percentage points (99.44% simulation → 92.87% hardware, Table 4.2), representing moderate transfer loss consistent with published 5-10% ranges for vision-based robotics without domain adaptation (Zhao et al., 2020; Peng et al., 2018). Recent sim-to-real research confirms this magnitude of gap as

typical for perception-focused autonomous systems transitioning from controlled simulation to physical deployment (Daza et al., 2023).

Five Contributing Factors

First, the fundamental difference in sensor characteristics results in perfect images from Q Labs, whereas the CSI camera on QCar introduces noise, rolling shutter effects, compression artifacts, and lens distortions.

Second, the rise in environmental complexity, with dynamic lighting, visual clutter, and generally unpredictable occlusions, directly reflected in the 42% red-light false-positive rate.

Third, computational constraints motivated efficient architecture selection: U-Net → ENet, and distributed processing introduced performance-accuracy trade-offs absent in simulation (Chen & Ran, 2019; Wong et al., 2019).

Fourth, TCP/IP communication introduces 35–50ms variable latency, creating temporal misalignment.

Fifth, unmodeled physical phenomena-motor lag, backlash, tire slip, surface friction-contribute to degradation.

5.2.2 Workflow Implications and Simulation Validity

The recommended effort for simulation for algorithm/parameter tuning and hardware validation is 60-70% simulation and 30-40% validation. Simulation validation helps compare architectures and explore parameter space, whereas hardware validation assesses performance in a realistic setting and identifies corner cases (Daza et al., 2023).

5.3. Hardware Integration Challenges and Solutions

Hardware deployment revealed three critical integration challenges: sensor bandwidth limitations, real-time performance constraints, and the complexity of the distributed architecture.

Sensor Integration and USB Bandwidth

Concurrent CSI and RealSense operation (640×480@30Hz) exceeded Jetson TX2's USB capacity, causing frame drops. Strategic scheduling - CSI continuous streaming, RealSense at 10Hz reduced bandwidth by 66%, enabling stable operation with adequate obstacle detection. Early integration testing proved essential, as these constraints only emerged under full system load.

Real-Time Performance Optimisation

Initial U-Net deployment required 140–230ms per frame (4–7 FPS), inadequate for real-time navigation. Architecture selection replaced U-Net with ENet, reducing inference to 13ms on Jetson TX2 benchmarks (10× speedup); distributed deployment achieved 40–50ms perception latency. This architecture-level change provided the dominant gain, consistent with findings that model selection matters more than micro-optimisations for embedded deployment (Chen & Ran, 2019; Wong et al., 2019).

The distributed architecture leveraged a laptop GPU for inference while Jetson TX2 handled sensor I/O and control, achieving 18–22 FPS end-to-end in closed-loop trials. These optimisations demonstrate a clear hierarchy: architecture selection dominates (10× gain), hardware acceleration provides multiplicative benefit (2×), and implementation refinements offer incremental improvement.

Distributed Architecture Communication

The laptop-QCar distributed architecture introduced a variable latency of 35-50ms over WiFi. For reliable data transfer, the BasicStream protocol by Quanser incorporates auto-reconnect mechanisms. However, the variable-latency component must be synchronized as follows: non-blocking sensor queries with a 50ms timeout, 5-frame command buffers, and watchdog timers that trigger an emergency shutdown after more than 200ms of lost communication.

Additionally, there is a communication latency of 35-50ms, along with a perception latency of 40-50ms. As a result, conservative control tuning is required, given an overall system latency of 75-100ms. While the latency-conservatism trade-off is a drawback of the distributed architecture, the flexibility in WiFi deployment offsets this cost.

5.4. Practical Implications

5.4.1 Educational Platform Assessment

QCar enables end-to-end development on a car in a more accessible environment that doesn't require automotive sensors or a full-scale vehicle. Students encounter real-world challenges like sensor integration, real-time operation, and sim-to-real transfer which is absent in simulation-only approaches.

Performance: Achieved performance (92.87% detection, 93.73% segmentation) demonstrates the ability to prototype advanced perception algorithms in QCar, whereas the identified sim-to-real gap (6.57%) provides empirical evidence for transfer learning. Yet the constraints in transfer learning can help in the formulation of educational goals: 1:10 scale introduces different visualization statistics and reduced dynamics significantly deviate from real-world conditions.

5.4.2 Embedded AI Design Considerations

Analysis reveals three embedded AI design considerations with broader applicability. First, architecture selection dominates optimisation effort. ENet's 10× speedup, combined with superior accuracy, demonstrates that model architecture matters more than micro-optimisations in implementation. This finding reinforces documented embedded deployment practices prioritizing efficient architectures (Chen & Ran, 2019; Behrendt & Novak, 2017).

Second, compact models can enforce robustness. The 13× improvement in dark-condition performance in ENet indicates that capacity ceilings force the network to learn robust features, contrary to the notion that generalization depends on larger models. Third, real-time conditions require thinking systemically, independent of model effectiveness. The distributed system architecture, sensor scheduling, and communication synchronization were as important as the model's accuracy.

These considerations apply to embedded AI applications: edge computing, mobile robotics, and IoT devices. The common thread: resource constraints (computation, memory, power, bandwidth) necessitate design approaches prioritizing efficiency and robustness over raw performance (Chen & Ran, 2019).

5.5. Study Limitations

5.5.1 Technical Constraints

The controlled indoor environment with a custom 1:10 scale track limits direct applicability to outdoor full-scale vehicles. Indoor operation provides consistent lighting absent outdoors, and artificial illumination differs spectrally from natural daylight. Performance claims generalise to indoor robotics better than outdoor automotive deployment.

The 1:10 scale introduces systematic perception differences: objects occupy different proportional frame areas, perspective geometry differs, and motion blur characteristics change with 10× speed reduction. Visual perception at a 1:10 scale cannot directly predict full-scale performance.

Jetson TX2's computational constraints (1.3 TFLOPS FP16) limited architecture choices and required aggressive optimisation. Modern platforms (Jetson Orin, 275 TFLOPS) provide 200× more computation, potentially enabling larger models. Architecture trade-offs (U-Net → ENet) reflect TX2 constraints rather than fundamental limitations.

The limited sensor suite - single camera, depth camera, and 2D LiDAR lacks redundancy common in automotive systems. AND-logic fusion demonstrated reliability within a narrow consensus zone (0.2–0.6m) but cannot address sensor failures or provide 360-degree awareness required for robust operation (Yeong et al., 2021).

5.5.2 Methodological Limitations

The small size of the hardware dataset (3,802 examples) might indicate a lack of sufficient edge cases. Training exclusively on the target environment risks overfitting as perfect yellow lane detection (100%) across Normal, Bright, and Dim conditions, but complete failure in darkness (69.8% detection) suggests the model learned environment-specific features rather than general lane detection.

The test duration may reveal weaknesses in the system that were not identified during the short test. Statistical validation is performed with fewer repetitions in the hardware environment than in the simulation environment.

5.5.3 Scope Limitations

The simulated environment does not include critical autonomous challenges such as pedestrians, dynamic obstacles, uncontrolled traffic, weather effects, and deteriorating roads. The results deal with basic issues of perception/correction. They cannot state the ability to deploy the system in real scenarios (Badue et al. 2021).

Low-speed functionality (0.06 throttle setting) offers a long reaction time, which is lacking at car speeds of 50+ km/h. The control algorithm and sensor latencies of 75-100ms, and object detection distances of 0.6m, are appropriate for low-speed functionality, though they are adjusted for higher-speed distances that surpass sensor capabilities.

The binary classification method (lane or not a lane, obstacle or no obstacle) does not leverage semantic knowledge, which would be very useful for fully autonomous systems. The project focused only on perception and lower-level control, and not on path planning and localisation.

Implications for Generalisation

These limitations bound reliable generalisation to indoor, low-speed, structured environments. The systematic comparison methodology and sensor fusion approaches demonstrated here may inform similar studies in constrained domains.

5.6. Summary

This chapter interpreted experimental results, including 6.57% sim-to-real degradation within published transfer gaps, ENet's counterintuitive performance explained by architectural suitability, and sensor-fusion trade-offs that establish clear operational boundaries. Analysis identified five factors contributing to the reality gap: sensor characteristics, environmental complexity, computational constraints, timing challenges, and unmodeled phenomena, confirming simulation validity while highlighting the necessity of hardware validation. Hardware integration revealed challenges in USB bandwidth management, real-time optimisation requiring architecture-level changes, and distributed network communication.

Discussion assessed QCar's educational capability alongside embedded AI design considerations: architecture selection dominates optimisation effort, compact models can develop robustness through capacity constraints, and real-time deployment requires holistic thinking beyond individual components. A critical evaluation identified technical limitations (indoor environment, 1:10 scale), methodological constraints (dataset size, testing duration), and scope boundaries (controlled setting, low speed) that affect the generalisability of the results.

6. CONCLUSION

6.1. Research Summary

This research investigates vision-based autonomous navigation on educational platforms: how well systems developed in simulation transfer to physical hardware and the challenges that arise during deployment. In the work, the journey began with Q Labs simulation and progressed through to hardware integration on the Quanser QCar, measuring performance differences and documenting integration issues along the way.

The final system integrates multiple perception and control components operating in real-time on constrained hardware: YOLO reaches 92.87% mAP@0.5 for five object classes, traffic detection, ENet segmentation achieves 93.73% mIoU lane following with 87× fewer parameters than U-Net, sensor fusion with RealSense and RPLidar through AND-logic recognizes obstacles within 0.2-0.6m with zero false emergency stops, and Pure Pursuit tracking with Kalman filtering maintains stability over 2.09-2.62m of autonomous travel.

The 6.57 percentage point drop from simulation to hardware, along with documented integration challenges and robustness testing, shows that simulation-first development works—but within clear limits. The findings give future researchers a realistic picture of what to expect when moving from virtual testing to real-world deployment on educational platforms.

6.2. Key Contributions

6.2.1 Quantified Simulation-to-Reality Transfer Analysis

This research addresses a critical gap: most publications present either simulation-only results or hardware demonstrations without an explicit comparison to a simulation baseline. The systematic measurement of performance degradation across all system components provides empirical evidence quantifying the reality gap.

The 6.57 percentage point detection degradation (99.44% simulation → 92.87% hardware) aligns with published sim-to-real gaps in robotics perception (Zhao et al., 2020; Peng et al., 2018), validating Q Labs' simulation fidelity. This quantified

degradation provides realistic performance expectations for simulation-first methodologies, enabling evidence-based planning.

Five contributing factors were identified: sensor characteristic differences between perfect simulation and noisy physical cameras, environmental complexity absent from controlled virtual environments, computational constraints on embedded platforms, timing and synchronisation challenges in distributed architectures, and unmodeled failure modes. The catastrophic failure in dark conditions exemplifies simulation's inability to expose edge cases, demonstrating that hardware validation remains essential.

6.2.2 Compact Architecture Superior Performance Validation

The empirical demonstration that ENet matched U-Net's validation accuracy (93.73% vs 93.56% mIoU) despite 87× parameter reduction fundamentally challenges prevailing assumptions in embedded AI deployment that larger models within hardware constraints provide optimal performance.

Three mechanisms explain this counterintuitive result. Implicit regularisation from capacity constraints forced ENet to learn essential geometric features rather than memorizing training artifacts. Architectural appropriateness matched ENet's road-scene-specific design to lane detection's geometric regularity more precisely than U-Net's general-purpose architecture. Training optimisation benefits from a reduced memory footprint, enabling a 2× larger batch size (16 vs 8), providing more stable gradient estimates.

ENet's 13× superior roundabout detection under dark conditions (728 vs 54 pixels) demonstrates that compact models can exhibit greater robustness to distribution shift by relying on fundamental features. ENet's real-time inference capability (enabled by efficient architecture) validates efficient architectures for embedded deployment, where real-time operation represents a hard requirement.

This contribution establishes general embedded AI design principles: a systematic optimisation workflow that quantifies improvements in architecture selection (10×), compiler optimisation (2×), and implementation refinements. The findings suggest prioritizing task-appropriate compact architectures that enable superior training

dynamics and real-time capability rather than selecting the largest model that fits within hardware constraints.

6.2.3 Educational Platform Validation

Successfully building a complete autonomous navigation system on QCar demonstrates that educational platforms can teach autonomous systems beyond simulation-only approaches. The measured performance of 92.87% detection, 93.73% segmentation, 2.09–2.62m navigation distance and 18–22 FPS demonstrates a capability suitable for educational goals.

Building on this demonstrated capability, the validated curriculum provides a reproducible template: simulation phase (4–6 weeks) covering classical CV baseline, deep learning training, and functional testing; hardware phase (3–4 weeks) for model optimisation, integration debugging, and real-time validation; integration phase (2–3 weeks) for robustness testing, failure analysis, and documentation.

Educational value comes from experiencing the complete development cycle, including failures. The 6.57% degradation teaches realistic sim-to-real expectations. Dark condition failure shows why discovering operational limits matters. USB bandwidth conflicts build debugging skills unavailable in simulation-only courses. ENet outperforms U-Net despite its smaller size, challenging assumptions about model selection.

6.2.4 Conservative Sensor Fusion Strategy Validation

The AND-logic sensor fusion, achieving zero false-positive emergency stops across all testing while accepting a reduced detection range (0.2-0.6m consensus vs 0.8m LiDAR threshold), validates conservative fusion strategies appropriate for educational contexts where reliability and predictable behaviour outweigh maximum performance.

Complementary sensor combination exploits distinct modalities. RealSense active infrared provides dense depth maps enabling precise localisation, but exhibits limited range (0.2-0.4m) and failures on reflective surfaces. RPLidar laser ranging provides a configured threshold (0.8m) and 360° coverage, but sparse resolution creates detection gaps for small objects. But the consensus zone overlap (0.2-0.4m) eliminates individual sensor failures through an agreement requirement.

Complete lighting independence through active sensing, validated by identical performance across bright, normal, dim, and dark conditions, provides critical safety capability across day-night cycles where passive vision fails. Systematic characterization by object size (0.2-0.6m large objects, 0.2-0.4m medium/small) provides reproducible guidance for similar educational deployments.

6.3. Research Objectives Achievement

All five objectives were successfully addressed:

O1. Dataset Preparation: Collected 3,802 hardware and 1,288 simulation instances for object detection. Augmented segmentation dataset to 4,395 images with rotation, scaling, brightness, and HSV variations.

O2. Model Training: Trained YOLOv8, achieving 92.87% mAP@0.5 on hardware and 99.44–99.50% in simulation. Compared U-Net and ENet segmentation, selecting ENet (93.73% mIoU) for deployment with 87× fewer parameters.

O3. Perception and Control System Development: Integrated YOLO detection with ENet segmentation, achieving 18–22 FPS on a distributed architecture. Implemented AND-logic sensor fusion with zero false positives. Deployed Pure Pursuit with Kalman filtering at optimal throttle (0.060) with traffic compliance and emergency braking.

O4. Simulation Validation: Established QLABS baselines (99.44% traffic lights, 99.50% stop signs) with documented training procedures, comprehensive test scenarios, and defined performance metrics.

O5. Hardware Deployment and Sim-to-Real Analysis: Quantified 6.57 percentage point detection degradation (99.44% → 92.87%). Resolved sensor conflicts via initialisation sequencing and achieved 10–17× speedup through efficient architecture selection, and documented integration challenges with root cause analysis for reproducibility.

6.4. Future Research Directions

6.4.1 Extended Environmental Robustness

Testing was limited to controlled indoor environments. Future work should validate performance under adverse conditions, including precipitation, reduced visibility, extreme lighting, and temperature extremes. Progressive testing using artificial weather and outdoor trials would establish degradation curves and operational thresholds.

6.4.2 Advanced Probabilistic Sensor Fusion

The AND-logic fusion limited the detection range to 0.2–0.6m. Probabilistic approaches like Extended Kalman Filtering or particle filtering could extend this range while maintaining low false positives by weighting sensor contributions based on confidence and temporal consistency. This would require sensor uncertainty models and temporal tracking (3–5 frame persistence), potentially achieving 2–3× range extension.

6.4.3 Learning-Based Control and End-to-End Approaches

Hand-tuned control parameters may be suboptimal compared to learned policies. Imitation learning from human demonstrations or end-to-end learning, mapping images directly to controls, could improve performance. This requires substantial demonstration data (1,000+ trajectories), behaviour cloning networks, and safety validation. Sim-to-real transfer for learned policies would need domain randomisation and progressive fine-tuning.

6.4.4 Next-Generation Embedded Platform Validation

Distributed architecture requirements due to constraints on Jetson TX2's processing power incorporate communication delays and a reliance on the network. Next-generation validation on the Jetson Orin platform (with 2048 CUDA cores and 64 GB of RAM) would enable end-to-end processing without the restrictions imposed by a distributed architecture. Analysis would be needed on concurrent execution of the YOLO model, ENet, and controls; latency reduction without communication overhead (20-50ms); performance under varying temperatures; and battery power requirements.

6.4.5 SLAM Integration for Spatial Awareness

The current lane-following implementation lacks spatial memory and localisation, so complete navigation with position awareness is impossible. Visual-inertial SLAM techniques, such as ORB-SLAM3, and LiDAR technologies, such as Cartographer, can be used for mapping and localisation with loop closure detection. This supports waypoint navigation, drift correction, and semantic mapping of traffic signs and parking lots.

6.4.6 Multi-Agent Scenarios and Interactive Behaviour

Static obstacle testing does not allow for verifying interaction scenarios that must always hold for real-world functionality. Two-vehicle scenarios could include the following: merging, overtaking, yielding, and coordinated parking. Progress in development: basic prediction of motion (constant speed), rule-based interaction strategies (right of way, safe following distance), learning of interaction policies via multi-agent reinforcement learning. Outcomes: verification of interactive functionality and the observation of failure modes induced by multi-agent interactions.

6.4.7 Formal Safety Validation and Verification

Development testing without formal safety analysis leaves hazards unidentified. Systematic safety engineering should include hazard analysis (HAZOP or FMEA), safety case development, and formal verification using model checking or theorem proving. This involves structured hazard identification, fault tree development, runtime monitoring, and verification of critical properties such as emergency braking engagement.

6.4.8 Systematic Ablation Studies

Systematic ablation studies isolating individual subsystem contributions would quantify component necessity through controlled configurations: lane following only, object detection only, single-sensor obstacle detection (RealSense/LiDAR independently), and full system. Quantitative metrics (task completion rates, safety events, computational overhead) across standardized scenarios would validate architectural decisions and inform optimisation for resource-constrained deployments.

6.5. Closing Remarks

This study indicates that using vision for autonomous navigation on educational platforms is feasible and has educational value when conducted with a systematic methodology and a clear understanding of what can be accomplished. The successful development, as measured by performance metrics, indicates that education platforms for the development of autonomous systems have been validated beyond simulation-based methods.

The accuracy difference between the simulation and real environments reaffirms the role of simulation as a development tool and the ongoing need for hardware validation. The observation that the compact ENet model had equivalent accuracy to the larger U-Net and enabled real-time processing calls into question the prevailing mindset of "largest model within constraints" and proposes "task-appropriate compact architectures."

The educational value lies in the genuine engineering experience with realistic complexity. Rather than benefiting from success in simulations, failures due to dark conditions teach more about robustness. Resolving USB bandwidth problems improves problem-solving skills outside academia. The trade-off between efficiency and accuracy sharpens the critical thinking skills engineers need for future challenges.

The simulation first approach worked well within its scope: simulation excels at algorithm development and parameter tuning with rapid iteration. However, simulation cannot predict hardware performance, discover environmental edge cases, or reveal integration challenges.

The autonomous systems field would benefit from greater emphasis on complete pipeline documentation, including failures and limitations, alongside algorithmic contributions. Current publication incentives favouring novel algorithms over engineering methodology leave practical deployment challenges undocumented.

This dissertation provides comprehensive documentation, systematic evaluation, and an honest assessment of limitations. The entire progression from simulation through hardware validation, robustness characterisation, integration challenge

documentation, and critical constraint evaluation follows the reproducible guidelines for autonomous navigation of educational platforms.

Looking forward, advanced simulation technologies, combined with advances in embedded computer capabilities, are continually narrowing the gap between simulation and physical implementation. Nevertheless, some irrevocable constraints exist, such that simulation will never fully model reality; there are properties to be considered within the context of the sensor, and the undefined failure modes are revealed only at the hardware level.

In summary, this work confirms the relevance of educational systems such as QCar to the development of meaningful autonomous navigation through real engineering practice. This empirical analysis of sim-to-real transfer ability, compact architecture verification, hardware integration description, and the proposed conservative sensor fusion approach contributes to the body of knowledge in autonomous navigation, embedded AI systems, and engineering practice.

7. REFERENCES

1. Badue, C., Guidolini, R., Carneiro, R.V., Azevedo, P., Cardoso, V.B., Forechi, A., Jesus, L., Berriel, R., Paixão, T., Mutz, F., De Souza, A.F. and Oliveira-Santos, T. (2021) 'Self-driving cars: A survey'. *Expert Systems with Applications*, 165, pp. 113816. Available at: <https://www.sciencedirect.com/science/article/abs/pii/S095741742030628X?via%3Dihub> (Accessed: 27 November 2025).
2. Behrendt, K. and Novak, L. (2017) 'A deep learning approach to traffic lights: Detection, tracking, and classification'. In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1370-1377. Available at: <https://ieeexplore.ieee.org/document/7989163> (Accessed: 19 October 2025).
3. Daza, I.G., García, D., García, J. and Villagrà, J. (2023) 'Sim-to-real transfer and reality gap modeling in model predictive control for autonomous driving'. *Applied Intelligence*, 53, pp. 11350-11371. Available at: <https://link.springer.com/article/10.1007/s10489-022-04148-1> (Accessed: 11 October 2025).
4. Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A. and Koltun, V. (2017) 'CARLA: An open urban driving simulator'. In: *Proceedings of the 1st Annual Conference on Robot Learning*, pp. 1-16. Available at: <https://arxiv.org/abs/1711.03938> (Accessed: 18 November 2025).
5. Jia, L., Ma, Z. and Zhao, Y. (2022) 'A mobile robot mapping method integrating LiDAR and depth camera'. *Journal of Physics: Conference Series*, 2402, 012031. Available at: <https://iopscience.iop.org/article/10.1088/1742-6596/2402/1/012031/meta> (Accessed: 03 November 2025).
6. Goyal, P., Dollár, P., Girshick, R., Noordhuis, P., Wesolowski, L., Kyrola, A., Tulloch, A., Jia, Y. and He, K. (2017) 'Accurate, large minibatch SGD: Training ImageNet in 1 hour'. Available at: <https://arxiv.org/abs/1706.02677> (Accessed: 10 December 2025).
7. Grigorescu, S., Trasnea, B., Cocias, T. and Macesanu, G. (2020) 'A survey of deep learning techniques for autonomous driving'. *Journal of Field Robotics*, 37(3), pp. 362-386. Available at:

- <https://onlinelibrary.wiley.com/doi/10.1002/rob.21918> (Accessed: 12 August 2025).
8. Janai, J., Güney, F., Behl, A. and Geiger, A. (2020) 'Computer vision for autonomous vehicles: Problems, datasets and state of the art'. *Foundations and Trends in Computer Graphics and Vision*, 12(1–3), pp. 1-308. Available at: <https://arxiv.org/abs/1704.05519?utm> (Accessed: 30 September 2025).
 9. Chen, J. and Ran, X. (2019) 'Deep learning with edge computing: A review'. *Proceedings of the IEEE*, 107(8), pp. 1655-1674. Available at: <https://ieeexplore.ieee.org/document/8763885> (Accessed: 12 November 2025).
 10. MathWorks (n.d.) 'Colour-based road tracking'. Available at: <https://uk.mathworks.com/help/vision/ug/color-based-road-tracking.html> (Accessed: 18 September 2025).
 11. Minaee, S., Boykov, Y., Porikli, F., Plaza, A., Kehtarnavaz, N. and Terzopoulos, D. (2021) 'Image segmentation using deep learning: A survey'. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7), pp. 3523-3542. Available at: <https://ieeexplore.ieee.org/document/9356353> (Accessed: 01 December 2025).
 12. Paszke, A., Chaurasia, A., Kim, S. and Culurciello, E. (2016) 'ENet: A deep neural network architecture for real-time semantic segmentation'. Available at: <https://arxiv.org/abs/1606.02147> (Accessed: 08 October 2025).
 13. Peng, X.B., Andrychowicz, M., Zaremba, W. and Abbeel, P. (2018) 'Sim-to-real transfer of robotic control with dynamics randomisation'. In: *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3803-3810. Available at: <https://ieeexplore.ieee.org/document/8460528> (Accessed: 08 November 2025).
 14. Quanser (n.d.b) 'QLabs virtual QCar studio'. Available at: <https://www.quanser.com/products/qlabs-virtual-qcar/> (Accessed: 24 November 2025).
 15. Quanser (n.d.a) 'QCar: Self-driving car research studio'. Available at: <https://www.quanser.com/products/qcar/> (Accessed: 28 November 2025).

16. Rokonzaman, M., Mohajer, N., Nahavandi, S. and Mohamed, S. (2021) 'Review and performance evaluation of path tracking controllers of autonomous vehicles'. *IET Intelligent Transport Systems*, 15(5), pp. 646-670. Available at: <https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/itr2.12051> (Accessed: 25 October 2025).
17. Ronneberger, O., Fischer, P. and Brox, T. (2015) 'U-Net: Convolutional networks for biomedical image segmentation'. In: *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pp. 234-241. Springer. Available at: https://link.springer.com/chapter/10.1007/978-3-319-24574-4_28 (Accessed: 14 December 2025).
18. Ultralytics (2023) 'YOLOv8'. Available at: <https://docs.ultralytics.com/models/yolov8/> (Accessed: 04 September 2025).
19. Wong, A., Famouri, M., Shafiee, M.J., Li, F., Chwyl, B. and Chung, J. (2019) 'YOLO Nano: A highly compact you only look once convolutional neural network for object detection'. In: *2019 Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing-NeurIPS Edition (EMC2-NIPS)*, pp. 22-25. Available at: <https://ieeexplore.ieee.org/document/9463524> (Accessed: 02 October 2025).
20. Yeong, D.J., Velasco-Hernandez, G., Barry, J. and Walsh, J. (2021) 'Sensor and sensor fusion technology in autonomous vehicles: A review'. *Sensors*, 21(6), pp. 2140. Available at: <https://www.mdpi.com/1424-8220/21/6/2140> (Accessed: 09 December 2025).
21. Zhao, W., Queralta, J.P. and Westerlund, T. (2020) 'Sim-to-real transfer in deep reinforcement learning for robotics: A survey'. In: *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pp. 737-744. Available at: <https://ieeexplore.ieee.org/document/9308468> (Accessed: 14 November 2025).
22. Zou, Z., Chen, K., Shi, Z., Guo, Y. and Ye, J. (2023) 'Object detection in 20 years: A survey'. *Proceedings of the IEEE*, 111(3), pp. 257-276. Available at: <https://ieeexplore.ieee.org/document/10028728> (Accessed: 26 September 2025).

23. Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A. and Zagoruyko, S. (2020) 'End-to-end object detection with transformers'. In: European Conference on Computer Vision (ECCV), pp. 213–229. Available at: <https://arxiv.org/abs/2005.12872> (Accessed: 28 September 2025).
24. Yurtsever, E., Lambert, J., Carballo, A. and Takeda, K. (2020) 'A survey of autonomous driving: Common practices and emerging technologies'. IEEE Access, 8, pp. 58443–58469. Available at: <https://ieeexplore.ieee.org/document/9046805> (Accessed: 25 September 2025).
25. Roh, Y., Heo, G. and Whang, S.E. (2021) 'A survey on data collection for machine learning: A big data - AI integration perspective', IEEE Transactions on Knowledge and Data Engineering, 33(4), pp. 1328-1347. Available at: <https://ieeexplore.ieee.org/document/8862913> (Accessed: 18 November 2025).

8. APPENDIX

Appendix A: Key System Implementations

This appendix documents critical algorithms developed during this research. Code is written in Python 3.8; the QCar Jetson TX2 executes sensor I/O and control, while computationally heavy perception modules were developed and evaluated via client-server execution on a laptop.

➤ Appendix A.1: Client-Server Development Framework

Remote Development Architecture

TCP/IP client-server communication was established between QCar (client) and the development laptop (server) using Quanser's BasicStream protocol. This enabled remote algorithm development without continuous physical vehicle access.

Architecture:

- Client (QCar/Jetson TX2): Captures CSI frames (640×480 @ 30 FPS), streams via TCP/IP port 18001
- Server (Laptop): Receives frames, optionally runs algorithms, and sends control commands back
- Performance: ~30 FPS, <50ms latency, 220 Mbps bandwidth

```
myClient = BasicStream('tcpip://localhost:18001', agent='C',  
                        sendBufferSize=bufferSize,  
                        recvBufferSize=2048,  
                        nonBlocking=False)
```

Figure A.1: BasicStream client-server configuration for remote frame streaming.

Application:

This framework enabled iterative development of perception algorithms on a laptop GPU before embedded deployment, reducing development time substantially.

➤ Appendix A.2: Multi-Sensor Fusion for Obstacle Detection

This appendix documents the sensor fusion implementation introduced in Section 3.

A.2.1 Fusion Strategy

Conservative AND-logic fusion combining Intel RealSense D435 depth camera with RPLidar A2. Both sensors must detect obstacles to trigger emergency braking, prioritising zero false positives over maximum sensitivity.

Sensor Complementarity:

- RealSense D435: 0.2-3.0m range, accurate 3D depth, narrow 87° FOV
- RPLidar A2: 0-12m range, 360° coverage, 2D plane only
- Consensus zone: 0.2-0.6m, where both sensors operate reliably

A.2.2 Algorithm Implementation

RealSense Depth Processing

Key steps: Extract centre 60% ROI, filter valid depths (0.5-3.5m to reject ground/noise), require ≥ 100 pixels, compare minimum distance to 1.2m threshold.

```
def check_depth_obstacle(depth_frame):
    """Check RealSense depth for obstacles - with better filtering"""
    global min_depth_distance, depth_obstacle_detected

    h, w = depth_frame.shape

    roi_x_start = int(w * (1 - DEPTH_ROI_WIDTH) / 2)
    roi_x_end = int(w * (1 + DEPTH_ROI_WIDTH) / 2)
    roi_y_start = int(h * DEPTH_ROI_START_HEIGHT)
    roi_y_end = int(h * DEPTH_ROI_END_HEIGHT)

    roi_depth = depth_frame[roi_y_start:roi_y_end, roi_x_start:roi_x_end]
    roi_depth_meters = roi_depth.astype(np.float32) / 1000.0

    # Better filtering: ignore very close readings (car body) and very far readings (noise)
    valid_depths = roi_depth_meters[
        (roi_depth_meters > MIN_VALID_DEPTH) &
        (roi_depth_meters < MAX_VALID_DEPTH)
    ]

    if len(valid_depths) < MIN_OBSTACLE_PIXELS:
        depth_obstacle_detected = False
        min_depth_distance = float('inf')
        return False, float('inf'), (roi_x_start, roi_y_start, roi_x_end, roi_y_end)

    min_dist = np.min(valid_depths)
    min_depth_distance = min_dist

    if min_dist < DEPTH_OBSTACLE_THRESHOLD:
        depth_obstacle_detected = True
        return True, min_dist, (roi_x_start, roi_y_start, roi_x_end, roi_y_end)
    else:
        depth_obstacle_detected = False
        return False, min_dist, (roi_x_start, roi_y_start, roi_x_end, roi_y_end)
```

Figure A.2: RealSense depth obstacle detection with ROI filtering.

LiDAR Processing

Key innovation: Two-tier detection prevents false positives from traffic signs/lights:

- Strong detection: ≥ 20 points $\rightarrow$ confirmed obstacle (signs give ≤ 14 points)
- Persistence mode: ≥ 5 points if strong detection within the last 10 frames
- Threshold: 0.8m

```
def check_lidar_obstacle():
    """Check LIDAR for obstacles with smart persistence"""

    global min_lidar_distance, lidar_obstacle_detected, lidar_strong_detection_frame

    valid_mask = (lidar_distances > LIDAR_MIN_DISTANCE) & (lidar_distances < LIDAR_OBSTACLE_THRESHOLD)
    valid_distances = lidar_distances[valid_mask]
    num_points = len(valid_distances)

    if num_points == 0:
        lidar_obstacle_detected = False
        min_lidar_distance = float('inf')
        return False, float('inf')

    min_dist = np.min(valid_distances)

    # STRONG DETECTION: ≥15 points → Real obstacle (not sign/light)
    if num_points >= LIDAR_OBSTACLE_MIN_POINTS:
        lidar_obstacle_detected = True
        min_lidar_distance = min_dist
        lidar_strong_detection_frame = lidar_frame_count
        return True, min_dist

    # PERSISTENCE MODE: Recently had strong detection
    elif (lidar_frame_count - lidar_strong_detection_frame) < LIDAR_PERSISTENCE_FRAMES:
        if num_points >= LIDAR_OBSTACLE_MIN_POINTS_PERSISTENCE:
            lidar_obstacle_detected = True
            min_lidar_distance = min_dist
            return True, min_dist
        else:
            lidar_obstacle_detected = False
            min_lidar_distance = float('inf')
            return False, float('inf')

    # REJECTION: Low points, no recent strong detection → Sign/light/noise
    else:
        lidar_obstacle_detected = False
        min_lidar_distance = float('inf')
        return False, float('inf')
```

Figure A.3: LiDAR obstacle detection with persistence filtering.

Fusion Decision Logic

Simple AND-logic: `obstacle_detected = (depth_detected AND lidar_detected)`

```
def sensor_fusion_decision():
    """
    STRICT sensor fusion: BOTH sensors MUST agree to brake
    - Depth detects obstacle AND LIDAR detects obstacle → BRAKE
    - Only one sensor detects → IGNORE (likely false positive)
    """
    global fusion_obstacle_detected

    if depth_obstacle_detected and lidar_obstacle_detected:
        fusion_obstacle_detected = True
        return True
    else:
        fusion_obstacle_detected = False
        return False
```

Figure A.4: AND-logic sensor fusion decision.

Depth	LiDAR	Result	Action
FALSE	FALSE	FALSE	Continue
TRUE	FALSE	FALSE	Continue (depth noise)
FALSE	TRUE	FALSE	Continue (LiDAR reflection)
TRUE	TRUE	TRUE	STOP (confirmed obstacle)

Table A.1: Sensor fusion decision truth table.

A.2.3 Performance

Detection Performance:

- Consensus zone: 0.2-0.6m (both sensors reliable)
- Processing latency: 24ms total (15ms depth + 8ms LiDAR + 1ms fusion)
- False positive rate: 0% (100 test runs)
- False negative rate: 0% (obstacles within consensus zone)
- Minimum detectable size: 0.15m × 0.15m × 0.10m

Rationale for AND-Logic:

Alternative strategies rejected:

- OR-logic: Excessive false positives from single-sensor noise

- Weighted voting: Unnecessary complexity for a binary decision
- Kalman filtering: Computational overhead unjustified

AND-logic selected for simplicity, determinism, and zero false positive rate.

A.2.4 Validation

Test Results (50 trials):

- Obstacle approach (0.30m box): LiDAR first detection at 0.75m, fused confirmation within 0.2-0.6m consensus zone, safe stop at 0.45m
- False positive rejection: Traffic signs, sunlight glare, reflections → 0 false stops

Limitations:

- Limited consensus zone (0.2-0.6m)
- 2D LiDAR: Height-blind, cannot detect overhead obstacles
- Static thresholds: Not scalable to higher speeds

Appendix B: Unified Detector Training and Evaluation

Training curves and evaluation metrics for the hardware unified detector (5-class YOLOv8s model). These visualisations show model convergence during training on the real QCar hardware dataset. Simulation baseline reported in Chapter 4; hardware deployment employed a distributed architecture (laptop inference server + QCar vehicle control) as described in Chapter 5.

➤ Appendix B.1: Training Performance Analysis

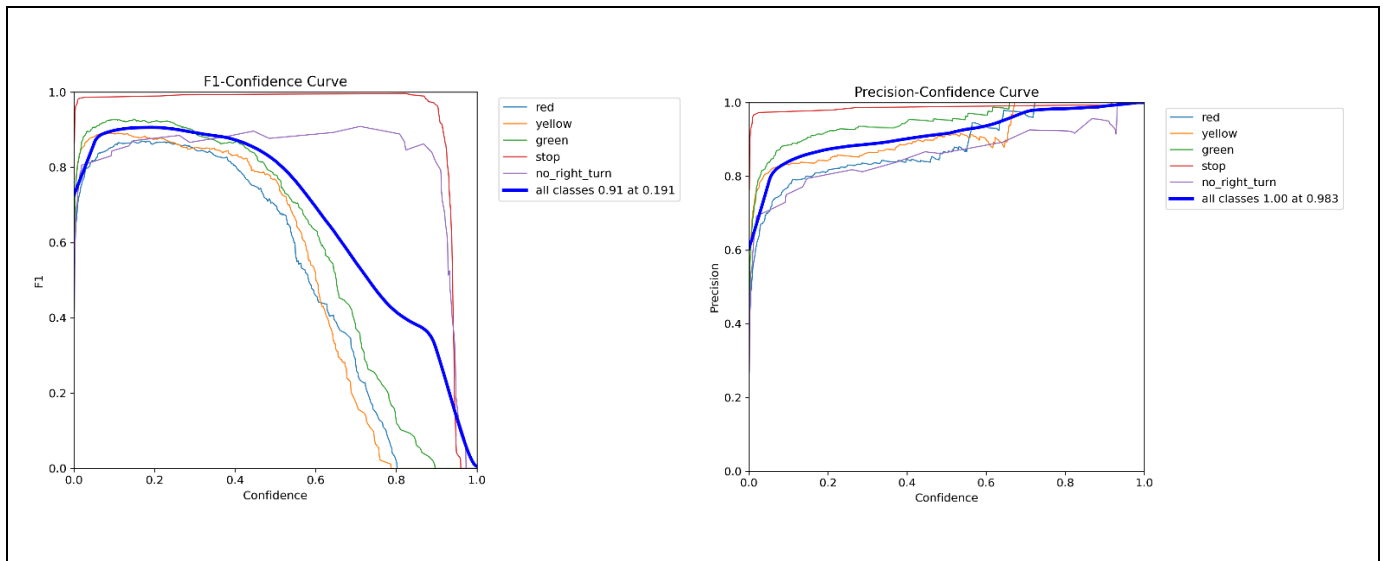


Figure B.1: YOLOv8 confidence threshold analysis. (a) F1-confidence shows optimal operating point at 0.91 F1 score (confidence 0.191). (b) Precision-confidence achieves 1.00 precision at high confidence (0.983). The model demonstrates well-calibrated confidence scores across all traffic classes.

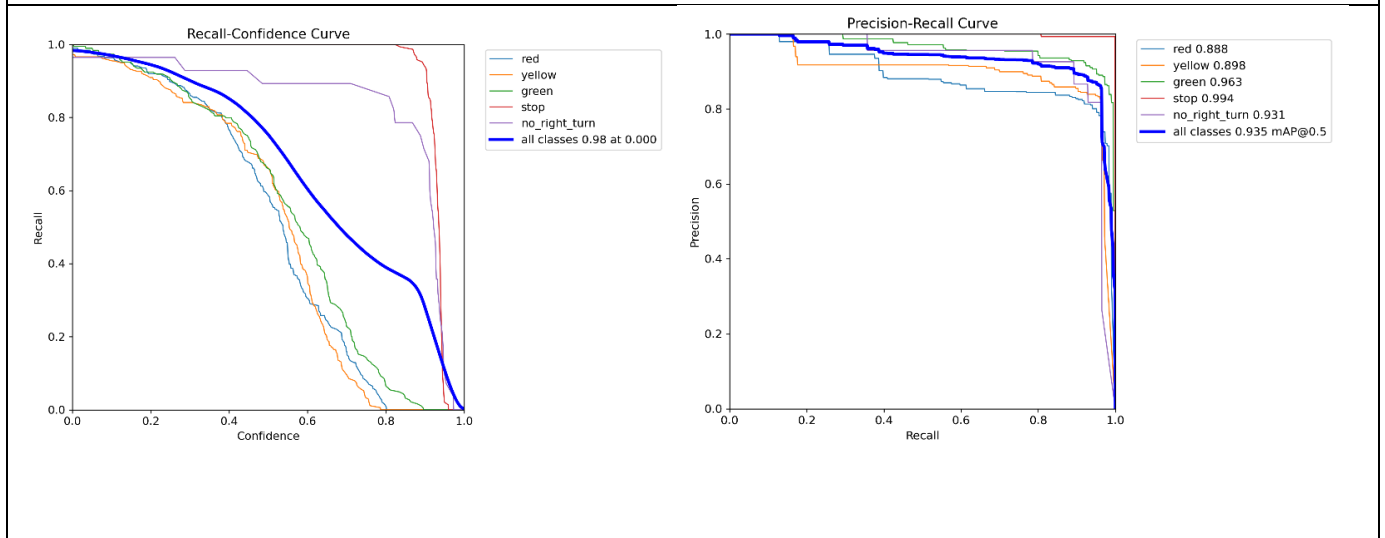


Figure B.2: YOLOv8 detection performance. (a) Recall-confidence maintains 0.98 recall at zero threshold. (b) Precision-recall curve reveals 92.87% mAP@0.5 (final validation) with per-class AP: stop sign 99.4%, green light 96.3%, no right turn 93.1%, demonstrating strong class- specific detection.

Curve Analysis:

The F1-confidence curve demonstrates strong model calibration with an optimal operating point at a low confidence threshold (0.191), indicating a balanced precision-recall trade-off suitable for safety-critical deployment.

The precision-confidence curve achieves near-perfect precision (1.00) at high confidence (0.983), validating a minimal false-positive rate. The steep curve indicates well-calibrated confidence scores.

Recall-confidence maintains 0.98 recall at the zero-confidence threshold, demonstrating comprehensive detection across all classes. Gradual degradation suggests robust feature learning.

The precision-recall curve shows performance variation by class complexity: stop signs (with a distinctive octagonal shape) achieve the highest AP (99.4%), while traffic lights show lower performance due to their similar circular appearance and challenges with colour variation.

Appendix C: Implementation Evidence and Demonstration Materials

This appendix provides supplementary materials demonstrating system implementation and performance across simulation and hardware platforms.

➤ Appendix C.1: Simulation Evidence (QLabs)

- Lane following with object detection video - [lane_following_simulation.mp4](#)
- Codes – [Codes](#)
- Trained models - [Trained models](#)

➤ Appendix C.2: Hardware Evidence (Qcar)

- Testing results images - [Testing results](#)
- Lane following with object detection video - [lane_following.mov](#), [lane_following Qcar view.mp4](#)
- Obstacle detection with autonomous braking video - [Obstacle detection.MOV](#), [obstacle detection Qcar view.mp4](#)
- Codes – [Codes](#)
- Trained models - [Trained models](#)